Java & Spring Boot Senior Developer Interview Preparation Guide

Comprehensive Interview Handbook for 8 Years of Experience

Java 8 to 21 • Spring Boot 2/3 • Microservices • SQL • System Design
Concepts • Code • Scenarios • Interview-ready answers

Table of Contents

How to Use This Book

The study method
The standard structure of a deep question
Honest notes about this book
What interviewers at 8 years evaluate
Suggested 6-week plan

Part 1 — Java Core and OOP

1.1 JDK vs JRE vs JVM and how Java runs
1.2 Primitive vs reference types, wrappers, autoboxing
1.3 Pass-by-value
1.4 == vs equals(), and the equals/hashCode contract
1.5 String immutability and the String pool
1.6 final, finally, finalize()
1.7 Static, initialization order, constructors
1.8 Abstract class vs interface
1.9 Overloading vs overriding; polymorphism
1.10 Composition vs inheritance; association, aggregation, composition
1.11 Shallow vs deep copy, cloning
1.12 Marker interfaces, annotations, enums, nested classes
1.13 Serialization, transient, volatile
1.14 Records and modern language features (Java 14–21)

Part 2 — OOP and Design Principles

2.1 The four pillars
2.2 SOLID with enterprise examples
2.3 Other principles
2.4 Spotting and fixing poor design

Part 3 — Collections Framework

3.1 The big picture
3.2 ArrayList vs LinkedList
3.3 HashSet, LinkedHashSet, TreeSet
3.4 HashMap internal working (most asked)
3.5 HashMap vs Hashtable vs ConcurrentHashMap vs TreeMap
3.6 Comparable vs Comparator
3.7 Iterators, fail-fast and ConcurrentModificationException
3.8 PriorityQueue, Queue, Deque
3.9 Choosing the right collection
3.10 Collections coding problems

Part 4 — Java 8 and Functional Programming

4.1 Lambdas and functional interfaces
4.2 Stream API
4.3 Optional
4.4 Parallel streams
4.5 Other Java 8 features
4.6 Java 8 coding questions
4.7 Java 9–21 feature timeline

Part 5 — Exception Handling

5.1 Checked vs unchecked
5.2 try-catch-finally and try-with-resources
5.3 Best practices
5.4 Global exception handling in Spring Boot

Part 6 — JVM, Memory and Garbage Collection

6.1 JVM architecture and memory areas
6.2 Garbage collection fundamentals
6.3 Reference types
6.4 Memory leaks in Java
6.5 Investigating memory problems

Part 7 — Multithreading and Concurrency

7.1 Basics
7.2 Race condition, synchronized and visibility
7.3 Atomic classes and CAS
7.4 ReentrantLock and ReadWriteLock
7.5 wait / notify
7.6 Deadlock, starvation, livelock
7.7 Thread pools
7.8 BlockingQueue and producer–consumer
7.9 ForkJoinPool
7.10 CompletableFuture
7.11 Virtual threads (Java 21)
7.12 Thread-safety strategies, ordered by preference
7.13 Classic concurrency coding problems
7.14 Debugging thread problems in production

Part 8 — Spring Core and Spring Boot

8.1 What is Spring, IoC and DI?
8.2 The container: BeanFactory vs ApplicationContext
8.3 Bean lifecycle (very common)
8.4 Stereotype annotations and configuration
8.5 AOP and proxies
8.6 Common Spring mistakes
8.7 Spring vs Spring Boot
8.8 Spring Boot internals

Part 9 — Spring MVC and REST APIs

9.1 DispatcherServlet and request flow
9.2 REST design principles
9.3 Complete CRUD example (Boot 3, Java 17)
9.4 Designing reliable APIs: checklist

Part 10 — Spring Data JPA, Hibernate and Transactions

10.1 JPA vs Hibernate vs Spring Data JPA
10.2 Entity lifecycle and the persistence context
10.3 Fetching, LazyInitializationException, N+1
10.4 Relationships
10.5 Queries
10.6 Locking
10.7 Other production topics
10.8 @Transactional in depth

Part 11 — Spring Security and Testing

11.1 Authentication vs authorization
11.2 Spring Security architecture
11.3 Passwords
11.4 Sessions vs tokens; JWT
11.5 OAuth 2.0 and OpenID Connect
11.6 CORS vs CSRF
11.7 A practical JWT flow (Spring Boot 3 / Spring Security 6)
11.8 Common vulnerabilities and secure practices
11.9 Spring Boot testing

Part 12 — Microservices Architecture

12.1 Monolith vs microservices
12.2 Communication
12.3 Infrastructure building blocks
12.4 Resilience patterns
12.5 Data consistency across services
12.6 Kafka fundamentals
12.7 Caching and Redis
12.8 Containers and Kubernetes basics
12.9 Realistic architecture walkthrough: Clinic Appointment Platform
12.10 Observability and production troubleshooting

Part 13 — SQL and Database Concepts

13.1 Fundamentals
13.2 Sample data used in the problems
13.3 Joins
13.4 GROUP BY, HAVING and aggregates
13.5 Classic problems
13.6 Indexes and execution plans
13.7 Locks, deadlocks, procedures, scale-out

Part 14 — Coding Interview Problems

14.1 Two Sum (Easy) — HashMap
14.2 Remove duplicates from a sorted array in place (Easy) — two pointers
14.3 Reverse a string and Palindrome (Easy)
14.4 Valid Anagram, First Non-Repeating Character, Character Frequency (Easy)
14.5 Find duplicates and Group Anagrams (Medium)
14.6 Longest Substring Without Repeating Characters (Medium) — sliding window
14.7 Merge Intervals (Medium) — sort
14.8 Binary Search (Easy)
14.9 Linked lists: Reverse and Detect Cycle (Easy)
14.10 Valid Parentheses (Easy) — stack
14.11 Merge Sorted Arrays (Easy)
14.12 Kth Largest and Top K Frequent (Medium) — heap
14.13 Tree traversals (Easy–Medium)
14.14 Find Missing Number and Maximum Subarray (Easy / Medium)
14.15 Recursion, graphs and dynamic programming (basics)
14.16 Modern Java alternatives (Java 17+)

Part 15 — Design Patterns and Clean Code

15.1 Singleton
15.2 Factory Method and Abstract Factory

15.3 Builder
15.4 Strategy
15.5 Observer
15.6 Decorator
15.7 Adapter and Facade
15.8 Proxy
15.9 Template Method
15.10 Chain of Responsibility
15.11 Repository, DI, MVC, Circuit Breaker
15.12 Clean code, refactoring and code reviews

Part 16 — System Design Interviews

16.1 A framework you can follow every time
16.2 Design: URL shortener
16.3 Design: E-commerce order processing
16.4 Design: Online appointment booking
16.5 Design: Payment processing
16.6 Design: Notification service
16.7 Design: Rate limiter
16.8 Design: File upload service
16.9 Design: Employee management system
16.10 Follow-up questions a senior interviewer may ask

Part 17 — DevOps, Cloud and Production Support

17.1 Build tools and Git
17.2 Docker
17.3 Kubernetes fundamentals
17.4 CI/CD
17.5 Deployment strategies
17.6 Logging, metrics, tracing
17.7 Cloud, secrets and TLS

Part 18 — Production Incident Playbooks

18.1 API suddenly slow
18.2 High CPU
18.3 OutOfMemoryError and memory leaks
18.4 Database connection pool exhaustion
18.5 Thread pool exhaustion
18.6 Excessive database queries
18.7 Kafka consumer lag
18.8 Downstream service unavailable
18.9 HTTP 500 errors after deployment
18.10 Deadlocks
18.11 Intermittent timeouts
18.12 "Works locally, fails in production"

Part 19 — Behavioral and Project-Based Questions

19.1 What interviewers evaluate
19.2 Questions, intent, and answer structure
19.3 Example: explaining an end-to-end project architecture (template)
19.4 Answer-quality checklist

Part 20 — 50+ Scenario-Based Questions

A. Concurrency and data consistency

B. Spring and JPA
C. API, security and design
D. Performance and production
E. Microservices and architecture
F. Extra practice

Part 21 — Frequently Asked Comparison Questions

21.1 Java core
21.2 Spring, REST and JPA
21.3 Architecture and tooling

Part 22 — Final Revision Checklist and Rapid-Fire Questions

22.1 One-page memory aids
22.2 Rapid-fire (answer each in one or two sentences)
22.3 Mock interview routine (last two weeks)
22.4 Final advice

How to Use This Book

This handbook is written for a developer with about **8 years of experience** preparing for Senior Java Developer / Senior Software Engineer / Spring Boot roles. At this level, interviewers do not only ask "*what is X?*". They ask **why** X exists, **how** it works inside, **what breaks** in production, and **what you would choose and why**.

The study method

1. **Read the simple answer first.** Say it out loud in your own words.
2. **Read "Understand it"** until you could draw the idea on a whiteboard.
3. **Type the code yourself.** Do not copy and paste. Break it on purpose and see the error.
4. **Read "Common mistakes" and "Follow-ups".** Senior interviews are won on follow-ups.
5. **Finish with the "Interview-ready answer"** and practise it in 30–90 seconds.

The standard structure of a deep question

Section	Purpose
Question	The exact words an interviewer may use
Simple answer	A short spoken answer
Understand it	Step-by-step explanation from the basics
Real-life example	An analogy from e-commerce, banking, hospital, booking
Code + explanation	Working Java / Spring Boot code and why each part exists
Internals	What the JVM / Spring / Hibernate does under the hood
When to use, pros, cons	Trade-offs, not hype
Common mistakes	Traps interviewers like
Follow-ups	What comes next
Interview-ready answer	Your final 30–90 second answer

Shorter topics use a compressed form of this structure. Comparison topics use tables.

Honest notes about this book

- **Version labels.** Features are labelled with the Java / Spring version that introduced them. Code marked *Java 8* uses only Java 8 syntax.
- **No invented numbers.** This book does not quote benchmarks or "interview frequency" statistics. Where performance is discussed, it is described as relative behaviour (for example "O(1) average"), which you can verify yourself.
- **Verify versions before the interview.** Spring Boot, Spring Security and Hibernate move fast. Boot 3.x requires Java 17+ and the `jakarta.*` namespace. Check the latest stable release notes for anything

newer than your preparation date; this book will tell you which parts are most likely to have changed.
- **Your own projects.** Behavioral chapters use placeholders such as `<YOUR PROJECT>`. Replace them with **your real experience**. Never claim work you did not do.

What interviewers at 8 years evaluate

- **Depth:** can you explain internals, not just APIs?
- **Judgement:** can you pick between options and defend trade-offs?
- **Production sense:** have you debugged slow queries, memory leaks, deadlocks, bad deployments?
- **Design ability:** can you break a vague requirement into services, data, APIs and failure handling?
- **Ownership and leadership:** mentoring, code review, incidents, disagreement, delivery.

Suggested 6-week plan

Week	Focus
1	Java core, OOP, collections, exceptions
2	Java 8+, concurrency, JVM and GC
3	Spring core, Boot, MVC/REST, JPA, transactions
4	Security, testing, microservices, Kafka
5	SQL, coding problems (a few daily), design patterns
6	System design, production scenarios, behavioral, mock interviews

Part 1 — Java Core and OOP

1.1 JDK vs JRE vs JVM and how Java runs

Interview question: *Explain JDK, JRE and JVM. How does a `.java` file become a running program?*

Simple answer: The **JVM** runs bytecode. The **JRE** is the JVM plus the standard libraries needed to run programs. The **JDK** is the JRE plus developer tools such as `javac`, `jar`, `jshell` and `jcmd`. Source code is compiled by `javac` into platform-neutral **bytecode** (`.class` files); the JVM loads that bytecode and executes it, interpreting it and compiling hot parts to machine code with the **JIT compiler**.

Understand it:

```
Hello.java --javac--> Hello.class (bytecode) --JVM--> machine code on Windows/Linux/Mac
                                    |
                          ClassLoader -> Verifier -> Interpreter + JIT -> Garbage Collector
```

1. `javac` checks syntax and types, then writes bytecode.
2. The **class loader** loads classes lazily, when first used.
3. The **bytecode verifier** checks the code is safe (no stack corruption, valid types).
4. The **interpreter** starts running immediately. The **JIT** (C1 and C2 in HotSpot) notices "hot" methods and compiles them to optimized native code.
5. The **GC** frees memory of unreachable objects.

Platform independence: your bytecode is portable; the *JVM* is platform specific. "Write once, run anywhere" means one bytecode, many JVMs.

Real-life example: Bytecode is like a sheet of music (same everywhere); the JVM is the musician who knows how to play it on a particular instrument (OS/CPU).

Since Java 11 there is no separate JRE download from Oracle; use the JDK, or build a small runtime with `jlink`.

Common mistakes: saying "Java is purely interpreted" (JIT compiles hot code) or "JVM is platform independent" (it is not; bytecode is).

Follow-ups: What is JIT tiered compilation? What is `jlink`? What are Java's limitations (startup time, memory footprint, no manual memory control, verbose compared with some languages)?

Interview-ready answer: "javac compiles source to bytecode. The JVM loads that bytecode, verifies it, and runs it by interpreting and JIT-compiling hot methods to native code. The JDK includes the compiler and tools, the JRE is just the runtime, and the JVM is the engine inside. Bytecode is portable; each OS has its own JVM."

1.2 Primitive vs reference types, wrappers, autoboxing

Interview question: *What are the differences between primitives and wrapper classes? What problems does autoboxing cause?*

Simple answer: Primitives (`int` , `long` , `double` , `boolean` , ...) hold raw values and are fast. Wrappers (`Integer` , `Long` , ...) are objects, can be `null` , and work with generics and collections. Autoboxing converts between them automatically, but it can create garbage, hide `NullPointerException` s, and break `==` comparisons.

```
Integer a = 127, b = 127;
System.out.println(a == b);       // true  (Integer cache -128..127)
Integer c = 1000, d = 1000;
System.out.println(c == d);       // false (different objects)
System.out.println(c.equals(d));  // true

Integer maybeNull = null;
int x = maybeNull;                // NullPointerException (unboxing)
```

Explanation: `Integer.valueOf()` returns cached objects for small values, so `==` accidentally works for 127 but not for 1000. Always use `equals()` for wrappers.

Performance trap: `Long sum = 0L; for (...) sum += i;` boxes a new `Long` on every iteration. Use `long` .

Interview-ready answer: "Primitives are values on the stack or inside objects; wrappers are objects needed for generics and nullability. Autoboxing is convenient but can cause hidden allocations, NPEs on unboxing null, and wrong `==` comparisons because of the Integer cache. I use primitives in hot loops and `equals()` on wrappers."

1.3 Pass-by-value

Interview question: *Is Java pass-by-value or pass-by-reference?*

Simple answer: Always **pass-by-value**. For objects, the *value of the reference* is copied, so both variables point to the same object.

```
static void change(StringBuilder sb, int n) {
    sb.append(" world");      // modifies the shared object -> visible to caller
    sb = new StringBuilder("new"); // only changes local copy of the reference
    n = 99;                   // local copy only
}
public static void main(String[] args) {
    StringBuilder s = new StringBuilder("hello");
    int n = 1;
    change(s, n);
    System.out.println(s + " " + n);  // hello world 1
}
```

Analogy: You give a friend a *copy of your house address*. They can paint your house (mutate the object), but if they cross out their copy and write a new address, your address is unchanged.

1.4 `==` vs `equals()` , and the `equals` / `hashCode` contract

Interview question: *What is the contract between `equals()` and `hashCode()` ? What breaks if you override only one?*

Simple answer: If two objects are `equals()`, they **must** have the same `hashCode()`. The reverse is not required. If you break this, hash-based collections (`HashMap`, `HashSet`) lose objects.

The contract of `equals`: reflexive, symmetric, transitive, consistent, and `x.equals(null)` is `false`.

```
public final class Money {
    private final String currency;
    private final long cents;
    public Money(String currency, long cents) { this.currency = currency; this.cents = cents; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Money)) return false;
        Money m = (Money) o;
        return cents == m.cents && currency.equals(m.currency);
    }
    @Override public int hashCode() { return java.util.Objects.hash(currency, cents); }
}
```

What goes wrong if only `equals` is overridden:

```
Set<Money> set = new HashSet<>();
set.add(new Money("USD", 100));
set.contains(new Money("USD", 100)); // false! different hash -> different bucket
```

Internals: `HashMap` first uses `hashCode()` to find the bucket, then `equals()` to find the key within it.

Common mistakes: using mutable fields in `hashCode()` and then changing them after inserting into a `HashSet` (the object becomes "lost"); using `getClass()` vs `instanceof` inconsistently; for JPA entities see Part 7.

Interview-ready answer: "Equal objects must return equal hash codes because HashMap uses hashCode to choose the bucket and equals to confirm the match. Override both together, base them on the same immutable fields, and prefer records or Lombok where appropriate."

1.5 String immutability and the String pool

Interview question: *Why is `String` immutable? What is the String Constant Pool?*

Simple answer: A `String` object can never change after creation. This makes it **thread-safe**, **cacheable** (hash code computed once), **safe as a HashMap key** and **safe for security-sensitive values** like class names and file paths. The String pool is a JVM-managed cache of string literals so identical literals share one object.

```
String a = "java";
String b = "java";
String c = new String("java");
System.out.println(a == b);          // true  (same pooled object)
System.out.println(a == c);          // false (new heap object)
System.out.println(a == c.intern()); // true
System.out.println(a.equals(c));     // true
```

Internals: Since Java 7 the pool lives on the heap (earlier in PermGen). `intern()` returns the pooled instance. Since Java 9, `String` is stored as `byte[]` (compact strings) when text is Latin-1, saving memory.

Security note: for passwords, prefer `char[]` because you can overwrite it; a `String` stays in memory until GC.

	`String`	`StringBuilder`	`StringBuffer`
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Speed for repeated edits	Slow (new object each time)	Fastest	Slower due to locking
Use	Constants, keys	Single-thread building	Rarely needed today

Common trap: `s1 + s2` in a loop creates many objects; use `StringBuilder`. A single-line `a + b + c` is fine — the compiler optimizes it (via `StringBuilder` or `invokedynamic` in Java 9+).

Creating your own immutable class: make the class `final`, fields `private final`, no setters, deep-copy mutable inputs in the constructor and defensive-copy on getters.

```
public final class Appointment {
    private final String id;
    private final List<String> attendees;
    public Appointment(String id, List<String> attendees) {
        this.id = id;
        this.attendees = Collections.unmodifiableList(new ArrayList<>(attendees)); // defensive copy
    }
    public String getId() { return id; }
    public List<String> getAttendees() { return attendees; }
}
```

Java 16+: `record Appointment(String id, List<String> attendees) {}` is shallowly immutable — you still must copy the list in a compact constructor.

1.6 `final`, `finally`, `finalize()`

- `final` — variable cannot be reassigned, method cannot be overridden, class cannot be extended.
- `finally` — block that runs after try/catch, used for cleanup (prefer try-with-resources). It will not run if the JVM exits (`System.exit`) or crashes.
- `finalize()` — deprecated (Java 9) and deprecated for removal (Java 18). Unpredictable and slow. Use try-with-resources or `Cleaner`.

Trap: `return` inside `finally` swallows exceptions and overrides returned values. Never do it.

1.7 Static, initialization order, constructors

Order of initialization when you create `new Child()`:

1. Parent static blocks / static fields (once, when the class loads)
2. Child static blocks
3. Parent instance fields and instance initializer blocks
4. Parent constructor
5. Child instance fields and blocks
6. Child constructor

Constructor chaining: `this(...)` calls another constructor of the same class; `super(...)` calls the parent. Either must be the first statement (until Java 22 preview relaxations).

Static methods belong to the class, are not polymorphic (they are *hidden*, not overridden) and cannot access instance state.

1.8 Abstract class vs interface

	Abstract class	Interface
State (instance fields)	Yes	No (only constants)
Constructors	Yes	No
Multiple inheritance	One class only	Many interfaces
Methods	abstract + concrete	abstract, `default`, `static`, `private` (Java 9)
Use for	Shared base state/behaviour ("is-a" with code reuse)	A capability/contract ("can do")

Rule of thumb: program to interfaces; use an abstract class when subclasses truly share state or a template algorithm.

Diamond problem with default methods: if two interfaces provide the same default method, the class must override it, and can choose with `A.super.hello()`.

1.9 Overloading vs overriding; polymorphism

- **Overloading** — same name, different parameters, resolved at **compile time** (static polymorphism).
- **Overriding** — subclass replaces a parent method, resolved at **runtime** by the actual object type (dynamic dispatch, virtual method table).

```
class Payment { void pay() { System.out.println("generic"); } }
class CardPayment extends Payment { @Override void pay() { System.out.println("card"); } }
Payment p = new CardPayment();
p.pay(); // card -> chosen at runtime from the object's real class
```

Overriding rules: same signature; return type may be a subtype (covariant); cannot reduce visibility; cannot throw broader checked exceptions; `private`, `static`, `final` methods cannot be overridden. Always use `@Override`.

1.10 Composition vs inheritance; association, aggregation, composition

Simple answer: Inheritance is **"is-a"**, composition is **"has-a"**. Prefer composition because inheritance couples the child to the parent's implementation (the "fragile base class" problem) and is fixed at compile time.

Relationship	Meaning	Lifetime	Example
Association	Uses	Independent	Doctor — Patient
Aggregation	Has, weak	Part can outlive whole	Department — Employee
Composition	Owns, strong	Part dies with whole	Order — OrderLine

```
// Composition + interface: behaviour is swappable
interface NotificationSender { void send(String to, String msg); }
class OrderService {
    private final NotificationSender sender;           // has-a
    OrderService(NotificationSender sender) { this.sender = sender; }
    void confirm(String email) { sender.send(email, "Order confirmed"); }
}
```

1.11 Shallow vs deep copy, cloning

- **Shallow copy** copies field values; nested objects are shared.
- **Deep copy** recursively copies nested objects.

`Object.clone()` is awkward (needs `Cloneable`, shallow by default, bypasses constructors, throws checked exception). Prefer **copy constructors** or static factories, or serialization/mapping libraries for deep copies.

```
class Address { String city; Address(Address a) { this.city = a.city; } Address(String c){city=c;} }
class Customer {
    String name; Address address;
    Customer(Customer c) { this.name = c.name; this.address = new Address(c.address); } // deep copy
}
```

1.12 Marker interfaces, annotations, enums, nested classes

- **Marker interface** has no methods (`Serializable`, `Cloneable`) and tags a class. Annotations are the modern, more flexible way to add metadata.
- **Annotations** are metadata read at compile time (Lombok, `@Override`) or runtime via reflection (Spring `@Component`). Retention: `SOURCE`, `CLASS`, `RUNTIME`.
- **Enums** are full classes with fixed instances; can have fields, constructors, methods and implement interfaces. They are the safest singleton and work well with `switch` and `EnumMap`.
- **Nested classes:** *static nested* (no outer instance), *inner* (holds a hidden reference to the outer instance — can cause memory leaks), *local*, *anonymous*. Prefer static nested unless you need the outer instance.

```
enum OrderStatus {
    NEW(false), PAID(false), SHIPPED(false), CANCELLED(true);
    private final boolean terminal;
    OrderStatus(boolean terminal) { this.terminal = terminal; }
    public boolean isTerminal() { return terminal; }
}
```

1.13 Serialization, `transient`, `volatile`

- **Serialization** converts an object to bytes. Implement `Serializable` and declare `serialVersionUID` so versions stay compatible.
- `transient` fields are skipped (passwords, caches).
- **Security warning:** Java native deserialization of untrusted data is a well-known attack vector. Prefer JSON (Jackson) or Protobuf; if you must use it, apply serialization filters.
- `volatile` guarantees **visibility** and ordering of a variable between threads, not atomicity (see concurrency chapter).

1.14 Records and modern language features (Java 14–21)

Feature	Version (final)	One-line use
`var` local type inference	10	Less boilerplate for locals
Switch expressions	14	`case X -> value;` returns a value
Text blocks	15	Multi-line strings `"""`
Records	16	Immutable data carriers
Pattern matching `instanceof`	16	`if (o instanceof String s)`
Sealed classes	17	Restrict which classes may extend
Pattern matching for `switch`	21	Type patterns in switch
Record patterns	21	Destructure records
Virtual threads	21	Lightweight threads

```
record Point(int x, int y) {
    Point { if (x < 0) throw new IllegalArgumentException("x"); } // compact constructor
}
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double s) implements Shape {}

static double area(Shape s) {          // Java 21
    return switch (s) {
        case Circle c -> Math.PI * c.r() * c.r();
        case Square q -> q.s() * q.s();
    };
}
```

Compatibility: Spring Boot 3 requires Java 17. If your company is on Java 8 or 11, know the features above exist but do not use them in code labelled Java 8.

Part 2 — OOP and Design Principles

2.1 The four pillars

Pillar	Meaning	Example
Encapsulation	Hide state, expose behaviour	`BankAccount.withdraw()` checks balance; balance field is private
Abstraction	Show *what*, hide *how*	`PaymentGateway.pay()` hides the bank API details
Inheritance	Reuse and extend a type	`SavingsAccount extends Account`
Polymorphism	One interface, many behaviours	`List` can be an `ArrayList` or `LinkedList`

```
public class BankAccount {
    private long balanceCents;                      // encapsulation
    public void withdraw(long amount) {
        if (amount <= 0 || amount > balanceCents) throw new IllegalArgumentException("Invalid amount");
        balanceCents -= amount;
    }
}
```

2.2 SOLID with enterprise examples

S — Single Responsibility: a class has one reason to change. `OrderService` should not also render PDFs and send emails; delegate to `InvoiceRenderer` and `EmailNotifier`.

O — Open/Closed: extend behaviour without editing tested code.

```
interface DiscountPolicy { double apply(double total); }
class FestivalDiscount implements DiscountPolicy { public double apply(double t){ return t*0.9; } }
class NoDiscount implements DiscountPolicy { public double apply(double t){ return t; } }
// Adding a new policy = a new class; Checkout never changes.
```

L — Liskov Substitution: a subtype must work wherever the parent is expected. The classic violation: `Square extends Rectangle` where `setWidth` also changes height. Or a `ReadOnlyList extends List` that throws on `add()`.

I — Interface Segregation: many small interfaces beat one fat one. A `Printer` should not be forced to implement `fax()`.

D — Dependency Inversion: high-level modules depend on abstractions. `OrderService` depends on `PaymentGateway` (interface), not `StripeClient`. Spring's DI container makes this natural.

2.3 Other principles

- **DRY** — one source of truth for each piece of knowledge. Do not over-DRY: two similar-looking pieces of code that change for different reasons should stay separate.
- **KISS** — simplest design that works.

- **YAGNI** — do not build what you do not need yet.
- **Low coupling, high cohesion** — classes should know little about each other and each class should do one focused thing.
- **Program to an interface** — makes testing (mocking) and replacement easy.

2.4 Spotting and fixing poor design

Smell	Symptom	Fix
God class	3000-line `OrderManager`	Split by responsibility
Feature envy	Method uses another class's data more than its own	Move the method
Long parameter list	10 arguments	Parameter object / Builder
Primitive obsession	Strings for money/email	Value objects (`Money` , `Email`)
Anemic domain model	Entities are only getters/setters; logic in giant services	Move invariants into entities
Switch on type	`if (type == CARD) ... else if ...`	Strategy / polymorphism
Hidden dependencies	`new` inside methods, static singletons	Dependency injection

Interview-ready answer: "I look at responsibilities and dependencies. If a class has several reasons to change or hard-wired dependencies, I extract interfaces, inject dependencies, and apply patterns like Strategy. I protect the refactoring with tests first, then change in small safe steps."

Part 3 — Collections Framework

3.1 The big picture

Collection (singular) is the root interface for `List`, `Set`, `Queue`. **Collections** (plural) is a utility class with static helpers (`sort`, `unmodifiableList`, `synchronizedList`). **Map** is *not* a `Collection` but is part of the framework.

Type	Order	Duplicates	Typical use
`List`	Insertion order, indexed	Yes	Ordered results, lookups by position
`Set`	Depends on implementation	No	Uniqueness checks
`Queue/Deque`	FIFO / both ends	Yes	Task queues, BFS, sliding windows
`Map`	Depends on implementation	Unique keys	Lookups by key, caches, counting

3.2 ArrayList vs LinkedList

Simple answer: `ArrayList` is backed by a dynamic array: fast random access, slow insert/remove in the middle. `LinkedList` is a doubly linked list: fast add/remove at the ends *once you have the node*, slow `get(i)`.

Operation	ArrayList	LinkedList
`get(i)`	O(1)	O(n)
`add(e)` at end	O(1) amortized	O(1)
`add(i, e)` / `remove(i)` middle	O(n) (shifts elements)	O(n) to find, O(1) to link
Memory	Compact	Extra object + 2 pointers per element
Cache locality	Excellent	Poor

Reality check: in practice `ArrayList` wins in almost every case, even for many inserts, because of memory locality. Use `ArrayDeque` for queues/stacks instead of `LinkedList`.

Internals: `ArrayList` starts with capacity 10 (lazily allocated), grows by about 1.5x (`newCap = old + old>>1`) by copying to a new array. If you know the size, pass it to the constructor.

3.3 HashSet, LinkedHashSet, TreeSet

	HashSet	LinkedHashSet	TreeSet
Order	None	Insertion	Sorted (natural or Comparator)
add/contains	O(1) average	O(1) average	O(log n)
Backed by	HashMap	LinkedHashMap	TreeMap (red-black tree)
null	One null	One null	No null (with natural ordering)

`HashSet` is just a `HashMap` whose values are a dummy constant.

3.4 HashMap internal working (most asked)

Interview question: *How does HashMap work internally?*

Simple answer: HashMap stores entries in an array of **buckets**. For a key it computes `hashCode()`, spreads the bits, takes `index = (n-1) & hash`, and puts the entry in that bucket. Collisions (several keys in one bucket) are chained in a linked list, and since Java 8 a long chain (8+ nodes when table size ≥ 64) becomes a **red-black tree**. When the map exceeds `capacity × loadFactor` (default 16 × 0.75), the table doubles and entries are redistributed.

Understand it step by step, `put("Asha", 1)`:

1. `h = key.hashCode()`; then `hash = h ^ (h >>> 16)` mixes high bits into low bits so poor hash functions spread better.
2. `index = (table.length - 1) & hash`. This works as modulo because the length is always a **power of two**.
3. If the bucket is empty, create a node there.
4. If not empty, compare each node: same hash **and** (`==` or `equals()`) key → replace the value. Otherwise append.
5. If the chain length reaches 8 (and table ≥ 64), "treeify". If table < 64, it resizes instead.
6. If `size > threshold`, **resize**: new array is twice as large. Each node goes either to the same index or to `index + oldCapacity`, which makes rehashing cheap.

```
table (size 16)
[0] -> null
[1] -> (Asha,1) -> (Ravi,2)        <- collision, linked list (or tree if long)
[2] -> null
...
```

`get(key)`: same hash → bucket → walk or tree-search using `equals()`. O(1) average, O(log n) worst in Java 8+ (was O(n) in Java 7).

Java 7 vs Java 8:

	Java 7	Java 8+
Collision structure	Linked list only	Linked list → red-black tree
Worst-case lookup	O(n)	O(log n) (keys must be Comparable to order well, otherwise tie-broken)
Insert in bucket	Head insertion	Tail insertion
Resize in concurrent use	Could create infinite loop (circular list)	Fixed that specific issue, but HashMap is **still not thread-safe**
Hash function	More complex	`h ^ (h >>> 16)`

Null: HashMap allows one null key (hash 0) and null values. `Hashtable` and `ConcurrentHashMap` allow neither.

Common mistakes: * Using mutable objects as keys. If a field in `hashCode` changes after insertion, the entry can never be found again. * Forgetting to override `equals` + `hashCode` for custom keys. * Not pre-sizing a map with a known number of entries (causes repeated resizes). * Sharing a `HashMap` across threads.

Follow-ups: Why is capacity a power of 2? What is the load factor trade-off? What happens when two keys have equal hash but are not equal? Why treeify threshold 8?

Interview-ready answer: "A HashMap is an array of buckets. I hash the key, spread the bits, and use `(n-1) & hash` for the index. Collisions are chained; in Java 8 long chains become red-black trees so the worst case is O(log n). When size passes 75% of capacity the table doubles. It relies on equals and hashCode of keys and it is not thread-safe, so for concurrent use I choose ConcurrentHashMap."

3.5 HashMap vs Hashtable vs ConcurrentHashMap vs TreeMap

	HashMap	Hashtable	ConcurrentHashMap	TreeMap
Thread-safe	No	Yes (whole-map lock)	Yes (fine-grained)	No
Null key/value	Yes	No	No	Values yes; key no (natural order)
Ordering	None	None	None	Sorted
get/put	O(1) avg	O(1) avg, slow lock	O(1) avg, high concurrency	O(log n)
Status	Default	Legacy, avoid	Use for shared maps	Sorted/range queries

ConcurrentHashMap internals (high level): Java 7 used segment locks. Java 8+ uses a table of nodes, **CAS** to insert into empty buckets, and `synchronized` on the **first node of a bucket** for collisions. Reads are mostly lock-free. Iterators are **weakly consistent** (never throw `ConcurrentModificationException`).

Important: `ConcurrentHashMap` makes *individual* operations atomic. A check-then-act sequence is still a race:

```
// Race: two threads both see "absent"
if (!map.containsKey(k)) map.put(k, v);
// Correct, atomic:
map.putIfAbsent(k, v);
map.computeIfAbsent(k, key -> load(key));
map.merge(word, 1, Integer::sum); // atomic counter
```

3.6 Comparable vs Comparator

	Comparable	Comparator
Package	`java.lang`	`java.util`
Method	`compareTo(T o)`	`compare(T a, T b)`
Defines	The class's *natural* order	Any *external* order
Where	Inside the class	Separate class, lambda

```
class Employee {
    String name; double salary; String dept;
    // constructor omitted
}
```

```
List<Employee> list = ...;
list.sort(Comparator.comparing((Employee e) -> e.dept)
        .thenComparing(e -> e.salary, Comparator.reverseOrder())
        .thenComparing(e -> e.name));
```

Trap: `(a, b) -> a.x - b.x` overflows for large ints. Use `Integer.compare(a.x, b.x)`.

3.7 Iterators, fail-fast and ConcurrentModificationException

- **Iterator** goes forward and can `remove()`. **ListIterator** goes both ways and can `add` / `set`.
- **Fail-fast** iterators (ArrayList, HashMap) check an internal `modCount`; if the collection is structurally changed during iteration by something other than the iterator, they throw `ConcurrentModificationException` on a best-effort basis. It is a bug detector, not a guarantee.
- "Fail-safe" is informal terminology. The accurate terms are **weakly consistent** (ConcurrentHashMap) or **snapshot** iterators (`CopyOnWriteArrayList`).

```
// WRONG
for (String s : names) if (s.startsWith("A")) names.remove(s); // CME

// RIGHT (Java 8+)
names.removeIf(s -> s.startsWith("A"));
// or with iterator
Iterator<String> it = names.iterator();
while (it.hasNext()) if (it.next().startsWith("A")) it.remove();
```

`CopyOnWriteArrayList` copies the array on every write: great when reads vastly outnumber writes (listeners), terrible for frequent writes.

3.8 PriorityQueue, Queue, Deque

- `PriorityQueue` is a **binary min-heap** (smallest by default). `offer/poll` O(log n), `peek` O(1). Iteration order is *not* sorted.
- `ArrayDeque` is the best general-purpose stack and queue. `Stack` and `Vector` are legacy.
- `BlockingQueue` (see concurrency) for producer-consumer.

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
```

3.9 Choosing the right collection

Need	Choose
Fast lookup by key	`HashMap`
Keep insertion order	`LinkedHashMap` / `LinkedHashSet`
Sorted keys / range query	`TreeMap` / `TreeSet`
LRU cache	`LinkedHashMap` with `accessOrder=true` + `removeEldestEntry`
Shared across threads	`ConcurrentHashMap`
Queue / stack	`ArrayDeque`
Top-K	`PriorityQueue`
Immutable small list	`List.of(...)` (Java 9+)
Enum keys	`EnumMap`

3.10 Collections coding problems

```
// 1. Find duplicates (Java 8)
public static Set<Integer> findDuplicates(List<Integer> nums) {
    Set<Integer> seen = new HashSet<>(), dup = new LinkedHashSet<>();
    for (int n : nums) if (!seen.add(n)) dup.add(n);   // add returns false if already present
    return dup;
}

// 2. Frequency count
Map<String, Long> freq = words.stream()
        .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));

// 3. Group employees by department
Map<String, List<Employee>> byDept = employees.stream()
        .collect(Collectors.groupingBy(e -> e.dept));

// 4. Sort map by value descending
List<Map.Entry<String, Long>> sorted = new ArrayList<>(freq.entrySet());
sorted.sort(Map.Entry.<String, Long>comparingByValue().reversed());

// 5. First non-repeated character
public static Character firstUnique(String s) {
    Map<Character, Integer> m = new LinkedHashMap<>();
    for (char c : s.toCharArray()) m.merge(c, 1, Integer::sum);
    for (Map.Entry<Character, Integer> e : m.entrySet()) if (e.getValue() == 1) return e.getKey();
    return null;
}
```

Part 4 — Java 8 and Functional Programming

4.1 Lambdas and functional interfaces

A **functional interface** has exactly one abstract method (optionally annotated `@FunctionalInterface`). A **lambda** is a short way to implement it.

Interface	Method	Meaning	Example
`Predicate<T>`	`boolean test(T)`	Test a condition	`s -> s.isEmpty()`
`Function<T,R>`	`R apply(T)`	Transform	`s -> s.length()`
`Consumer<T>`	`void accept(T)`	Use a value	`System.out::println`
`Supplier<T>`	`T get()`	Produce a value	`() -> new ArrayList<>()`
`BiFunction`, `UnaryOperator`, `BinaryOperator`		Two-arg / same-type variants	

Internals: lambdas are compiled using `invokedynamic`, not anonymous inner classes, so no extra `.class` file per lambda. Variables captured by a lambda must be **effectively final**. `this` inside a lambda refers to the enclosing class (unlike anonymous classes).

Method references: `String::length`, `System.out::println`, `Employee::new`.

4.2 Stream API

A stream is a **pipeline**: source → intermediate operations (lazy) → terminal operation (triggers execution). Streams do not store data and **cannot be reused**.

```
List<String> result = employees.stream()          // source
    .filter(e -> e.salary > 50_000)               // intermediate
    .map(e -> e.name.toUpperCase())               // intermediate
    .sorted()                                     // intermediate (stateful)
    .collect(Collectors.toList());                // terminal
```

Lazy evaluation: nothing runs until the terminal operation; elements flow one by one through the whole pipeline (so `findFirst()` can short-circuit). `peek()` is for debugging only.

map vs flatMap: `map` is one-to-one; `flatMap` is one-to-many and flattens.

```
List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3));
List<Integer> flat = nested.stream().flatMap(List::stream).collect(Collectors.toList()); // [1,2,3]
```

Other points: * `findFirst()` respects order; `findAny()` is cheaper in parallel streams. * `reduce(0, Integer::sum)` combines elements. For sum, prefer `mapToInt(...).sum()` (avoids boxing). * `forEach` should have side effects only for output; do not mutate shared collections inside streams.

Collectors

```
Map<String, List<Employee>> byDept = emps.stream().collect(groupingBy(e -> e.dept));
Map<String, Double> avgSalary = emps.stream().collect(groupingBy(e -> e.dept, averagingDouble(e ->
e.salary)));
Map<Boolean, List<Employee>> highLow = emps.stream().collect(partitioningBy(e -> e.salary > 80_000));
Map<Long, String> idToName = emps.stream().collect(toMap(e -> e.id, e -> e.name)); // throws on
duplicate keys!
Map<String, Employee> topPerDept = emps.stream().collect(
    toMap(e -> e.dept, e -> e, BinaryOperator.maxBy(Comparator.comparingDouble(e -> e.salary))));
String names = emps.stream().map(e -> e.name).collect(joining(", ", "[", "]"));
```

Trap: `toMap` throws `IllegalStateException` on duplicate keys unless you supply a merge function.

4.3 Optional

```
Optional<User> user = repo.findByEmail(email);
String name = user.map(User::getName).orElse("Guest");
String name2 = user.map(User::getName).orElseGet(() -> loadDefaultName()); // lazy
User u = user.orElseThrow(() -> new UserNotFoundException(email));
```

- `orElse(x)` **always evaluates** `x`; `orElseGet(supplier)` only if empty. Use `orElseGet` when the default is expensive (a DB call, `new` object).
- **Correct use:** return type of methods that may have no result.
- **Misuse:** as a field, as a method parameter, in entities, `Optional.get()` without `isPresent`, `Optional<List<X>>` (return an empty list instead), `optional.isPresent() ? optional.get() : null`.

4.4 Parallel streams

`list.parallelStream()` splits work across the common **ForkJoinPool**. It helps only for CPU-heavy, independent, stateless work on large, easily splittable data (arrays, `ArrayList`). It hurts with blocking I/O (starves the shared common pool), small data, `LinkedList`, shared mutable state, or when ordering matters. Measure first. In servers, avoid it for request handling.

4.5 Other Java 8 features

- **Default/static interface methods** allow adding methods to interfaces without breaking implementors (how `List.sort` was added).
- **Date/Time API (`java.time`)**: immutable and thread-safe — `LocalDate`, `LocalDateTime`, `Instant`, `ZonedDateTime`, `Duration`, `Period`, `DateTimeFormatter`. Replaces `Date`, `Calendar`, and the non-thread-safe `SimpleDateFormat`. Store timestamps as UTC `Instant`.
- **CompletableFuture** (see concurrency chapter).

```
long days = ChronoUnit.DAYS.between(LocalDate.of(2024,1,1), LocalDate.now());
```

4.6 Java 8 coding questions

```
// Second highest salary
Optional<Double> second = emps.stream().map(e -> e.salary).distinct()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst();

// Highest paid per department
Map<String, Optional<Employee>> top = emps.stream()
        .collect(groupingBy(e -> e.dept, maxBy(Comparator.comparingDouble(e -> e.salary))));

// Count characters
Map<Character, Long> chars = "banana".chars().mapToObj(c -> (char) c)
        .collect(groupingBy(Function.identity(), counting()));

// Check palindrome
boolean pal = IntStream.range(0, s.length() / 2).allMatch(i -> s.charAt(i) == s.charAt(s.length() - 1 -
i));

// Join non-empty names
String joined = Stream.of("a", "", "c").filter(s -> !s.isEmpty()).collect(joining("-"));
```

4.7 Java 9–21 feature timeline

Version	Highlights
9	Modules (JPMS), `List.of/Set.of/Map.of`, private interface methods, `Optional.ifPresentOrElse`
10	`var`
11 (LTS)	`String.isBlank/strip/lines/repeat`, new `HttpClient`, single-file run
14	Switch expressions
15	Text blocks
16	Records, pattern matching `instanceof`, `Stream.toList()`
17 (LTS)	Sealed classes, strong encapsulation of JDK internals
21 (LTS)	Virtual threads, pattern matching for switch, record patterns, sequenced collections

Compatibility implications: Java 17 removed some internals access (`--add-opens` may be needed by old libraries); `javax.*` → `jakarta.*` for Jakarta EE 9+ (Spring Boot 3); upgrading from 8 → 17 is largely about libraries, Lombok/Mockito/Hibernate versions, and removed APIs (e.g., Nashorn JS engine, Java EE modules in the JDK).

var: only for locals with an obvious right-hand side; do not use when the type aids readability.

Part 5 — Exception Handling

5.1 Checked vs unchecked

Simple answer: *Checked* exceptions (subclasses of `Exception` but not `RuntimeException`) must be caught or declared; the compiler forces it. *Unchecked* exceptions (`RuntimeException` and `Error`) need not be. `Error` signals JVM-level problems (`OutOfMemoryError`) and should not normally be caught.

```
Throwable
 ├── Error (OutOfMemoryError, StackOverflowError)         - do not catch
 └── Exception
      ├── IOException, SQLException ... (checked)
      └── RuntimeException (unchecked): NullPointerException, IllegalArgumentException,
IllegalStateException
```

Guideline: use checked exceptions when the caller can reasonably recover (rare in modern code). Use unchecked exceptions for programming errors and business rule violations in most Spring apps. Spring wraps `SQLException` into unchecked `DataAccessException`.

`throw` raises one exception; `throws` declares what a method may raise.

5.2 try-catch-finally and try-with-resources

```java
try (BufferedReader br = Files.newBufferedReader(path);
     Connection c = ds.getConnection()) {     // closed automatically in reverse order
    ...
} catch (IOException | SQLException e) {      // multi-catch
    throw new ReportException("Could not build report", e); // keep the cause
}
```

Resources must implement `AutoCloseable`. If both the body and `close()` throw, the body's exception is primary and the close exception is **suppressed** (`getSuppressed()`).

5.3 Best practices

- Never swallow exceptions (`catch (Exception e) {}`). At minimum log with the stack trace **once**, at the boundary that handles it. Do not log and rethrow at every layer (duplicate noise).
- Wrap with context and keep the cause: `new OrderException("Order " + id + " failed", e)`.
- Catch the most specific type. Do not catch `Throwable`.
- Do not use exceptions for normal control flow (slow stack-trace creation, hard to read).
- Do not leak internals (SQL, stack traces) to API clients.
- Create custom exceptions with meaning: `InsufficientBalanceException`, `ResourceNotFoundException`.

5.4 Global exception handling in Spring Boot

```
public record ApiError(Instant timestamp, int status, String code, String message, String path,
                       List<String> details) {}

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> notFound(ResourceNotFoundException ex, HttpServletRequest req) {
        return build(HttpStatus.NOT_FOUND, "NOT_FOUND", ex.getMessage(), req, List.of());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> validation(MethodArgumentNotValidException ex, HttpServletRequest
req) {
        List<String> details = ex.getBindingResult().getFieldErrors().stream()
                .map(f -> f.getField() + ": " + f.getDefaultMessage()).toList();
        return build(HttpStatus.BAD_REQUEST, "VALIDATION_FAILED", "Invalid request", req, details);
    }

    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
    public ResponseEntity<ApiError> conflict(Exception ex, HttpServletRequest req) {
        return build(HttpStatus.CONFLICT, "CONCURRENT_UPDATE", "Resource was modified, retry", req,
List.of());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> fallback(Exception ex, HttpServletRequest req) {
        log.error("Unhandled error on {}", req.getRequestURI(), ex);           // full detail in logs
        return build(HttpStatus.INTERNAL_SERVER_ERROR, "INTERNAL_ERROR", "Something went wrong", req,
List.of());
    }

    private ResponseEntity<ApiError> build(HttpStatus s, String code, String msg, HttpServletRequest r,
List<String> d) {
        return ResponseEntity.status(s).body(new ApiError(Instant.now(), s.value(), code, msg,
r.getRequestURI(), d));
    }
    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);
}
```

(Uses Java 16+ `record` and `Stream.toList()`; replace with a class and `Collectors.toList()` on Java 8.) Spring 6 / Boot 3 also supports RFC 7807 `ProblemDetail`.

Part 6 — JVM, Memory and Garbage Collection

6.1 JVM architecture and memory areas

```
                +------------------- JVM -------------------+
.class files -> | Class Loader Subsystem                    |
                | Runtime Data Areas:                       |
                |   Heap (shared)    : Young (Eden,S0,S1) + Old |
                |   Metaspace (shared, native memory): class metadata |
                |   Per thread: JVM Stack, PC register, Native stack  |
                | Execution Engine: Interpreter, JIT, GC    |
                +-------------------------------------------+
```

Area	Stores	Notes
Heap	All objects, arrays	Shared; GC-managed; `-Xms`, `-Xmx`
Stack (per thread)	Frames: local variables, operand stack, return address	Fast; `StackOverflowError` if too deep; `-Xss`
Metaspace	Class metadata, method info	Native memory (Java 8+, replaced PermGen); grows unless capped by `-XX:MaxMetaspaceSize`
PC register	Current instruction per thread	
String pool, static fields	On heap	

Class loaders: Bootstrap (core JDK) → Platform (Ext) → Application (your classpath), using **parent delegation**: ask the parent first. This prevents you from replacing `java.lang.String`. Application servers use custom loaders for isolation, which is why classloader leaks happen on redeploy.

Object creation: `new Order()` → class loaded? → allocate in Eden (thread-local allocation buffers make this very fast) → zero fields → run constructor.

6.2 Garbage collection fundamentals

An object is garbage when it is **unreachable** from **GC roots** (thread stacks, static fields, JNI references).

Generational hypothesis: most objects die young. So:

1. New objects go to **Eden**. When Eden is full, a **minor GC** copies survivors into a Survivor space (S0/S1) and increments their age.
2. After surviving enough cycles, objects are **promoted** to the **Old generation**.
3. When the old gen fills up, a **major/mixed/full GC** runs.

Stop-the-world (STW): pauses application threads. Collector choice is largely a trade-off between **throughput** and **pause time**.

Collector	Idea	Typical use
Serial	Single thread	Tiny apps
Parallel	Multi-thread, throughput-focused	Batch jobs
G1 (default since Java 9)	Heap split into regions, collects the most garbage-filled first, targets pause goal (`-XX:MaxGCPauseMillis`)	General servers
ZGC	Concurrent, very short pauses; generational mode added in Java 21	Large heaps, low latency
Shenandoah	Concurrent compaction (in some OpenJDK builds)	Low latency

I am not quoting numeric pause times because they depend on workload and hardware. Always measure with GC logs.

6.3 Reference types

Type	Collected when	Use
Strong	Never while reachable	Normal references
`SoftReference`	Memory is low	Memory-sensitive caches
`WeakReference`	Next GC if only weakly reachable	`WeakHashMap` , canonical maps
`PhantomReference`	After finalization-ready; used with `ReferenceQueue`	Cleanup actions (like `Cleaner`)

6.4 Memory leaks in Java

Java has GC, but leaks happen when objects stay **reachable but unused**:

- Static collections that only grow (`static Map` as cache without eviction)
- Listeners/callbacks never deregistered
- `ThreadLocal` values never removed in thread pools (call `remove()`)
- Unclosed resources (connections, streams)
- Inner classes holding outer references
- Caches without size/TTL (use Caffeine with limits)
- Classloader leaks on redeploys
- Huge `HttpSession` data

OOM vs SOE: `OutOfMemoryError` — heap/metaspace/native/threads exhausted (different message: `Java heap space` , `Metaspace` , `unable to create native thread` , `GC overhead limit exceeded`). `StackOverflowError` — recursion too deep (missing base case, cyclic `toString`).

6.5 Investigating memory problems

```
# Run with: -Xmx2g -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps -Xlog:gc*:file=gc.log
jcmd <pid> GC.heap_info
jcmd <pid> GC.class_histogram | head -30     # which classes use the most instances
```

```
jcmd <pid> GC.heap_dump /tmp/heap.hprof      # analyse in Eclipse MAT or VisualVM
jcmd <pid> Thread.print > threads.txt        # or jstack <pid>
```

Method: 1. Is it a leak (old gen grows after every full GC) or just too small a heap (stable high usage)? 2. Take a heap dump; open in **Eclipse MAT** → "Leak suspects" and the **dominator tree** to see who retains the memory. 3. Look at **GC logs**: frequency, pause times, heap after GC. 4. Compare two dumps over time. Find the GC-root path ("path to GC roots"). 5. Fix: bound the cache, remove listeners, close resources, paginate queries instead of loading entire tables.

Tuning: do not tune blindly. Set `-Xms=-Xmx` for servers, choose a collector for your latency goals, size containers properly (JVM is container-aware since Java 10; use `-XX:MaxRAMPercentage`), and fix allocation hot spots first.

Part 7 — Multithreading and Concurrency

7.1 Basics

- **Process:** independent program with its own memory. **Thread:** lightweight execution path *inside* a process, sharing heap memory (hence the concurrency problems).
- **Thread states:** `NEW → RUNNABLE ⇄ BLOCKED / WAITING / TIMED_WAITING → TERMINATED`.
- **Creating work:** extend `Thread` (avoid), implement `Runnable` (no result, no checked exception), implement `Callable<V>` (returns a value, may throw), or submit to an `ExecutorService`.

```
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> expensiveCalculation());
int result = f.get(2, TimeUnit.SECONDS);       // blocks; use a timeout
pool.shutdown();
```

7.2 Race condition, synchronized and visibility

Question: *Why is `count++` not thread-safe?*

Simple answer: `count++` is three steps — read, add, write. Two threads can read the same value and both write back `old+1`, losing an update. That is a **race condition**.

```
class Counter {
    private int count;
    public synchronized void inc() { count++; }          // atomicity + visibility
    public synchronized int get() { return count; }
}
```

`synchronized` gives two guarantees: **mutual exclusion** (one thread at a time per monitor) and **visibility** (changes before releasing a lock are seen by the next thread acquiring it — the *happens-before* relationship).

- On an instance method, the lock is `this`; on a static method, the `Class` object; on a block, any object you choose. Prefer a private lock object: `private final Object lock = new Object();`.
- Locks are **reentrant**: a thread can re-enter locks it already holds.
- Internally, every Java object has a monitor; the JVM optimizes uncontended locking (biased/lightweight locks have evolved across versions; do not quote specifics).

`volatile` vs `synchronized`:

	volatile	synchronized
Visibility	Yes	Yes
Atomicity of compound actions	**No**	Yes
Blocking	No	Yes
Use	Flags (`running`), safe publication of immutable object	Critical sections

```
private volatile boolean running = true;   // visible to the worker thread immediately
```

7.3 Atomic classes and CAS

`AtomicInteger`, `AtomicLong`, `AtomicReference` use **Compare-And-Swap** — a CPU instruction: "set to new only if the current value equals what I last read; otherwise retry." This is lock-free and fast under moderate contention. For heavily contended counters, `LongAdder` is better. CAS has the **ABA problem** (value changed A→B→A; solved with `AtomicStampedReference`).

```
AtomicInteger hits = new AtomicInteger();
hits.incrementAndGet();
```

7.4 ReentrantLock and ReadWriteLock

```
private final ReentrantLock lock = new ReentrantLock();
public boolean tryTransfer(...) throws InterruptedException {
    if (!lock.tryLock(1, TimeUnit.SECONDS)) return false;   // can time out, unlike synchronized
    try { ...; return true; }
    finally { lock.unlock(); }                               // ALWAYS in finally
}
```

`ReentrantLock` adds `tryLock`, timed/interruptible locking, fairness, and multiple `Condition`s. `ReadWriteLock` lets many readers or one writer — useful for read-heavy shared data. `StampedLock` supports optimistic reads.

7.5 wait / notify

Used inside `synchronized` for thread coordination (the building block for producer-consumer). Always call `wait()` in a **loop** (spurious wakeups) and prefer `BlockingQueue` or `CountDownLatch` in new code.

```
synchronized (lock) { while (!ready) lock.wait(); ... }
synchronized (lock) { ready = true; lock.notifyAll(); }
```

7.6 Deadlock, starvation, livelock

- **Deadlock:** A holds lock 1 and waits for lock 2; B holds lock 2 and waits for lock 1. Four conditions: mutual exclusion, hold-and-wait, no preemption, circular wait.
- **Prevention:** acquire locks in a **consistent global order**; use `tryLock` with timeout; keep critical sections small; avoid calling foreign code while holding a lock.
- **Starvation:** a thread never gets CPU or lock. **Livelock:** threads keep reacting to each other without progress.
- **Detect:** `jstack <pid>` prints "Found one Java-level deadlock"; `ThreadMXBean.findDeadlockedThreads()`.

```
void transfer(Account a, Account b, long amt) {
    Account first = a.id < b.id ? a : b, second = a.id < b.id ? b : a;   // consistent order
    synchronized (first) { synchronized (second) { a.debit(amt); b.credit(amt); } }
}
```

7.7 Thread pools

Factory	Behaviour	Caution
`newFixedThreadPool(n)`	n threads, **unbounded** queue	Queue can grow until OOM
`newCachedThreadPool()`	Creates threads on demand	Unbounded threads → can crash under load
`newScheduledThreadPool(n)`	Delayed/periodic tasks	
`newSingleThreadExecutor()`	One worker, ordered	
`newVirtualThreadPerTaskExecutor()`	Java 21 virtual threads	See below

In production, build **bounded** pools explicitly:

```
ThreadPoolExecutor ex = new ThreadPoolExecutor(
    8, 16, 60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(200),                    // bounded queue
    new ThreadFactoryBuilder().setNameFormat("order-%d").build(), // name threads (Guava) or custom
factory
    new ThreadPoolExecutor.CallerRunsPolicy());       // back-pressure instead of failure
```

Order of events: core threads first → then queue → then extra threads up to max → then the rejection policy.
Pool sizing: CPU-bound ≈ number of cores; I/O-bound can be higher; measure.

7.8 BlockingQueue and producer–consumer

```
BlockingQueue<Order> queue = new ArrayBlockingQueue<>(100);
// producer
queue.put(order);                 // blocks if full (back-pressure)
// consumer
Order o = queue.take();           // blocks if empty
```

Other tools: `CountDownLatch` (wait for N events), `CyclicBarrier`, `Semaphore` (limit concurrent access), `Phaser`, `Exchanger`.

7.9 ForkJoinPool

Divide-and-conquer pool where idle workers **steal** tasks from busy workers' queues. Used by parallel streams and `CompletableFuture` async methods by default (the common pool). Use `RecursiveTask<V>` and `fork()/join()`.

7.10 CompletableFuture

```
CompletableFuture<Order> orderF = CompletableFuture.supplyAsync(() -> orderClient.get(id), ioPool);
CompletableFuture<User> userF   = CompletableFuture.supplyAsync(() -> userClient.get(uid), ioPool);

CompletableFuture<Summary> summary = orderF.thenCombine(userF, Summary::new)   // combine independent
results
```

```
        .orTimeout(2, TimeUnit.SECONDS)                                  // Java 9+
        .exceptionally(ex -> Summary.fallback())                        // recover
        .whenComplete((r, ex) -> log.info("done"));
CompletableFuture.allOf(orderF, userF).join();                           // wait for all
```

	Future	CompletableFuture
Get result	Blocking `get()`	Callbacks + chaining
Compose	No	`thenApply`, `thenCompose`, `thenCombine`
Exceptions	Wrapped in `ExecutionException`	`exceptionally`, `handle`
Manual completion	No	`complete()`

`thenApply` = map; `thenCompose` = flatMap (when the next step returns a CompletableFuture). **Always pass your own executor** for blocking work; do not block the common pool.

7.11 Virtual threads (Java 21)

Virtual threads are cheap threads scheduled by the JVM onto a small pool of carrier OS threads. When code blocks on I/O, the virtual thread is unmounted so the carrier is free. **They help** for high-concurrency, blocking I/O workloads (many HTTP/DB calls), letting you keep the simple thread-per-request model. **They do not help** CPU-bound work and are not faster for a single task. Cautions: do not pool them (create one per task), watch for **pinning** (blocking inside `synchronized` pinned the carrier in Java 21; prefer `ReentrantLock`), `ThreadLocal` abuse, and limit downstream resources (a connection pool still caps DB concurrency). Spring Boot 3.2+: `spring.threads.virtual.enabled=true`.

7.12 Thread-safety strategies, ordered by preference

1. **Immutability** (final fields, records) — no sharing problem.
2. **Confinement** — keep data in one thread (`ThreadLocal`, local variables).
3. **Concurrent collections / atomics.**
4. **Locks** — last resort, smallest scope.

7.13 Classic concurrency coding problems

```
// Thread-safe lazy singleton (double-checked locking needs volatile)
public class Config {
    private static volatile Config instance;
    private Config() {}
    public static Config get() {
        if (instance == null) { synchronized (Config.class) { if (instance == null) instance = new
Config(); } }
        return instance;
    }
}
// Simpler: enum singleton, or the holder idiom
class Holder { private static class H { static final Holder I = new Holder(); } static Holder get() {
return H.I; } }

// Print odd/even alternately with two threads: use a shared lock + turn flag, or two Semaphores.
```

```
// Producer/consumer: BlockingQueue (shown above).
// Bank transfer without deadlock: ordered locking (shown above).
```

7.14 Debugging thread problems in production

- **High CPU:** `top -H -p <pid>` → convert the hot thread id to hex → find in `jstack` output (`nid=0x...`). Take several dumps a few seconds apart.
- **Hung requests:** many threads `WAITING` on the same lock or connection pool → thread dump shows who holds the lock.
- **Thread-pool exhaustion:** all workers busy, queue full, rejections. Find blocking calls without timeouts.
- **Prevent:** timeouts everywhere, bounded pools, bulkheads per dependency, metrics on pool usage (`executor.active`, queue size).

Part 8 — Spring Core and Spring Boot

8.1 What is Spring, IoC and DI?

Interview question: *What is Inversion of Control and Dependency Injection? Why does Spring use them?*

Simple answer: Normally *your* code creates the objects it needs (`new PaymentService()`). With **IoC**, you hand that control to a container: it creates objects (called **beans**), wires them together and manages their life. **Dependency Injection** is the way the container gives each object its dependencies, usually through the constructor.

Real-life example: A restaurant chef does not buy vegetables and build the oven; a manager delivers them. The chef just cooks. In code, `OrderService` just uses a `PaymentGateway` ; Spring supplies the concrete one.

```
@Service
public class OrderService {
    private final PaymentGateway gateway;          // depends on an interface
    private final OrderRepository repo;

    public OrderService(PaymentGateway gateway, OrderRepository repo) { // constructor injection
        this.gateway = gateway;
        this.repo = repo;
    }
}
```

Why it matters: loose coupling, easy testing (pass mocks), configuration in one place, cross-cutting features via proxies (transactions, security, caching).

Constructor vs setter vs field injection:

	Constructor	Setter	Field (`@Autowired` on field)
Immutability (`final`)	Yes	No	No
Mandatory dependencies obvious	Yes	No	Hidden
Easy unit test without Spring	Yes	Possible	Needs reflection
Circular dependency detection	Fails fast	Hidden	Hidden
Recommendation	**Default**	Optional deps	Avoid

Since Spring 4.3, a single constructor needs no `@Autowired` .

8.2 The container: BeanFactory vs ApplicationContext

`BeanFactory` is the basic container (lazy bean creation). `ApplicationContext` extends it and adds event publishing, internationalization, environment/profile support, annotation processing, AOP integration, and **eager singleton creation at startup** (so misconfiguration fails fast). In practice you always use `ApplicationContext` .

8.3 Bean lifecycle (very common)

1. Container reads bean definitions (annotations, `@Bean`, XML).
2. **Instantiate** the bean (constructor).
3. **Populate** dependencies (setter/field injection).
4. Aware callbacks (`BeanNameAware`, `ApplicationContextAware`).
5. `BeanPostProcessor.postProcessBeforeInitialization`.
6. **Initialization:** `@PostConstruct` → `InitializingBean.afterPropertiesSet()` → custom `initMethod`.
7. `BeanPostProcessor.postProcessAfterInitialization` — **this is where AOP proxies are created** (for `@Transactional`, `@Async`, `@Cacheable`).
8. Bean is ready to use.
9. On shutdown: `@PreDestroy` → `DisposableBean.destroy()` → custom `destroyMethod`.

```
@Component
class CacheWarmer {
    @PostConstruct void warm() { /* runs once after dependencies are injected */ }
    @PreDestroy void flush() { /* cleanup */ }
}
```

Bean scopes: `singleton` (default, one per container), `prototype` (new each request; Spring does not manage its destruction), `request`, `session`, `application`, `websocket`.

Spring singleton vs GoF singleton: a Spring singleton is one instance **per application context**, not per JVM or per class loader. Singleton beans must be **stateless** (or thread-safe) because many threads share them. Injecting a prototype into a singleton gives you one instance only; use `ObjectProvider<T>` or `@Lookup` to get a fresh one each time.

8.4 Stereotype annotations and configuration

- `@Component` — generic. `@Service` — business logic (semantic only). `@Repository` — data access; adds **exception translation** to `DataAccessException`. `@Controller` / `@RestController` — web layer.
- **`@Component` vs `@Bean`:** `@Component` is class-level and auto-detected by scanning; `@Bean` is a method-level factory inside a `@Configuration` class — use it for third-party classes you cannot annotate or when construction logic is non-trivial.
- `@Configuration` classes are CGLIB-proxied so inter-`@Bean` method calls return the same singleton (*full* mode); `@Configuration(proxyBeanMethods = false)` is faster when you do not call bean methods from each other.
- **Component scanning:** `@SpringBootApplication` scans the package of the main class and sub-packages. Beans outside that package are not found (a classic "bean not found" cause).

```
@Configuration
class HttpConfig {
    @Bean RestClient restClient(RestClient.Builder b) { return
b.baseUrl("https://api.partner.com").build(); }
}
```

`@Qualifier` vs `@Primary`: with multiple candidates of one type, `@Primary` marks the default; `@Qualifier("name")` at the injection point chooses explicitly (and wins over `@Primary`).

Circular dependencies: A needs B and B needs A. Constructor injection fails with `BeanCurrentlyInCreationException`. Spring Boot 2.6+ prohibits circular references by default. Fix the design (extract a third class, use events, or inject via `@Lazy`) rather than enabling `spring.main.allow-circular-references`.

`@Value` vs `@ConfigurationProperties`:

```
@ConfigurationProperties(prefix = "payment")
@Validated
public record PaymentProperties(@NotBlank String baseUrl, @DefaultValue("2s") Duration timeout, int
maxRetries) {}
// enable via @ConfigurationPropertiesScan or @EnableConfigurationProperties
```

`@Value("${x}")` is fine for one-off values; `@ConfigurationProperties` is typed, grouped, validated and supports relaxed binding (`max-retries`, `MAX_RETRIES`). Prefer it.

Conditional beans: `@Profile("prod")`, `@ConditionalOnProperty`, `@ConditionalOnMissingBean`, `@ConditionalOnClass`. The auto-configuration system is built on these.

Spring events:

```
publisher.publishEvent(new OrderPlacedEvent(order.getId()));
@Component class Mailer {
    @EventListener void on(OrderPlacedEvent e) { ... }
    @TransactionalEventListener(phase = AFTER_COMMIT) void afterCommit(OrderPlacedEvent e) { ... } //
only if the TX committed
}
```

8.5 AOP and proxies

AOP (Aspect-Oriented Programming) separates cross-cutting concerns (logging, security, transactions, metrics) from business code.

- **Aspect:** the module (a class). **Join point:** a point in execution (Spring supports method execution). **Pointcut:** expression that selects join points. **Advice:** the code to run — `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`.

```
@Aspect @Component
class TimingAspect {
    @Around("execution(* com.shop.service..*(..))")
    Object time(ProceedingJoinPoint pjp) throws Throwable {
        long t = System.nanoTime();
        try { return pjp.proceed(); }
        finally { log.info("{} took {} ms", pjp.getSignature(), (System.nanoTime() - t) / 1_000_000); }
    }
}
```

How it works: Spring wraps the bean in a **proxy**. Callers get the proxy; it runs advice then delegates to the real object.

	JDK dynamic proxy	CGLIB proxy
Requires	Target implements an interface	Subclasses the target class
Limits	Only interface methods	Cannot proxy `final` classes/methods
Default in Boot	No	**Yes** (`spring.aop.proxy-target-class=true`)

Famous limitation: self-invocation. Calling `this.otherMethod()` inside the same bean bypasses the proxy, so `@Transactional` , `@Async` , `@Cacheable` annotations on `otherMethod` do **not** apply. Fix by moving the method to another bean (or inject self; or use AspectJ weaving). Also, annotated methods must be `public` and called from outside.

8.6 Common Spring mistakes

- Package structure puts beans outside the scan path.
- Stateful singleton beans (a mutable field shared by requests).
- `@Transactional` on private methods or self-invoked methods.
- Field injection hiding too many dependencies (a sign the class is too big).
- Business logic in controllers.
- Using `new` for a Spring-managed bean (the result has no injection or proxies).
- Overusing `@Autowired` with `required=false` and null checks.

8.7 Spring vs Spring Boot

	Spring Framework	Spring Boot
Purpose	Core programming and configuration model (IoC, MVC, Data, TX)	Opinionated, fast setup on top of Spring
Configuration	You wire a lot yourself	**Auto-configuration**, sensible defaults
Server	Deploy a WAR to an external server	**Embedded** Tomcat/Jetty/Undertow, run a JAR
Dependencies	Choose compatible versions yourself	**Starters** and a managed BOM
Ops	Add your own	**Actuator**, externalized config

8.8 Spring Boot internals

Interview question: *Explain auto-configuration. What does `@SpringBootApplication` do?*

Simple answer: `@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan` . Auto-configuration looks at what is on the classpath and in your properties, then registers beans for you *only if you have not defined your own*.

How auto-config works:

1. `@EnableAutoConfiguration` imports a selector that reads a list of auto-configuration classes from `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (Boot 2.7+/3.x; older

Boot used `spring.factories`).
2. Each class is guarded by conditions: `@ConditionalOnClass(DataSource.class)`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`.
3. If `spring-boot-starter-data-jpa` and an H2 driver are present and you defined no `DataSource`, Boot creates one with HikariCP.
4. Your own bean wins thanks to `@ConditionalOnMissingBean`.
5. Debug it: run with `--debug` or `debug=true` to print the **conditions evaluation report**; also Actuator's `/actuator/conditions`.

Starters are dependency descriptors (`spring-boot-starter-web` pulls Spring MVC, Jackson, Tomcat, validation basics, logging). The Spring Boot BOM/parent manages compatible versions.

Startup lifecycle: from `main()` to ready

```
SpringApplication.run(App.class, args)
 1. Create SpringApplication, detect web app type (servlet / reactive / none)
 2. Load ApplicationContextInitializers & listeners (from AutoConfiguration/spring.factories)
 3. Publish ApplicationStartingEvent
 4. Prepare Environment: command-line args, env vars, application.yml, profiles, then
ApplicationEnvironmentPreparedEvent
 5. Print banner, create ApplicationContext (AnnotationConfigServletWebServerApplicationContext)
 6. Prepare context, load bean definitions (component scan + auto-configuration),
ApplicationContextInitializedEvent
 7. refresh():
      - BeanFactoryPostProcessors (e.g., process @Configuration, property placeholders)
      - Register BeanPostProcessors
      - Create the embedded web server (Tomcat)
      - Instantiate all non-lazy singleton beans (DI, @PostConstruct, proxies)
      - Finish refresh -> ContextRefreshedEvent; start the web server on its port
 8. ApplicationStartedEvent
 9. Run CommandLineRunner / ApplicationRunner beans
10. ApplicationReadyEvent  (+ readiness state = ACCEPTING_TRAFFIC)
```

Configuration precedence (high to low, simplified): command-line args → Java system properties → OS environment variables → `application-{profile}.yml` → `application.yml` → defaults. Keep secrets **out of the repository**: use environment variables, a secret manager (Vault, AWS Secrets Manager, Kubernetes Secrets), or Spring Cloud Config.

properties vs yml: YAML is hierarchical and readable (lists, nesting) but whitespace-sensitive; `.properties` is flat. Functionally equivalent.

Profiles: `spring.profiles.active=prod`; `application-prod.yml` overrides common values; `@Profile` conditionally registers beans.

Embedded servers: Tomcat is the default; Jetty and Undertow are alternatives (exclude Tomcat from the web starter and add the other). Boot 3.2+ also supports virtual threads.

Actuator: production endpoints — `/actuator/health` (with liveness/readiness groups for Kubernetes), `/metrics` (Micrometer), `/info`, `/env`, `/loggers` (change log levels live), `/threaddump`, `/heapdump`. **Expose only what you need and secure them.**

```yaml
management:
  endpoints.web.exposure.include: health,info,metrics,prometheus
  endpoint.health.probes.enabled: true
server:
```

```
  shutdown: graceful                # finish in-flight requests
spring.lifecycle.timeout-per-shutdown-phase: 30s
```

Logging: SLF4J API with Logback by default. `logging.level.com.shop=DEBUG`. Use parameterized messages (`log.info("order {}", id)`), never log secrets or personal data, add a correlation id via **MDC**, and use JSON logs in production.

Spring Boot 2 vs 3 (and later)

	Boot 2.x	Boot 3.x
Java	8+	**17+**
Spring Framework	5	6
Namespace	`javax.*`	**`jakarta.*`**
Security config	`WebSecurityConfigurerAdapter`	`SecurityFilterChain` bean (adapter removed)
Observability	Sleuth	**Micrometer Tracing / Observation**
Native images	Experimental	GraalVM native supported
Error format	Custom	`ProblemDetail` (RFC 7807) support
Open-source support	Ended (check the Spring project's support policy for exact dates)	Current line

Migration checklist (2 → 3): upgrade to the latest 2.7.x first; move to Java 17; replace `javax.` imports with `jakarta.`; remove `WebSecurityConfigurerAdapter`; update Hibernate (6.x changes ID generation defaults and some HQL behaviour), Flyway, Lombok, Mockito; replace Sleuth with Micrometer Tracing; use the `spring-boot-properties-migrator` and OpenRewrite recipes; update third-party starters. **Boot 4:** the Boot 4.x line is built on Spring Framework 7 (modularized auto-configuration, Jakarta EE 11 baseline, Jackson 3 support). Because the details are still evolving, check the official migration guide before quoting specifics in an interview.

Testing and deployment in one glance

- Tests: see Part 11.
- Deployment: build an executable "fat" JAR (`mvn package`), use layered JARs for smaller Docker layers, or build an image with `spring-boot:build-image` (Buildpacks). Use graceful shutdown, readiness probes, externalized config and resource limits. See the DevOps part.

Part 9 — Spring MVC and REST APIs

9.1 DispatcherServlet and request flow

```
Client -> Servlet Filters (security, logging, CORS)
       -> DispatcherServlet (front controller)
           1. HandlerMapping: which controller method matches URL + HTTP method?
           2. HandlerAdapter: invoke it
              - HandlerInterceptor.preHandle
              - ArgumentResolvers: @PathVariable, @RequestParam, @RequestBody (HttpMessageConverter /
Jackson), @Valid
              - Controller method runs -> calls Service -> Repository
              - Return value handled: @ResponseBody -> HttpMessageConverter -> JSON
           3. Exceptions -> HandlerExceptionResolver -> @ControllerAdvice
       <- Response (interceptor postHandle/afterCompletion, filters on the way out)
```

`@Controller` vs `@RestController`: `@RestController` = `@Controller` + `@ResponseBody`: return values are written to the body (JSON) rather than resolved as view names.

Filter vs Interceptor: Filters are servlet-level (run before DispatcherServlet; Spring Security lives here). Interceptors are Spring MVC-level and know the handler method.

Annotation	Reads from	Example
`@PathVariable`	URL path	`/orders/{id}` identifies a resource
`@RequestParam`	Query string / form	`/orders?status=NEW&page=0` filters
`@RequestBody`	JSON body (converted by Jackson)	create/update payloads
`@ModelAttribute`	Form fields / query params bound to an object	HTML forms, non-JSON
`@RequestHeader`	HTTP header	`Idempotency-Key`

9.2 REST design principles

- **Resources are nouns**, plural: `/orders`, `/orders/{id}/items`. Verbs are the HTTP methods.
- **GET** (safe, idempotent) read; **POST** create (not idempotent); **PUT** full replace (idempotent); **PATCH** partial update; **DELETE** (idempotent).
- **Status codes:** 200 OK, 201 Created (+ `Location` header), 204 No Content, 400 bad input, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 422 unprocessable (semantic validation), 429 too many requests, 500 server error, 502/503/504 upstream issues.
- **Stateless:** each request carries what the server needs; scale horizontally.
- **Idempotency** = repeating a request has the same effect as once. Make `POST /payments` safe to retry using an `Idempotency-Key` header: store the key + response; if the same key returns, return the stored response.
- **Pagination:** offset (`page`, `size`) is simple but slow/inconsistent for deep pages; **cursor/keyset** (`after=<id>`) is stable and fast for big, changing datasets.
- **Versioning:** URI (`/v1/orders`), header, or media type. Choose one; avoid breaking changes; deprecate with notice.

- **DTOs vs entities:** never expose JPA entities directly. Entities leak internals (and lazy associations and cycles), couple your API to the schema, and invite mass-assignment vulnerabilities. Use DTOs (records) and a mapper (MapStruct).
- **Validation:** `@Valid` (Jakarta Bean Validation, on `@RequestBody`) vs `@Validated` (Spring; supports **validation groups** and works at class level for method parameter validation, e.g., on `@RequestParam` and `@PathVariable`).
- **Content negotiation:** `Accept` header picks the representation (JSON/XML); `Content-Type` describes what you send.
- **CORS:** browser security rule; configure allowed origins explicitly (`CorsConfigurationSource`) — never `*` with credentials.
- **Docs:** springdoc-openapi (`springdoc-openapi-starter-webmvc-ui` for Boot 3) generates OpenAPI + Swagger UI.
- **Rate limiting:** at the API gateway, or per-client token bucket (Bucket4j, Redis); respond 429 + `Retry-After`.
- **Correlation id:** a filter reads or creates `X-Correlation-Id`, puts it in MDC and in the response, and forwards it to downstream calls.
- **File upload:** `MultipartFile`; set `spring.servlet.multipart.max-file-size`; stream large files; validate type/size; store in object storage and return a reference. Download with `ResponseEntity<Resource>` and `Content-Disposition`.

9.3 Complete CRUD example (Boot 3, Java 17)

Entity

```
@Entity
@Table(name = "products")
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, length = 120) private String name;
    @Column(nullable = false, precision = 12, scale = 2) private BigDecimal price;
    @Version private Long version;                          // optimistic locking
    protected Product() {}                                  // JPA needs a no-arg constructor
    public Product(String name, BigDecimal price) { this.name = name; this.price = price; }
    // getters/setters omitted
}
```

DTOs with validation

```
public record ProductRequest(
        @NotBlank @Size(max = 120) String name,
        @NotNull @DecimalMin("0.01") BigDecimal price) {}

public record ProductResponse(Long id, String name, BigDecimal price) {}
```

Mapper (manual; MapStruct generates the same thing)

```
@Component
public class ProductMapper {
    public Product toEntity(ProductRequest r) { return new Product(r.name(), r.price()); }
    public ProductResponse toResponse(Product p) { return new ProductResponse(p.getId(), p.getName(),
```

```
p.getPrice()); }
}
```

Repository

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    Page<Product> findByNameContainingIgnoreCase(String name, Pageable pageable);
}
```

Service

```
@Service
public class ProductService {
    private final ProductRepository repo;
    private final ProductMapper mapper;
    public ProductService(ProductRepository repo, ProductMapper mapper) { this.repo = repo; this.mapper
= mapper; }

    @Transactional
    public ProductResponse create(ProductRequest req) { return
mapper.toResponse(repo.save(mapper.toEntity(req))); }

    @Transactional(readOnly = true)
    public ProductResponse get(Long id) {
        return repo.findById(id).map(mapper::toResponse)
                   .orElseThrow(() -> new ResourceNotFoundException("Product " + id + " not found"));
    }

    @Transactional(readOnly = true)
    public Page<ProductResponse> search(String name, Pageable p) {
        return repo.findByNameContainingIgnoreCase(name, p).map(mapper::toResponse);
    }

    @Transactional
    public ProductResponse update(Long id, ProductRequest req) {
        Product p = repo.findById(id).orElseThrow(() -> new ResourceNotFoundException("Product " + id +
" not found"));
        p.setName(req.name()); p.setPrice(req.price());     // dirty checking writes the UPDATE at
commit
        return mapper.toResponse(p);
    }

    @Transactional
    public void delete(Long id) {
        if (!repo.existsById(id)) throw new ResourceNotFoundException("Product " + id + " not found");
        repo.deleteById(id);
    }
}
```

Controller

```
@RestController
@RequestMapping("/api/v1/products")
@Validated
public class ProductController {
    private final ProductService service;
    public ProductController(ProductService service) { this.service = service; }

    @PostMapping
    public ResponseEntity<ProductResponse> create(@Valid @RequestBody ProductRequest req,
UriComponentsBuilder uri) {
        ProductResponse created = service.create(req);
        return
```

```
ResponseEntity.created(uri.path("/api/v1/products/{id}").build(created.id())).body(created);
    }
    @GetMapping("/{id}") public ProductResponse get(@PathVariable Long id) { return service.get(id); }
    @GetMapping public Page<ProductResponse> search(@RequestParam(defaultValue = "") String name,
            @PageableDefault(size = 20, sort = "name") Pageable pageable) { return service.search(name,
pageable); }
    @PutMapping("/{id}") public ProductResponse update(@PathVariable Long id, @Valid @RequestBody
ProductRequest req) { return service.update(id, req); }
    @DeleteMapping("/{id}") @ResponseStatus(HttpStatus.NO_CONTENT) public void delete(@PathVariable
Long id) { service.delete(id); }
}
```

Explanation of key lines: `@Valid` triggers Bean Validation and `MethodArgumentNotValidException` (handled in the global handler in Part 5); `ResponseEntity.created` gives status 201 and a `Location` header; `readOnly = true` hints that no writes occur (Hibernate skips dirty checking/flush; some drivers can route to replicas); `update` has no `save()` call because the entity is **managed** and dirty checking persists changes; `@Version` produces a `409` path when two clients update concurrently; pageable parameters are `?page=0&size=20&sort=name,asc`.

9.4 Designing reliable APIs: checklist

- Idempotency keys for create/payment operations; timeouts and retries (safe methods only) on clients.
- Consistent error format (Part 5), correlation ids, structured logs, metrics (latency, error rate).
- Backward-compatible evolution: add fields, do not remove or rename; version when needed.
- Input limits (body size, page size caps), validation, and auth on every endpoint.
- **Duplicate request handling:** unique constraint on a business key (e.g., `(user_id, idempotency_key)`) + catch the violation and return the original result.

Part 10 — Spring Data JPA, Hibernate and Transactions

10.1 JPA vs Hibernate vs Spring Data JPA

- **JPA (Jakarta Persistence)** — the *specification* (interfaces/annotations: `@Entity`, `EntityManager`, JPQL).
- **Hibernate** — the most popular *implementation* of JPA (it also has extras like `@BatchSize`, `@NaturalId`, filters).
- **Spring Data JPA** — a layer on top that **generates repository implementations** (`JpaRepository`) and derived queries so you write less boilerplate. It still uses Hibernate/`EntityManager` underneath.
- **JDBC vs JPA:** JDBC = low level SQL + manual mapping (maximum control); JPA = object mapping, caching, dirty checking (productivity, but you must understand generated SQL).

10.2 Entity lifecycle and the persistence context

State	Meaning
Transient (new)	`new Product()`, not known to JPA
Managed (persistent)	Attached to a persistence context; changes are tracked
Detached	Was managed; the context closed (or `detach`); changes not tracked
Removed	Scheduled for deletion at flush

Persistence context = first-level cache. Within one transaction, loading the same id twice returns the **same object** with no second SQL. **Dirty checking:** at flush, Hibernate compares managed entities with their loaded snapshots and issues `UPDATE`s automatically. **Flush** sends SQL to the database (before queries in auto mode, and at commit); **commit** makes it permanent. A flush without commit is rolled back if the transaction fails.

- `save()` — `persist` for new entities, `merge` for existing (checks `isNew` via id or `@Version`). `saveAndFlush()` also flushes immediately (rarely needed).
- `findById()` hits the DB now (returns `Optional`); `getReferenceById()` returns a **lazy proxy** without a query — great for setting a foreign key without loading the whole row, but touching it outside a session may throw `LazyInitializationException` / `EntityNotFoundException`.
- **Second-level cache** is optional (shared across sessions; Ehcache/Caffeine/Redis via providers): good for rarely changed reference data; adds staleness and complexity.

10.3 Fetching, LazyInitializationException, N+1

- `FetchType.LAZY` loads an association only when accessed; `EAGER` loads it immediately. Defaults: `@ManyToOne` / `@OneToOne` are **EAGER**; `@OneToMany` / `@ManyToMany` are **LAZY**. Best practice: make everything **LAZY** and fetch what you need per use case.
- **`LazyInitializationException`:** accessing a lazy association after the session/transaction closed (typical in the controller or JSON serialization). Fixes: fetch in the query (`JOIN FETCH`, `@EntityGraph`), map to a

DTO **inside** the transaction, or use projections. **Avoid** `spring.jpa.open-in-view=true` as a crutch (it is the Boot default and keeps the connection open during view rendering; consider setting it to `false`).

N+1 problem: load N orders (1 query), then access `order.getItems()` for each (N queries) → N+1 queries.

```
// Problem
List<Order> orders = orderRepo.findAll();                // 1 query
orders.forEach(o -> o.getItems().size());                // +N queries

// Fix 1: fetch join
@Query("select distinct o from Order o join fetch o.items where o.status = :s")
List<Order> findWithItems(@Param("s") OrderStatus s);

// Fix 2: entity graph
@EntityGraph(attributePaths = "items")
List<Order> findByStatus(OrderStatus s);

// Fix 3: batch fetching (config) - loads items for many orders in IN (...) queries
// spring.jpa.properties.hibernate.default_batch_fetch_size=50

// Fix 4: DTO projection - select only needed columns, no entities
@Query("select new com.shop.OrderSummary(o.id, o.status, count(i)) from Order o left join o.items i
group by o.id, o.status")
List<OrderSummary> summaries();
```

Cautions: fetch-joining **multiple collections** causes `MultipleBagFetchException`/cartesian products; **pagination with collection fetch join** makes Hibernate paginate in memory (warning `HHH000104`) — fetch ids page first, then fetch details, or use batch size. **Detect N+1:** `spring.jpa.show-sql` (dev only), Hibernate statistics, `datasource-proxy`/p6spy, and tests asserting query counts.

10.4 Relationships

```
@Entity class Order {
    @Id @GeneratedValue Long id;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();            // inverse side
    public void addItem(OrderItem i) { items.add(i); i.setOrder(this); }   // keep both sides in sync
}
@Entity class OrderItem {
    @Id @GeneratedValue Long id;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "order_id")                                 // owning side: holds the FK
    private Order order;
}
```

- **Owning side** = the side that has the foreign key (`@JoinColumn`); `mappedBy` marks the inverse side. JPA only looks at the **owning side** to write the foreign key.
- **Cascade:** propagate operations (`PERSIST`, `MERGE`, `REMOVE`, `ALL`). `orphanRemoval` deletes children removed from the collection. Use cascade only for true parent-owned children (Order → OrderItem), never `REMOVE` across independent aggregates.
- **ManyToMany:** use a join table; for extra columns (quantity, date) model the join as its own entity.
- Prefer `Set` or a `List` with care; `@OneToMany` unidirectional without `@JoinColumn` generates an extra join table and inefficient SQL — use bidirectional or `@JoinColumn`.

Entity equality: don't use Lombok `@Data` on entities (it includes associations → recursion / lazy loading). Typical safe approach: `equals` based on the **id when not null** (with `getClass` or Hibernate proxy handling)

and a constant `hashCode` (`return getClass().hashCode();`), or use a business key (natural id).

10.5 Queries

- **Derived queries:** `findByEmailAndActiveTrue`, `countByStatus`, `existsByEmail`.
- **JPQL** works on entities and is DB-independent; **native SQL** (`nativeQuery = true`) for DB-specific features/performance.
- **Specifications / Criteria API** (`JpaSpecificationExecutor`) for dynamic filters; **Querydsl** is another popular option.
- **Pagination:** `Pageable` + `Page<T>` (issues an extra `count(*)` query) or `Slice<T>` (no count; "has next").
- **Projections:** interface or record-based DTO projections reduce data loaded.
- **Bulk updates:** `@Modifying @Query("update Product p set p.price = p.price * 1.1")` bypasses the persistence context. Managed entities become stale; use `@Modifying(clearAutomatically = true, flushAutomatically = true)`.

```
@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("update Order o set o.status = 'EXPIRED' where o.status = 'NEW' and o.createdAt < :cutoff")
int expireOld(@Param("cutoff") Instant cutoff);
```

10.6 Locking

- **Optimistic** (`@Version`): no DB lock; on update Hibernate adds `where id=? and version=?`. If 0 rows are updated → `ObjectOptimisticLockingFailureException` → return 409 and let the client retry/reload. Best for mostly-read or low-conflict data.
- **Pessimistic** (`@Lock(PESSIMISTIC_WRITE)` → `SELECT ... FOR UPDATE`): blocks other writers until commit. Use for high-conflict hot rows (inventory, seats), keep the transaction short, always set a timeout (`jakarta.persistence.lock.timeout`), and lock in consistent order to avoid deadlocks.

10.7 Other production topics

- **Auditing:** `@EnableJpaAuditing` with `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy` and `AuditingEntityListener`.
- **Migrations:** Flyway (`V1__init.sql`, versioned SQL) or Liquibase (changelogs). **Never** use `ddl-auto=update` in production; use `validate` or `none`. Make migrations backwards-compatible (expand → migrate → contract) so rolling deployments work.
- **HikariCP:** `maximumPoolSize` (don't set it huge — the database is the limit; more connections ≠ more throughput), `connectionTimeout`, `maxLifetime` (shorter than DB/firewall timeout), `leakDetectionThreshold`. Pool exhaustion shows up as `Connection is not available, request timed out`, usually caused by long transactions, slow queries, or leaked connections.
- **Performance tuning:** fix N+1; use projections; `hibernate.jdbc.batch_size` with `order_inserts/order_updates` for batch writes (note: `IDENTITY` ids disable JDBC insert batching — use `SEQUENCE`); add indexes; avoid loading huge result sets (stream or page); limit `EAGER`; consider the read model (CQRS) for heavy reporting.
- **Common production issues:** open-in-view holding connections, LazyInitializationException, N+1, missing indexes, long transactions with remote calls inside, `equals/hashCode` bugs in sets, `ddl-auto`

surprises, enum stored as ordinal (use `@Enumerated(STRING)`).

10.8 `@Transactional` in depth

How it works: Spring creates a **proxy** around the bean. When an outside caller invokes a `@Transactional` method, the proxy's `TransactionInterceptor` gets a connection from the pool, sets auto-commit to false, binds the transaction to the **current thread** (`ThreadLocal`), runs your method, then **commits** on normal return or **rolls back** on exception, and releases the connection.

Rollback rule (very common trap): by default rollback happens for **unchecked** exceptions (`RuntimeException`, `Error`). **Checked exceptions commit** unless you say `@Transactional(rollbackFor = Exception.class)`.

Why "@Transactional is not working": 1. **Self-invocation** (method called through `this`). 2. Method is `private`/`final`/not `public` (proxy limitations). 3. Bean not Spring-managed (`new`). 4. Checked exception thrown without `rollbackFor`. 5. Exception caught and swallowed inside the method. 6. Wrong import (`javax.transaction.Transactional` also works, but know which you used) or using it on an interface with a CGLIB/JDK mismatch. 7. A different `@Transactional` propagation (e.g., `REQUIRES_NEW` committed independently). 8. Work performed on another thread (`@Async`, parallel streams, `CompletableFuture`) — the transaction is thread-bound, so the other thread is **not** in it. 9. Non-transactional storage (some NoSQL).

Propagation

Propagation	Behaviour
`REQUIRED` (default)	Join the current TX; otherwise create one
`REQUIRES_NEW`	Suspend the current TX; run in a brand-new, independent TX
`SUPPORTS`	Join if exists, else run non-transactionally
`NOT_SUPPORTED`	Suspend any TX; run non-transactionally
`MANDATORY`	Must already be in a TX, otherwise exception
`NEVER`	Must **not** be in a TX, otherwise exception
`NESTED`	Savepoint inside the current TX (needs JDBC savepoints; JPA/Hibernate dialect support limited)

Real example (audit that survives rollback):

```
@Service class OrderService {
    private final AuditService audit;
    @Transactional
    public void placeOrder(OrderRequest r) {
        audit.record("ORDER_ATTEMPT", r);     // REQUIRES_NEW: saved even if the order fails
        ... // may throw -> order rolled back, audit remains
    }
}
@Service class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void record(String type, Object payload) { ... }
}
```

(`audit` is a *different bean* — the proxy applies. Putting `record()` in the same class would break this.) Note: `REQUIRES_NEW` holds **two connections** at once; under a small pool this can **deadlock the pool itself**.

Isolation levels and anomalies

Level	Dirty read	Non-repeatable read	Phantom read
READ_UNCOMMITTED	possible	possible	possible
READ_COMMITTED	prevented	possible	possible
REPEATABLE_READ	prevented	prevented	possible in the standard (MySQL InnoDB largely prevents with gap locks; PostgreSQL's snapshot isolation prevents it)
SERIALIZABLE	prevented	prevented	prevented

- *Dirty read:* see another transaction's uncommitted data. *Non-repeatable read:* the same row read twice gives different values. *Phantom:* the same query returns new rows. Defaults: PostgreSQL/Oracle/SQL Server → READ_COMMITTED; MySQL InnoDB → REPEATABLE_READ. Behaviour varies by database — verify for yours.

Other options: `readOnly=true`, `timeout=5` (seconds), `rollbackFor / noRollbackFor`. **Boundaries:** put `@Transactional` on the **service layer** (a use case = one transaction), not on controllers or individual repository calls. Keep transactions **short**: never call slow external HTTP services inside a transaction (holds a DB connection and locks).

Database vs distributed transactions: a local ACID transaction works for one database. Across services you cannot use one transaction; use **Saga/outbox** with eventual consistency (Part 12). 2PC/XA exists (JTA) but is rarely used in microservices because of coupling, blocking and availability issues.

How to debug an unexpected commit: enable `logging.level.org.springframework.transaction.interceptor=TRACE` and `org.springframework.orm.jpa=DEBUG`; check the exception type (checked?); check if the exception was caught; check propagation (an inner `REQUIRES_NEW` already committed); check proxies (is the call going through the proxy?); print `TransactionSynchronizationManager.isActualTransactionActive()`.

Interview-ready answer: "@Transactional works through a proxy that opens a transaction before the method and commits or rolls back after. Default rollback is only for unchecked exceptions. It breaks on self-invocation, non-public methods, swallowed exceptions or thread switches. I keep transactions on the service layer, short, with no remote calls inside, and use REQUIRES_NEW sparingly for things like audit logs."

Part 11 — Spring Security and Testing

11.1 Authentication vs authorization

- **Authentication (AuthN):** *Who are you?* (login, token). **Authorization (AuthZ):** *What may you do?* (roles, permissions). Failed AuthN → **401**; failed AuthZ → **403**.
- **Role-based access control (RBAC):** `ROLE_ADMIN`, `ROLE_DOCTOR`. **Authority-based:** finer permissions like `appointment:cancel`. Roles are coarse groups; authorities are specific permissions (in Spring, `hasRole("ADMIN")` checks the authority `ROLE_ADMIN`).

11.2 Spring Security architecture

Security is a chain of **servlet filters** that runs *before* `DispatcherServlet`.

```
Request -> DelegatingFilterProxy -> FilterChainProxy -> SecurityFilterChain
   [CORS] [CSRF] [Your JwtAuthenticationFilter / BearerTokenAuthenticationFilter]
   [UsernamePasswordAuthenticationFilter] [ExceptionTranslationFilter] [AuthorizationFilter]
 -> Controller
Authentication flow: Filter builds an Authentication token -> AuthenticationManager
   -> AuthenticationProvider (DaoAuthenticationProvider: UserDetailsService + PasswordEncoder)
   -> success: stored in SecurityContextHolder (thread-local) -> authorization checks
```

Key types: `SecurityFilterChain`, `AuthenticationManager`, `UserDetailsService`, `PasswordEncoder`, `GrantedAuthority`, `SecurityContext`.

11.3 Passwords

Never store plain text or fast hashes (MD5/SHA-1/plain SHA-256). Use an **adaptive, salted** hash: **BCrypt** (or Argon2/SCrypt/PBKDF2). `BCryptPasswordEncoder` embeds a random salt and a cost factor so you can increase the work over time. `DelegatingPasswordEncoder` stores an algorithm prefix (`{bcrypt}...`) for painless upgrades.

11.4 Sessions vs tokens; JWT

	Session (cookie)	JWT (stateless token)
State	Server stores session	Token carries claims; server stores nothing
Scaling	Needs sticky sessions or shared store (Redis)	Easy horizontal scaling
Revocation	Easy (delete session)	Hard until expiry (use short TTL, denylist, rotation)
CSRF	Vulnerable (cookies are sent automatically) → CSRF protection	Not by default if sent in `Authorization` header
Best for	Server-rendered web apps	APIs, mobile, microservices

JWT structure: `header.payload.signature`, each Base64URL encoded. Header = algorithm/type; payload = **claims** (`sub`, `exp`, `iat`, `iss`, `aud`, roles); signature = HMAC (HS256, shared secret) or RSA/ECDSA (RS256/ES256, private key signs, public key verifies). **The payload is encoded, not encrypted — never put secrets in it.** Validate signature, `exp`, `iss`, `aud`; reject `alg=none`; pin the expected algorithm.

Access vs refresh tokens: a short-lived **access token** (minutes) is sent with each API call; a longer-lived **refresh token** gets new access tokens. Use **refresh-token rotation** (issue a new refresh token on every use and invalidate the old one; reuse of an old one signals theft → revoke the family). Store refresh tokens server-side (hashed) so they can be revoked. For browsers, prefer `HttpOnly; Secure; SameSite` cookies over `localStorage` (XSS can steal `localStorage`).

11.5 OAuth 2.0 and OpenID Connect

- **OAuth 2.0** is an *authorization* framework: a client gets a **limited access token** to a resource on a user's behalf without seeing the password. Roles: resource owner, client, authorization server, resource server. Use the **Authorization Code flow with PKCE** for web/mobile; **Client Credentials** for service-to-service. Implicit and Resource Owner Password flows are discouraged/deprecated.
- **OpenID Connect (OIDC)** adds an **identity layer** on OAuth 2.0: an **ID token** (a JWT) says who the user is.
- **OAuth 2.0 vs JWT:** OAuth is a *protocol*; JWT is a *token format*. OAuth access tokens may be JWTs or opaque strings.
- Typical setup: Keycloak/Auth0/Okta/Cognito as the authorization server; your Spring Boot service is a **resource server** validating JWTs with `spring-boot-starter-oauth2-resource-server`.

11.6 CORS vs CSRF

- **CORS:** browser mechanism letting a page from origin A call your API on origin B. It *relaxes* the same-origin policy; the **server** declares allowed origins/methods/headers.
- **CSRF:** an attacker's site makes the victim's browser send an authenticated request using the victim's cookies. Defence: CSRF tokens, `SameSite` cookies. For a stateless API using `Authorization: Bearer` headers (not cookies), CSRF protection can be disabled safely; **if you use cookie auth, keep it enabled.**

11.7 A practical JWT flow (Spring Boot 3 / Spring Security 6)

Option A (recommended when you have an identity provider): resource server

```
@Configuration
@EnableMethodSecurity                          // enables @PreAuthorize
public class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http
          .csrf(csrf -> csrf.disable())                                          // stateless bearer-
token API
          .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
          .cors(Customizer.withDefaults())
          .authorizeHttpRequests(a -> a
              .requestMatchers("/actuator/health/**", "/api/v1/auth/**").permitAll()
              .requestMatchers(HttpMethod.GET, "/api/v1/products/**").hasAnyRole("USER", "ADMIN")
```

```
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
          .oauth2ResourceServer(o -> o.jwt(j -> j.jwtAuthenticationConverter(converter())));
        return http.build();
    }

    @Bean
    JwtAuthenticationConverter converter() {
        JwtGrantedAuthoritiesConverter g = new JwtGrantedAuthoritiesConverter();
        g.setAuthoritiesClaimName("roles");
        g.setAuthorityPrefix("ROLE_");
        JwtAuthenticationConverter c = new JwtAuthenticationConverter();
        c.setJwtGrantedAuthoritiesConverter(g);
        return c;
    }

    @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
// application.yml: spring.security.oauth2.resourceserver.jwt.issuer-uri:
https://idp.example.com/realms/shop
```

Option B: your own login endpoint issues the JWT (for self-contained apps)

Flow: 1. `POST /api/v1/auth/login {email, password}` → `AuthenticationManager.authenticate(...)` verifies the BCrypt hash via your `UserDetailsService`. 2. On success a service builds a signed JWT (for example with `NimbusJwtEncoder`, jjwt, or Nimbus library): `sub=userId`, `roles`, `exp=now+15min`, `iss`. Also create and store a hashed **refresh token**. 3. The client sends `Authorization: Bearer <token>` on each request. 4. A `JwtDecoder` bean (or a custom `OncePerRequestFilter`) validates signature and expiry, and builds the `Authentication` placed in the `SecurityContext`. 5. `authorizeHttpRequests` and `@PreAuthorize("hasRole('ADMIN') or #userId == authentication.name")` enforce access. 6. On expiry the client calls `/auth/refresh`; the server rotates the refresh token. 7. Logout deletes the refresh token (and optionally denylists the access token's `jti` until it expires).

```
@RestController @RequestMapping("/api/v1/auth")
class AuthController {
    private final AuthenticationManager manager; private final TokenService tokens;
    AuthController(AuthenticationManager m, TokenService t) { this.manager = m; this.tokens = t; }

    @PostMapping("/login")
    TokenResponse login(@Valid @RequestBody LoginRequest r) {
        Authentication a = manager.authenticate(new UsernamePasswordAuthenticationToken(r.email(),
r.password()));
        return tokens.issue(a);                // access + refresh
    }
}
```

Method-level security: `@PreAuthorize("hasAuthority('appointment:cancel')")` (needs `@EnableMethodSecurity`). Use it for **ownership checks** ("only the patient can cancel their own appointment") that URL rules cannot express.

Obsolete style to recognize in legacy code: `class SecurityConfig extends WebSecurityConfigurerAdapter` with `configure(HttpSecurity)`, `antMatchers`, `.and()` chains. Removed in Spring Security 6; migrate to a `SecurityFilterChain` bean with the lambda DSL and `requestMatchers`.

11.8 Common vulnerabilities and secure practices

- **Broken access control / IDOR:** `GET /orders/42` returns someone else's order. Always check ownership server-side.
- **Mass assignment:** binding request JSON directly to entities (`role` field set by attacker) → use DTOs.
- **SQL injection:** never concatenate input; use parameterized queries/JPA parameters.
- **XSS, CSRF, SSRF, insecure deserialization, secrets in code, verbose errors** leaking stack traces.
- **Brute force:** rate limit login, lockout/backoff, MFA.
- **Transport:** HTTPS/TLS everywhere, HSTS; secure cookies.
- **Secrets:** environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets with encryption); rotate regularly; use short-lived credentials; never log them; scan repositories (git-secrets, gitleaks).
- **Dependencies:** scan with OWASP Dependency-Check / Dependabot / Snyk; keep Spring patched.
- **Error handling:** generic messages to clients ("Invalid credentials" for both unknown user and wrong password), details in logs.

11.9 Spring Boot testing

Unit vs integration tests:

	Unit	Integration
Scope	One class, dependencies mocked	Several real components (Spring context, DB, HTTP)
Speed	Milliseconds	Seconds
Tools	JUnit 5 + Mockito	`@SpringBootTest`, `@DataJpaTest`, Testcontainers
Find	Logic bugs	Wiring, SQL, serialization, config bugs

Follow the **test pyramid**: many unit tests, fewer slice/integration tests, very few end-to-end tests.

Mockito: `@Mock` creates a mock; `@InjectMocks` builds the class under test and injects mocks; a **Spy** wraps a *real* object so you can stub some methods (partial mock) while the rest run for real. In Spring tests, `@MockBean` (Boot ≤ 3.3) replaced a bean in the context; from Boot 3.4 use `@MockitoBean` (Spring Framework 6.2) — `@MockBean` is deprecated in newer versions. Check the Boot version you use.

Service unit test

```
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {
    @Mock ProductRepository repo;
    @Spy  ProductMapper mapper = new ProductMapper();
    @InjectMocks ProductService service;

    @Test void get_returnsProduct() {
        Product p = new Product("Pen", new BigDecimal("2.50"));
        when(repo.findById(1L)).thenReturn(Optional.of(p));

        ProductResponse r = service.get(1L);

        assertThat(r.name()).isEqualTo("Pen");
        verify(repo).findById(1L);
```

```
    }

    @Test void get_whenMissing_throws() {
        when(repo.findById(9L)).thenReturn(Optional.empty());
        assertThatThrownBy(() -> service.get(9L)).isInstanceOf(ResourceNotFoundException.class);
    }
}
```

Controller slice test with MockMvc

```
@WebMvcTest(ProductController.class)       // loads only the web layer, not services/repos
class ProductControllerTest {
    @Autowired MockMvc mvc;
    @MockBean ProductService service;      // use @MockitoBean on newer Boot
    @Autowired ObjectMapper om;

    @Test @WithMockUser(roles = "ADMIN")
    void create_returns201() throws Exception {
        when(service.create(any())).thenReturn(new ProductResponse(1L, "Pen", new BigDecimal("2.50")));
        mvc.perform(post("/api/v1/products").with(csrf())
                .contentType(MediaType.APPLICATION_JSON)
                .content(om.writeValueAsString(new ProductRequest("Pen", new BigDecimal("2.50")))))
           .andExpect(status().isCreated())
           .andExpect(header().exists("Location"))
           .andExpect(jsonPath("$.name").value("Pen"));
    }

    @Test @WithMockUser
    void create_invalid_returns400() throws Exception {
        mvc.perform(post("/api/v1/products").with(csrf()).contentType(MediaType.APPLICATION_JSON)
                .content("{\"name\":\"\",\"price\":-1}"))
           .andExpect(status().isBadRequest());
    }
}
```

Repository test with a real database (Testcontainers)

```
@DataJpaTest                                                    // JPA slice, rolls back after each
test
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE) // use the container, not
embedded H2
@Testcontainers
class ProductRepositoryTest {
    @Container @ServiceConnection                               // Boot 3.1+: auto-wires the
datasource
    static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired ProductRepository repo;

    @Test void findByName_isCaseInsensitive() {
        repo.save(new Product("Notebook", new BigDecimal("4.00")));
        Page<Product> page = repo.findByNameContainingIgnoreCase("note", PageRequest.of(0, 10));
        assertThat(page.getContent()).hasSize(1);
    }
}
```

Full integration test: `@SpringBootTest(webEnvironment = RANDOM_PORT)` + `TestRestTemplate` / `WebTestClient` / `RestAssured` + Testcontainers.

Why Testcontainers instead of H2: H2 behaves differently from your production database (SQL dialect, locking, JSON types, isolation), so tests can pass while production fails.

Good testing habits: name tests by behaviour (`transfer_failsWhenBalanceTooLow`); one reason to fail; avoid testing implementation details/over-verifying mocks (brittle); use builders/fixtures; no sleeping — use Awaitility for async; keep tests independent and deterministic (inject `Clock`); test exceptions and security paths; measure coverage but do not chase 100% — focus on business-critical logic and boundaries; test `@Transactional` behaviour with a real DB (test-managed transactions roll back by default, which can hide commit-time errors).

Part 12 — Microservices Architecture

12.1 Monolith vs microservices

Simple answer: A monolith is one deployable unit; microservices are many small, independently deployable services, each owning one business capability and its data.

	Monolith	Microservices
Deployment	One unit	Per service
Scaling	Whole app	Per service
Team autonomy	Low	High
Complexity	In the code	In the network, ops, data consistency
Latency / failures	In-process calls	Network calls can fail or be slow
Transactions	Local ACID	Distributed → sagas
Debugging	One log, one stack trace	Needs tracing and central logs

When microservices are appropriate: multiple teams working independently, parts of the system with very different scaling needs, a clear domain structure, and a mature DevOps culture (CI/CD, monitoring). **When not:** small team, unclear domain, early-stage product. A well-structured **modular monolith** is often the best start. Say this in an interview; it shows judgement.

Service decomposition & DDD basics: split by **business capability / bounded context** (a boundary within which a model and its language are consistent), not by technical layer. "Customer" in Billing differs from "Customer" in Support. **Aggregates** are consistency boundaries — keep one transaction per aggregate. Signs of bad boundaries: services that must always deploy together, chatty calls, shared database tables, distributed transactions everywhere ("distributed monolith").

12.2 Communication

	Synchronous (REST/gRPC)	Asynchronous (messaging)
Coupling	Temporal coupling (callee must be up)	Decoupled in time
Latency	Immediate response	Eventual
Failure	Cascading failures possible	Broker buffers messages
Use	Queries, user waiting for answer	Events, workflows, notifications

- **REST/JSON:** simple, universal. **gRPC:** binary Protobuf over HTTP/2, strict contracts, streaming, efficient for internal calls. **Messaging (Kafka, RabbitMQ):** events and commands.
- **Kafka vs RabbitMQ:** Kafka is a **distributed, partitioned, replicated log**: consumers pull and track offsets, messages are retained and can be replayed, great for high-throughput event streaming. RabbitMQ is a **traditional broker** with routing (exchanges/queues), push delivery, acknowledgments and per-message handling; great for task queues and complex routing.

12.3 Infrastructure building blocks

- **API Gateway** (Spring Cloud Gateway, Kong, NGINX, cloud gateways): single entry point; routing, authentication, rate limiting, TLS termination, request aggregation. Avoid putting business logic there.
- **Service discovery:** services register (Eureka, Consul) and are found by name; on Kubernetes, **Service DNS** does this for you.
- **Configuration management:** Spring Cloud Config, Consul, or Kubernetes ConfigMaps/Secrets.
- **Load balancing:** client-side (Spring Cloud LoadBalancer) or server-side (K8s Service, ALB).
- **Centralized logging:** ship JSON logs to ELK/OpenSearch/Loki. **Distributed tracing:** OpenTelemetry/Micrometer Tracing → Zipkin/Jaeger/Tempo; a **trace id** follows a request across services (propagated in headers like `traceparent`), spans show time per hop. **Correlation id** ties logs together.
- **Health checks:** liveness (restart me?) vs readiness (send me traffic?).
- **Service-to-service authentication:** mTLS (often via service mesh like Istio/Linkerd) and/or OAuth2 client-credentials tokens. Never trust "internal network = safe".

12.4 Resilience patterns

Calls to other services *will* fail or hang. Protect yourself:

- **Timeouts:** every remote call needs one. The most common cause of cascading failure is a missing timeout.
- **Retry with exponential backoff and jitter:** retry only **idempotent** operations and transient errors; add random jitter so clients do not retry in lockstep (a "retry storm" can crush a recovering service).
- **Circuit breaker:** after many failures the circuit **opens** and calls fail fast (or use a fallback), then goes **half-open** to probe recovery, then **closed**. Protects threads and the downstream service.
- **Bulkhead:** isolate resources (separate thread pools/semaphores per dependency) so one slow dependency cannot consume everything.
- **Rate limiter / load shedding:** protect yourself from overload.
- **Fallbacks:** cached data, default values, degraded features.

```
@Service
class PricingClient {
    private final RestClient http;
    PricingClient(RestClient http) { this.http = http; }

    @CircuitBreaker(name = "pricing", fallbackMethod = "cachedPrice")
    @Retry(name = "pricing")
    @TimeLimiter(name = "pricing")      // for async types; for blocking calls set HTTP client timeouts
instead
    public Price price(String sku) { return http.get().uri("/prices/{sku}",
sku).retrieve().body(Price.class); }

    Price cachedPrice(String sku, Throwable t) { return Price.unknown(sku); }
}
```

```
resilience4j:
  circuitbreaker.instances.pricing:
    slidingWindowSize: 20
    failureRateThreshold: 50
    waitDurationInOpenState: 15s
  retry.instances.pricing:
    maxAttempts: 3
    waitDuration: 200ms
```

```
    enableExponentialBackoff: true
    exponentialBackoffMultiplier: 2
    enableRandomizedWait: true
```

(Resilience4j via `resilience4j-spring-boot3`. The numbers above are illustrative settings, not recommendations — tune from real traffic.)

12.5 Data consistency across services

Database per service: each service owns its schema; others access data only through APIs/events. This gives autonomy but removes cross-service joins and ACID transactions.

Saga pattern: a business transaction is a sequence of local transactions; if a step fails, **compensating transactions** undo previous steps.

- **Orchestration:** a central coordinator (an Order Orchestrator or a workflow engine such as Temporal/Camunda) tells each service what to do. Easier to understand and monitor; the orchestrator is a coupling point.
- **Choreography:** services react to each other's events. No central brain, loosely coupled, but the flow is hard to see and cyclic dependencies creep in.

```
Order placed -> Inventory reserves stock -> Payment charges -> Order confirmed
 Failure at Payment: -> Inventory releases stock (compensation) -> Order cancelled
```

Compensation is a *business* undo (refund, release), not a database rollback; steps must be idempotent.

Transactional outbox pattern (solves "update DB **and** publish an event atomically", the dual-write problem):

1. In **one local transaction**, update the business table and insert a row into an `outbox` table.
2. A relay (a poller, or CDC with Debezium) reads the outbox and publishes to Kafka, marking rows as sent.
3. The relay may publish a message more than once (at-least-once) → consumers must be **idempotent**.

```
@Transactional
public void placeOrder(OrderRequest r) {
    Order o = orderRepo.save(Order.from(r));
    outboxRepo.save(new OutboxEvent("OrderPlaced", o.getId().toString(), toJson(o)));   // same TX
}
```

Idempotent consumer: record processed message ids (a `processed_messages` table with a unique key) in the **same transaction** as the business effect, or make the effect naturally idempotent (upsert, "set status = PAID" rather than "add 100").

Eventual consistency: after a change, different services converge to a consistent state *after a delay*. Design UX and APIs for it (pending states, polling/notifications).

CQRS: separate the **write model** (commands, validations) from the **read model** (denormalized views optimized for queries), usually synchronized with events. Benefits: scale reads independently, tailor views. Cost: complexity, eventual consistency. **Event sourcing:** store the sequence of events as the source of truth and rebuild state by replaying; gives audit history and temporal queries, but brings schema-evolution, snapshotting, and query complexity. Use only when the benefits are needed.

Distributed locking: Redis (`SET key value NX PX ttl`, Redisson, RedLock debates), database row locks, ZooKeeper/etcd. Locks need **TTLs**, and correctness-critical work should use **fencing tokens** or database constraints rather than trusting a lock alone.

12.6 Kafka fundamentals

- **Topic** is a named log split into **partitions**; each partition is an ordered, append-only sequence with **offsets**. **Ordering is guaranteed only within a partition**, so use the same **key** (e.g., `orderId`) to keep related events in order.
- **Producer** chooses the partition (by key hash). **Consumer group:** partitions are divided among consumers in a group; max parallelism = number of partitions. Different groups each get all messages.
- **Replication:** each partition has a leader and followers (`replication.factor`); `acks=all` + `min.insync.replicas` for durability. **Producer idempotence** (`enable.idempotence=true`) prevents duplicates from retries.
- **Delivery semantics: at-most-once** (may lose), **at-least-once** (may duplicate — the common default with manual/after-processing commits), **exactly-once** (Kafka transactions within Kafka-to-Kafka flows; end-to-end exactly-once with external side effects still requires idempotent handling).
- **Duplicates:** arise from producer retries, consumer rebalances, and crashes before committing offsets. Defence = idempotent processing.
- **Dead-letter topic (DLT):** messages that fail after retries are routed to a DLT for inspection/replay instead of blocking the partition. Spring Kafka: `DefaultErrorHandler` + `DeadLetterPublishingRecoverer`, and `@RetryableTopic`.
- **Schema evolution:** use a schema registry (Avro/Protobuf/JSON Schema) and compatibility rules (backward/forward/full); add optional fields, never reuse or retype fields.
- **Consumer lag:** the gap between the latest offset and the consumer's committed offset; the key health metric.

```
@Component
class OrderEventsConsumer {
    private final ProcessedMessageRepo processed; private final ShipmentService shipments;

    @KafkaListener(topics = "orders.placed", groupId = "shipping")
    @Transactional
    public void on(ConsumerRecord<String, OrderPlaced> rec) {
        String id = rec.topic() + "-" + rec.partition() + "-" + rec.offset();   // or a business event
id in the payload
        if (!processed.markIfNew(id)) return;           // unique-key insert; duplicate => skip
        shipments.createFor(rec.value());
    }
}
// spring.kafka.consumer.enable-auto-commit=false ; ack-mode record/batch after processing
```

12.7 Caching and Redis

- **Cache-aside** (most common): read cache → on miss, read DB and populate. Writes: update the DB then **invalidate** the cache entry. **Write-through/write-behind** are alternatives.
- **Spring Cache:** `@Cacheable("products")`, `@CacheEvict`, `@CachePut` (proxy-based, so the same self-invocation caveat applies).
- **Redis:** in-memory key-value store; data structures (strings, hashes, sets, sorted sets, streams); TTLs; used for caching, sessions, rate limiting, distributed locks, leaderboards.

- **Problems:** stale data; **cache stampede** (many requests rebuild a hot key at once → use per-key locking, request coalescing, jittered TTLs); **cache penetration** (queries for non-existent keys → cache negative results/Bloom filter); **hot keys**; memory eviction policy (`allkeys-lru`).
- **Distributed caches** need serialization strategy and versioned keys.

12.8 Containers and Kubernetes basics

- **Docker:** package app + runtime into an image; containers are isolated processes.
- **Kubernetes:** orchestrates containers. **Pod** (one or more containers), **Deployment** (desired replicas, rolling updates), **Service** (stable network name/load balancing), **Ingress** (HTTP routing), **ConfigMap/Secret**, **HPA** (autoscaling), **liveness/readiness/startup probes**, **resource requests/limits**.

12.9 Realistic architecture walkthrough: Clinic Appointment Platform

```
 Mobile/Web --> API Gateway (auth, rate limit, routing)
                  |-- User Service -----------> users DB
                  |-- Clinic Service ---------> clinics/doctors/slots DB  (+ Redis cache for slot
lists)
                  |-- Appointment Service ----> appointments DB (+ outbox)
                  |-- Payment Service --------> payments DB  --> external payment provider
                  '-- Notification Service ---> email/SMS/push providers
 Kafka: appointment.requested, slot.reserved, payment.completed/failed, appointment.confirmed/cancelled
 Service Registry/DNS, Config server, Central logging, Tracing (OpenTelemetry), Metrics (Prometheus)
```

Request flow (book an appointment, orchestrated saga):

1. Client → Gateway (JWT validated) → `POST /appointments {doctorId, slotId}` with an `Idempotency-Key` .
2. **Appointment Service** creates `Appointment(PENDING)` and writes `AppointmentRequested` to the outbox in one TX; returns `202/201` with a status link.
3. The orchestrator (or choreographed events) asks **Clinic Service** to **reserve the slot** (conditional update: `UPDATE slots SET status='HELD' WHERE id=? AND status='FREE'` ; 0 rows → slot taken).
4. On success → **Payment Service** charges (with idempotency key to the provider).
5. On payment success → Appointment `CONFIRMED` → event → **Notification Service** sends confirmation.
6. On payment failure/timeout → compensation: release the slot, mark `CANCELLED` , notify.

Failure scenarios:

Failure	Handling
Two users book the same slot	Atomic conditional update or unique constraint `(doctor_id, slot_time)`; loser gets 409
Payment service down	Circuit breaker, slot hold expires after TTL, appointment stays `PENDING_PAYMENT`, retry or cancel
Payment succeeded but event lost	Outbox guarantees publishing; reconciliation job compares provider records vs DB
Notification provider down	Retry with backoff, DLT; notification failure must not fail the booking
Duplicate Kafka event	Idempotent consumer
Gateway or one instance dies	Multiple replicas + readiness probes
Slot hold never released	Scheduled expiry job + TTL

Consistency: each service is strongly consistent internally and **eventually consistent** across services. **Trade-offs:** more moving parts, harder debugging, need for tracing and reconciliation versus independent scaling and team autonomy.

12.10 Observability and production troubleshooting

Three pillars: **logs** (what happened), **metrics** (how much/how fast: RED = Rate, Errors, Duration; USE = Utilization, Saturation, Errors), **traces** (where time went). Define **SLIs/SLOs** and alert on symptoms (error rate, latency) rather than only causes. Add dashboards per service: request rate, p95/p99 latency, error rate, JVM memory/GC, thread pools, DB pool, Kafka lag.

Interview-ready answer (microservices in general): "I start with business capabilities and bounded contexts, give each service its own data, communicate synchronously for queries and asynchronously for events, and design for failure with timeouts, retries with jitter, circuit breakers and idempotency. For cross-service workflows I use sagas with an outbox so state and events never diverge. And I invest in observability early, because without it microservices are undebuggable. I'd only choose microservices when the organization is big enough to benefit; otherwise a modular monolith."

Part 13 — SQL and Database Concepts

13.1 Fundamentals

- **DDL** (define structure): `CREATE`, `ALTER`, `DROP`, `TRUNCATE`. **DML** (change data): `INSERT`, `UPDATE`, `DELETE`, `MERGE`. **DQL:** `SELECT`. **DCL** (permissions): `GRANT`, `REVOKE`. **TCL** (transactions): `COMMIT`, `ROLLBACK`, `SAVEPOINT`.
- **Primary key:** uniquely identifies a row, not null, one per table. **Foreign key:** references a primary/unique key in another table and enforces referential integrity. **Unique constraint:** no duplicate values (nulls handling differs by database).
- **ACID: A**tomicity (all or nothing), **C**onsistency (constraints always hold), **I**solation (concurrent transactions do not see each other's partial work), **D**urability (committed data survives crashes, via the write-ahead log).
- **Normalization** removes redundancy: **1NF** atomic values, no repeating groups; **2NF** no partial dependency on part of a composite key; **3NF** no transitive dependency (non-key columns depend only on the key). **Denormalization** deliberately duplicates data to speed reads (reporting, read models) at the cost of update complexity.

13.2 Sample data used in the problems

```sql
CREATE TABLE department (id INT PRIMARY KEY, name VARCHAR(50));
CREATE TABLE employee (
  id INT PRIMARY KEY, name VARCHAR(50), salary INT, dept_id INT, manager_id INT);

INSERT INTO department VALUES (1,'Engineering'),(2,'Sales'),(3,'HR');
INSERT INTO employee VALUES
 (1,'Asha',  150, 1, NULL),
 (2,'Ravi',  120, 1, 1),
 (3,'Meena', 120, 1, 1),
 (4,'John',   90, 2, 1),
 (5,'Sara',  160, 2, 4),
 (6,'Tom',    70, 2, 4),
 (7,'Lee',    60, NULL, 1);
```

13.3 Joins

Join	Returns
`INNER JOIN`	Only rows that match in both tables
`LEFT JOIN`	All left rows; right columns are NULL if no match
`RIGHT JOIN`	All right rows (mirror of left)
`FULL OUTER JOIN`	All rows from both sides (not in MySQL)
`CROSS JOIN`	Every combination (cartesian product)

```sql
-- Employees with department name (Lee has no department -> use LEFT JOIN to keep him)
SELECT e.name, d.name AS dept FROM employee e LEFT JOIN department d ON d.id = e.dept_id;
```

```sql
-- Departments without employees (the "missing records" pattern)
SELECT d.name FROM department d LEFT JOIN employee e ON e.dept_id = d.id WHERE e.id IS NULL;
-- Output: HR
```

Trap: a condition on the right table in `WHERE` turns a LEFT JOIN into an INNER JOIN; put it in the `ON` clause instead.

13.4 GROUP BY, HAVING and aggregates

`WHERE` filters rows **before** grouping; `HAVING` filters groups **after** aggregation. Logical order: `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT`. `COUNT(*)` counts rows; `COUNT(col)` ignores NULLs.

```sql
-- Departments with more than 2 employees
SELECT dept_id, COUNT(*) AS cnt FROM employee GROUP BY dept_id HAVING COUNT(*) > 2;
-- Output: dept 1 -> 3
```

13.5 Classic problems

1. Second highest salary

```sql
-- Works on all databases; returns NULL if there is no second value
SELECT MAX(salary) AS second_highest FROM employee WHERE salary < (SELECT MAX(salary) FROM employee);
-- Output: 150   (salaries are 160,150,120,120,90,70,60)

-- Nth highest with DENSE_RANK (N=2)
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rk FROM employee) t WHERE rk = 2;
```

2. Duplicate records

```sql
SELECT name, COUNT(*) FROM employee GROUP BY name HAVING COUNT(*) > 1;

-- Delete duplicates keeping the lowest id (portable form)
DELETE FROM employee WHERE id NOT IN (SELECT MIN(id) FROM employee GROUP BY name);
-- (MySQL needs the subquery wrapped in a derived table.)
```

3. Highest salary per department (department-wise maximum)

```sql
SELECT d.name, MAX(e.salary) AS max_salary
FROM employee e JOIN department d ON d.id = e.dept_id GROUP BY d.name;
-- Engineering 150, Sales 160
```

4. Top N per group (top 2 earners in each department)

```sql
SELECT * FROM (
  SELECT e.*, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) rn
  FROM employee e WHERE dept_id IS NOT NULL) t
WHERE rn <= 2;
-- Engineering: Asha 150, Ravi 120 (Meena tied; ROW_NUMBER picks one arbitrarily)
-- Use DENSE_RANK() to keep ties.
```

5. Employees earning more than their manager

```sql
SELECT e.name, e.salary, m.name AS manager, m.salary AS manager_salary
FROM employee e JOIN employee m ON m.id = e.manager_id WHERE e.salary > m.salary;
-- Sara (160) earns more than manager John (90)
```

6. Ranking functions

Function	Ties	Example for salaries 160,150,120,120,90
`ROW_NUMBER()`	Unique numbers	1,2,3,4,5
`RANK()`	Same rank, **gaps** after ties	1,2,3,3,5
`DENSE_RANK()`	Same rank, **no gaps**	1,2,3,3,4

```sql
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) r, DENSE_RANK() OVER (ORDER BY salary DESC) dr
FROM employee;
```

7. Running total, previous row

```sql
SELECT id, salary, SUM(salary) OVER (ORDER BY id) running, LAG(salary) OVER (ORDER BY id) prev FROM
employee;
```

8. CTE (readable, can be recursive)

```sql
WITH dept_avg AS (SELECT dept_id, AVG(salary) a FROM employee GROUP BY dept_id)
SELECT e.name FROM employee e JOIN dept_avg d ON d.dept_id = e.dept_id WHERE e.salary > d.a;

-- Recursive: management chain below Asha (id 1)
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, 1 lvl FROM employee WHERE id = 1
  UNION ALL
  SELECT e.id, e.name, e.manager_id, c.lvl + 1 FROM employee e JOIN chain c ON e.manager_id = c.id)
SELECT * FROM chain;
```

9. Missing records / gaps: `NOT EXISTS` is usually the safest anti-join.

```sql
SELECT d.name FROM department d WHERE NOT EXISTS (SELECT 1 FROM employee e WHERE e.dept_id = d.id);
```

(`NOT IN` returns nothing if the subquery contains a NULL — a classic trap.)

10. Subqueries: *scalar* (one value), *correlated* (refers to the outer row, runs per row logically — can be slow; often rewritten as a join/window function).

```sql
-- Employees earning above their department average (correlated)
SELECT name FROM employee e WHERE salary > (SELECT AVG(salary) FROM employee WHERE dept_id =
e.dept_id);
```

13.6 Indexes and execution plans

- An **index** is a separate sorted structure (usually a **B-tree**) that lets the database find rows without scanning the whole table. Costs: extra storage and slower writes (every index must be updated).
- **Clustered index** (e.g., SQL Server, MySQL InnoDB primary key): the table rows are physically organized by the index key — one per table. **Non-clustered/secondary:** a separate structure pointing to rows (in

InnoDB it points to the primary key). PostgreSQL tables are heaps; indexes are all secondary (`CLUSTER` is a one-time reorder).

- **Composite index and column order:** an index on `(dept_id, salary)` supports queries filtering on `dept_id`, or `dept_id` + `salary` (the **leftmost prefix** rule), but not `salary` alone. Put equality columns first, range column last; consider sort order.
- **Selectivity:** how well a column narrows rows (unique email = high; boolean flag = low). Indexes on low-selectivity columns are often ignored.
- **Covering index:** includes all columns the query needs, so no table lookup is required.
- **Why an index might not be used:** function on the column (`WHERE LOWER(email)=...`), implicit type conversion, leading wildcard `LIKE '%abc'`, low selectivity, stale statistics, OR conditions.
- **Execution plan:** `EXPLAIN` / `EXPLAIN ANALYZE`. Look for full table scans on big tables, wrong join order, big gaps between estimated and actual rows, sorts spilling to disk, nested loops over large sets.

Slow-query optimization checklist: (1) get the plan; (2) add/fix the right index; (3) select only needed columns; (4) avoid N+1 queries from the app; (5) paginate with keyset instead of large `OFFSET`; (6) update statistics; (7) rewrite subqueries/functions on indexed columns; (8) consider caching, partitioning, or a read replica; (9) check lock waits.

13.7 Locks, deadlocks, procedures, scale-out

- **Optimistic vs pessimistic locking:** see Part 10. Database-level: `SELECT ... FOR UPDATE` (pessimistic), version column / compare-and-set (optimistic). Also `SKIP LOCKED` for job-queue tables.
- **Database deadlock:** two transactions wait for each other's locks; the DB aborts one (retry it). Reduce by accessing rows in a consistent order, keeping transactions short, and indexing the filtering columns (to lock fewer rows).
- **Stored procedures/triggers:** server-side logic. Pros: fewer round trips, close to data. Cons: harder to version/test/debug, business logic hidden, vendor lock-in. Triggers can surprise developers; prefer explicit application logic or auditing frameworks unless there is a reason.
- **Connection pooling:** opening connections is expensive; a pool reuses them (HikariCP). Pool size is bounded by what the database can serve.
- **Replication:** a primary accepts writes and replicas copy them (async or sync); read replicas scale reads but have **replication lag** (stale reads). **Partitioning** splits a big table inside one database (by range/hash/list) to prune scans and ease maintenance; **sharding** distributes data across **multiple databases/servers** (hard: cross-shard joins/transactions, rebalancing).
- **SQL vs NoSQL:** relational databases give schema, joins and strong ACID transactions — default for transactional business data. NoSQL families: **document** (MongoDB: flexible schema, nested data), **key-value** (Redis, DynamoDB: simple fast lookups), **wide-column** (Cassandra: huge write throughput, designed around queries), **graph** (Neo4j). Choose NoSQL for a specific access pattern or scale need, not by fashion; many systems use polyglot persistence.
- **Isolation levels** are covered in Part 10.

Part 14 — Coding Interview Problems

> Format for each problem: statement → example → assumptions → brute force → optimized approach → **Java 8 compilable code** → explanation → dry run → complexity → edge cases → follow-ups. Easy and Medium first, a few harder ones last.

Big-O in one minute: Big-O describes how time or space grows with input size `n`, ignoring constants: O(1) < O(log n) < O(n) < O(n log n) < $O(n^2)$ < $O(2^n)$. It is the *upper bound trend*, not the exact speed. Interviewers expect you to state time **and** space for every solution.

14.1 Two Sum (Easy) — HashMap

Statement: Given an int array `nums` and `target`, return indices of two numbers adding up to `target`. **Example:** `[2,7,11,15], 9 → [0,1]`. **Assumptions:** exactly one answer; same element cannot be used twice.

Brute force: check all pairs: $O(n^2)$ time, O(1) space.

Optimized: one pass; for each number look for its **complement** (`target - x`) in a map of already-seen values: O(n) time, O(n) space.

```
import java.util.*;

public class TwoSum {
    public static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> seen = new HashMap<>();          // value -> index
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (seen.containsKey(need)) return new int[]{seen.get(need), i};
            seen.put(nums[i], i);                               // store after checking so we do not
reuse the same element
        }
        throw new IllegalArgumentException("No solution");
    }
    public static void main(String[] a) { System.out.println(Arrays.toString(twoSum(new int[]
{2,7,11,15}, 9))); }
}
```

Dry run: i=0, x=2, need 7, map empty → store {2:0}. i=1, x=7, need 2 → found at index 0 → `[0,1]`. **Edge cases:** duplicates (`[3,3], 6`), negatives, no solution, empty array. **Follow-ups:** sorted input → two pointers O(1) space; return all pairs; count pairs; Three Sum.

14.2 Remove duplicates from a sorted array in place (Easy) — two pointers

Example: `[1,1,2,2,3] → length 3, array starts [1,2,3]`.

```
public static int removeDuplicates(int[] a) {
    if (a.length == 0) return 0;
```

```
    int write = 1;                                  // next position to write a unique value
    for (int read = 1; read < a.length; read++) {
        if (a[read] != a[read - 1]) a[write++] = a[read];
    }
    return write;
}
```

Dry run: `[1,1,2,2,3]` → read=2 (2≠1) a[1]=2,w=2; read=4 (3≠2) a[2]=3,w=3 → returns 3. **Complexity:** O(n) time, O(1) space. **Edge cases:** empty, all equal, no duplicates. **Follow-up:** allow at most two duplicates (compare with `a[write-2]`). **Remove Element** (value `val`) is the same pattern: `if (a[read] != val) a[write++] = a[read];`.

14.3 Reverse a string and Palindrome (Easy)

```
public static String reverse(String s) {
    char[] c = s.toCharArray();
    for (int i = 0, j = c.length - 1; i < j; i++, j--) { char t = c[i]; c[i] = c[j]; c[j] = t; }
    return new String(c);                           // O(n) time, O(n) space (strings are immutable)
}

public static boolean isPalindrome(String s) {      // ignoring case and non-alphanumerics
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i++)) != Character.toLowerCase(s.charAt(j--))) return false;
    }
    return true;                                    // O(n) time, O(1) space
}
```

Dry run `"A man, a plan"` style: pointers skip punctuation and compare lowercase letters. **Edge cases:** empty string, single char, only punctuation (true), null input.

14.4 Valid Anagram, First Non-Repeating Character, Character Frequency (Easy)

```
public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] cnt = new int[26];                        // assumes lowercase a-z; use a HashMap for Unicode
    for (int i = 0; i < a.length(); i++) { cnt[a.charAt(i) - 'a']++; cnt[b.charAt(i) - 'a']--; }
    for (int n : cnt) if (n != 0) return false;
    return true;                                    // O(n) time, O(1) space
}

public static int firstUniqueIndex(String s) {      // "leetcode" -> 0, "aabb" -> -1
    int[] cnt = new int[256];
    for (char c : s.toCharArray()) cnt[c]++;
    for (int i = 0; i < s.length(); i++) if (cnt[s.charAt(i)] == 1) return i;
    return -1;
}

public static Map<Character, Integer> frequency(String s) {
    Map<Character, Integer> m = new TreeMap<>();
    for (char c : s.toCharArray()) m.merge(c, 1, Integer::sum);
    return m;
}
```

Brute force for first unique: for each char scan the rest, $O(n^2)$. **Follow-ups:** Unicode/streaming input; return the character; case-insensitive.

14.5 Find duplicates and Group Anagrams (Medium)

```
public static List<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> groups = new HashMap<>();
    for (String w : words) {
        char[] c = w.toCharArray();
        Arrays.sort(c);                              // sorted letters = canonical key
        groups.computeIfAbsent(new String(c), k -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(groups.values());
}
```

Example: `["eat","tea","tan","ate","nat","bat"] → [[eat,tea,ate],[tan,nat],[bat]]`. **Complexity:** O(n·k log k) for n words of length k; space O(n·k). **Optimization:** use a 26-count signature as the key for O(n·k). **Edge cases:** empty strings, single word, uppercase.

14.6 Longest Substring Without Repeating Characters (Medium) — sliding window

Example: `"abcabcbb" → 3 ("abc")`. **Brute force:** check every substring, $O(n^3)$ (or $O(n^2)$ with sets). **Optimized:** keep a window `[left, right]` with no duplicates; store each char's last index; when a repeat appears inside the window, jump `left` past its previous position.

```
public static int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int best = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (last.containsKey(c) && last.get(c) >= left) left = last.get(c) + 1;   // shrink past the
duplicate
        last.put(c, right);
        best = Math.max(best, right - left + 1);
    }
    return best;                                     // O(n) time, O(min(n, alphabet)) space
}
```

Dry run `abcabcbb`**:** a,b,c → len 3; right=3 'a' seen at 0 ≥ left 0 → left=1, len 3; … the best stays 3. **Edge cases:** empty, all same (`"bbbb"` → 1), `"abba"` (the `>= left` check matters: at the final 'a', its last index 0 < left 2, so we must not move left backward).

14.7 Merge Intervals (Medium) — sort

Example: `[[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]]`.

```
public static int[][] merge(int[][] in) {
    if (in.length <= 1) return in;
    Arrays.sort(in, (x, y) -> Integer.compare(x[0], y[0]));      // avoid x[0]-y[0] overflow
    List<int[]> out = new ArrayList<>();
    int[] cur = in[0];
```

```
    for (int i = 1; i < in.length; i++) {
        if (in[i][0] <= cur[1]) cur[1] = Math.max(cur[1], in[i][1]);   // overlap -> extend
        else { out.add(cur); cur = in[i]; }
    }
    out.add(cur);
    return out.toArray(new int[0][]);              // O(n log n) time, O(n) space
}
```

Edge cases: touching intervals `[1,4],[4,5]` merge; one interval contained in another; unsorted input; empty. **Follow-up:** Insert Interval, meeting rooms (min-heap on end times).

14.8 Binary Search (Easy)

```
public static int binarySearch(int[] a, int target) {          // sorted ascending
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;                           // avoids int overflow of (lo+hi)/2
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1; else hi = mid - 1;
    }
    return -1;                                                  // O(log n) time, O(1) space
}
```

Dry run `[1,3,5,7,9], 7`: lo0 hi4 mid2(5)<7 → lo3; mid3(7) found. **Follow-ups:** first/last occurrence, rotated sorted array, search on answer ("minimum capacity to ship in D days"). **Common bug:** infinite loops from wrong bounds (`lo < hi` vs `<=`).

14.9 Linked lists: Reverse and Detect Cycle (Easy)

```
class ListNode { int val; ListNode next; ListNode(int v) { val = v; } }

public static ListNode reverse(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur != null) {
        ListNode next = cur.next;                   // remember the rest
        cur.next = prev;                            // flip the link
        prev = cur; cur = next;
    }
    return prev;                                    // O(n) time, O(1) space
}

public static boolean hasCycle(ListNode head) {     // Floyd's tortoise and hare
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;                                   // O(n) time, O(1) space (a HashSet approach is
O(n) space)
}
```

Dry run reverse `1→2→3`**:** prev=null; cur=1: 1.next=null; cur=2: 2.next=1; cur=3: 3.next=2 → head 3→2→1. **Edge cases:** empty list, one node. **Follow-ups:** find the cycle's start (reset one pointer to head, move both one step), find middle, merge two sorted lists, recursive reverse (stack depth O(n)).

14.10 Valid Parentheses (Easy) — stack

```
public static boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') st.push(')'); else if (c == '[') st.push(']'); else if (c == '{') st.push('}');
        else if (st.isEmpty() || st.pop() != c) return false;    // closing bracket must match latest
opener
    }
    return st.isEmpty();                                  // O(n) time, O(n) space
}
```

Dry run `"([)]"`: push `)`, push `]`; read `)` → pop `]` ≠ `)` → false. **Edge cases:** empty string (true), only closers, leftover openers.

14.11 Merge Sorted Arrays (Easy)

```
public static int[] mergeSorted(int[] a, int[] b) {
    int[] r = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) r[k++] = a[i] <= b[j] ? a[i++] : b[j++];
    while (i < a.length) r[k++] = a[i++];
    while (j < b.length) r[k++] = b[j++];
    return r;                                             // O(n+m) time and space
}
```

In-place variant (LeetCode 88): fill from the **end** of the bigger array so you do not overwrite unread values.

14.12 Kth Largest and Top K Frequent (Medium) — heap

```
public static int kthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();     // holds the k largest seen so far
    for (int n : nums) { minHeap.offer(n); if (minHeap.size() > k) minHeap.poll(); }
    return minHeap.peek();                                       // O(n log k) time, O(k) space
}

public static List<Integer> topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.merge(n, 1, Integer::sum);
    PriorityQueue<Map.Entry<Integer, Integer>> pq = new PriorityQueue<>((x, y) -> x.getValue() -
y.getValue());
    for (Map.Entry<Integer, Integer> e : freq.entrySet()) { pq.offer(e); if (pq.size() > k) pq.poll();
}
    List<Integer> res = new ArrayList<>();
    while (!pq.isEmpty()) res.add(pq.poll().getKey());
    return res;                                                  // O(n log k)
}
```

Brute force: sort everything O(n log n). **Alternatives:** QuickSelect O(n) average; bucket sort by frequency O(n) for Top-K Frequent. **Dry run** `[3,2,1,5,6,4], k=2`: heap keeps {5,6} → peek 5.

14.13 Tree traversals (Easy–Medium)

```
class TreeNode { int val; TreeNode left, right; TreeNode(int v) { val = v; } }

static void inorder(TreeNode n, List<Integer> out) {            // left, root, right: sorted order for
a BST
    if (n == null) return;
    inorder(n.left, out); out.add(n.val); inorder(n.right, out);
}

static List<List<Integer>> levelOrder(TreeNode root) {          // BFS with a queue
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new ArrayDeque<>(); q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();                                     // nodes on this level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode n = q.poll(); level.add(n.val);
            if (n.left != null) q.offer(n.left);
            if (n.right != null) q.offer(n.right);
        }
        res.add(level);
    }
    return res;                                                  // O(n) time, O(width) space
}

static int maxDepth(TreeNode n) { return n == null ? 0 : 1 + Math.max(maxDepth(n.left),
maxDepth(n.right)); }

static boolean isValidBST(TreeNode n, long lo, long hi) {       // call with Long.MIN_VALUE,
Long.MAX_VALUE
    if (n == null) return true;
    if (n.val <= lo || n.val >= hi) return false;
    return isValidBST(n.left, lo, n.val) && isValidBST(n.right, n.val, hi);
}
```

Preorder = root, left, right; postorder = left, right, root. Recursion depth is O(height) (O(n) for a skewed tree → use an explicit stack for very deep trees). **Common mistake in BST validation:** only comparing a node with its direct children.

14.14 Find Missing Number and Maximum Subarray (Easy / Medium)

```
public static int missingNumber(int[] nums) {                    // numbers 0..n with one missing
    int n = nums.length;
    long expected = (long) n * (n + 1) / 2, sum = 0;
    for (int x : nums) sum += x;
    return (int) (expected - sum);                               // O(n) time, O(1) space; XOR also
works
}

public static int maxSubArray(int[] a) {                         // Kadane's algorithm
    int cur = a[0], best = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);                        // extend the current run or start
fresh
        best = Math.max(best, cur);
    }
    return best;                                                 // O(n) time, O(1) space
}
```

Kadane dry run `[-2,1,-3,4,-1,2,1,-5,4]`: cur: -2,1,-2,4,3,5,6,1,5 → best 6 (subarray `4,-1,2,1`). **Edge cases:** all negatives (answer is the largest single element — why `cur` starts at `a[0]`, not 0), one element. **Follow-up:** return the subarray indices; circular array.

14.15 Recursion, graphs and dynamic programming (basics)

```
// Fibonacci: naive recursion O(2^n); memoization/DP O(n)
static long fib(int n) { if (n < 2) return n; long a = 0, b = 1; for (int i = 2; i <= n; i++) { long t
= a + b; a = b; b = t; } return b; }

// Climbing stairs (1 or 2 steps): ways(n) = ways(n-1) + ways(n-2)

// Graph BFS - shortest path in an unweighted graph
static int shortestPath(Map<Integer, List<Integer>> g, int s, int t) {
    Queue<Integer> q = new ArrayDeque<>(); Map<Integer, Integer> dist = new HashMap<>();
    q.offer(s); dist.put(s, 0);
    while (!q.isEmpty()) {
        int u = q.poll();
        if (u == t) return dist.get(u);
        for (int v : g.getOrDefault(u, Collections.<Integer>emptyList()))
            if (!dist.containsKey(v)) { dist.put(v, dist.get(u) + 1); q.offer(v); }
    }
    return -1;
}

// Coin change (min coins), bottom-up DP: dp[x] = min(dp[x - coin] + 1)
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1]; Arrays.fill(dp, amount + 1); dp[0] = 0;
    for (int x = 1; x <= amount; x++) for (int c : coins) if (c <= x) dp[x] = Math.min(dp[x], dp[x - c]
+ 1);
    return dp[amount] > amount ? -1 : dp[amount];                // O(amount * coins)
}
```

DP recipe: define the state, write the recurrence, set the base case, choose the order (memoize top-down or tabulate bottom-up), then optimize space. **Graph basics:** BFS (queue) for shortest unweighted path and levels; DFS (stack/recursion) for connectivity, cycles, topological sort; Dijkstra (priority queue) for weighted non-negative edges. Always handle **visited** sets to avoid infinite loops.

14.16 Modern Java alternatives (Java 17+)

```
// Frequency in one expression (Java 8 style works on all versions; var and toList need newer)
var freq = "banana".chars().mapToObj(c -> (char) c).collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
List<Integer> evens = Stream.of(1,2,3,4).filter(n -> n % 2 == 0).toList();   // Java 16+, immutable
record Pair(int i, int j) {}                                                  // clean result type for
Two Sum variants
```

Interview coding tips for seniors: clarify the requirements; start with a brute-force idea, then optimize and say why; name variables clearly; state complexity unprompted; test with edge cases aloud (empty, one element, duplicates, negatives, large inputs / overflow); discuss production concerns (null handling, input validation, thread safety) when relevant.

Part 15 — Design Patterns and Clean Code

For each pattern: **problem → analogy → Java/Spring use → when NOT to use.**

15.1 Singleton

Problem: exactly one instance (configuration, registry). **Analogy:** a country has one president at a time. **Best Java form:** an `enum` singleton, or the holder idiom (Part 7). In Spring, make the bean singleton-scoped and inject it — do not hand-write `getInstance()`. **Avoid when:** it carries mutable global state, hurts testing, or hides dependencies.

15.2 Factory Method and Abstract Factory

Problem: decouple object creation from usage. **Analogy:** a restaurant kitchen makes the dish; you only order by name.

```
interface Notifier { void send(String to, String msg); }
class EmailNotifier implements Notifier { public void send(String t, String m) { /* SMTP */ } }
class SmsNotifier implements Notifier { public void send(String t, String m) { /* SMS */ } }
class NotifierFactory {
    static Notifier create(String channel) {
        switch (channel) { case "EMAIL": return new EmailNotifier(); case "SMS": return new
SmsNotifier();
                           default: throw new IllegalArgumentException(channel); }
    }
}
```

Spring version: inject `Map<String, Notifier>` — Spring fills the map with bean name → bean — and look up by key (no `switch`). **Abstract Factory** creates *families* of related objects (e.g., a UI toolkit's Button+Checkbox for Windows vs Mac). **Avoid when:** only one implementation exists.

15.3 Builder

Problem: many optional parameters; avoid telescoping constructors. **Analogy:** ordering a custom sandwich step by step.

```
public final class Order {
    private final String id; private final String customer; private final String coupon;
    private Order(Builder b) { id = b.id; customer = b.customer; coupon = b.coupon; }
    public static Builder builder() { return new Builder(); }
    public static final class Builder {
        private String id, customer, coupon;
        public Builder id(String v) { id = v; return this; }
        public Builder customer(String v) { customer = v; return this; }
        public Builder coupon(String v) { coupon = v; return this; }
        public Order build() { if (id == null || customer == null) throw new
IllegalStateException("id/customer required"); return new Order(this); }
    }
}
```

Lombok `@Builder` generates this; `StringBuilder`, `Stream.Builder`, Spring's `RestClient.Builder` use it. **Avoid when:** the object has 2–3 simple fields (a constructor or record is clearer).

15.4 Strategy

Problem: swap algorithms at runtime. **Analogy:** choosing a route (fastest, cheapest) in a maps app. Replaces long `if/else` chains.

```
interface PricingStrategy { BigDecimal price(BigDecimal base); }
@Component("regular") class Regular implements PricingStrategy { public BigDecimal price(BigDecimal b)
{ return b; } }
@Component("premium") class Premium implements PricingStrategy { public BigDecimal price(BigDecimal b)
{ return b.multiply(new BigDecimal("0.90")); } }

@Service class Checkout {
    private final Map<String, PricingStrategy> strategies;                // Spring injects all beans
by name
    Checkout(Map<String, PricingStrategy> strategies) { this.strategies = strategies; }
    BigDecimal total(String tier, BigDecimal base) { return strategies.getOrDefault(tier,
strategies.get("regular")).price(base); }
}
```

In Java 8+, a strategy can simply be a lambda/`Function`. **Avoid when:** there is only one variant that never changes.

15.5 Observer

Problem: notify many dependents when something changes. **Analogy:** newsletter subscription. **Java/Spring:** Spring application events (`ApplicationEventPublisher` + `@EventListener`), Kafka topics (distributed observer), `java.util.Observer` is deprecated. **Avoid when:** order of notification or guaranteed delivery matters and you use plain in-memory listeners (they are synchronous by default, and failures can break the publisher; use `@Async` or a message broker).

15.6 Decorator

Problem: add behaviour dynamically without subclass explosion. **Analogy:** adding toppings to a pizza. **Java:** `new BufferedReader(new InputStreamReader(in))`, `Collections.unmodifiableList`. **Spring:** wrapping a `PaymentGateway` with a `LoggingPaymentGateway` / `CachingPaymentGateway` that implements the same interface and delegates. AOP achieves a similar effect. **Avoid when:** many layers make debugging hard.

15.7 Adapter and Facade

- **Adapter** converts one interface into another that the client expects. *Analogy:* a travel plug adapter. Use it to wrap a third-party SDK/legacy API behind your own interface (an `ACL`, anti-corruption layer).
- **Facade** provides a simple interface to a complex subsystem. *Analogy:* a hotel front desk. `OrderFacade.checkout()` coordinates inventory, payment and shipping calls. **Avoid** making the facade a god class.

15.8 Proxy

Problem: control access to an object (lazy loading, security, caching, transactions). **Analogy:** a receptionist in front of a manager. **Spring:** `@Transactional`, `@Cacheable`, `@Async`, security are all implemented with proxies (Part 8). Hibernate lazy entities are proxies. **Watch for:** self-invocation and `final` methods.

15.9 Template Method

Problem: fixed skeleton, variable steps. **Analogy:** a recipe where "bake" differs per cake.

```
abstract class ReportJob {
    public final void run() { List<String> rows = fetch(); String out = format(rows); deliver(out); }
    protected abstract List<String> fetch();
    protected abstract String format(List<String> rows);
    protected void deliver(String out) { System.out.println(out); }          // hook with a default
}
```

Spring's `JdbcTemplate`, `RestTemplate` use the *template* idea (with callbacks). **Prefer composition (Strategy)** when subclassing becomes rigid.

15.10 Chain of Responsibility

Problem: pass a request along handlers until one handles it. **Analogy:** a support ticket escalating from L1 to L2 to L3. **Java:** servlet `FilterChain`, Spring Security's filter chain, validation pipelines, approval workflows.

15.11 Repository, DI, MVC, Circuit Breaker

- **Repository:** an abstraction of a collection of aggregates hiding persistence (`ProductRepository`). Benefits: testable, swap storage. Pitfall: leaking JPA types/queries into the domain, or generic repositories everywhere.
- **Dependency Injection:** Part 8. **MVC:** Model (data), View (presentation), Controller (handles input); in REST apps the "view" is the JSON representation.
- **Circuit Breaker:** Part 12; originated as a resilience pattern and is implemented by Resilience4j.

15.12 Clean code, refactoring and code reviews

- **Naming:** reveal intent (`isEligibleForRefund`), avoid abbreviations. **Small functions** that do one thing at one level of abstraction. Few parameters. No hidden side effects. Prefer immutability. Comments explain *why*, not *what*.
- **Refactoring:** change structure without changing behaviour, in small steps, protected by tests. Typical moves: extract method/class, rename, replace conditional with polymorphism, introduce parameter object, remove duplication.
- **Anti-patterns:** God object, spaghetti code, copy-paste programming, golden hammer (one tool for everything), premature optimization, magic numbers, shotgun surgery (one change touches many classes), anemic domain model, **distributed monolith**, catching `Exception` and ignoring it, N+1 in loops, logging sensitive data.

- **Code review checklist:** correctness and edge cases; tests; readability and naming; error handling and logging; security (validation, authorization, secrets); performance risks (queries in loops, unbounded results); backward compatibility; transactions and concurrency; observability. Review the *design* first, nitpicks last; be kind and specific; automate style with linters/formatters so humans focus on logic.

Part 16 — System Design Interviews

16.1 A framework you can follow every time

1. **Clarify requirements.** *Functional* (what the system does) and *non-functional* (scale, latency, availability, consistency, durability, security, cost). Ask who the users are and what the peak load is.
2. **Estimate capacity** (rough math, state assumptions; do not pretend precision). Example: assume 100M new URLs/month → about 40 writes/sec average; reads 100× → ~4,000 reads/sec average, peaks higher. Storage = items × size × retention.
3. **Define APIs** (endpoints, request/response, idempotency, pagination).
4. **Model data** (entities, relationships, access patterns → choose the database and indexes).
5. **Draw the high-level design** (clients, gateway, services, DB, cache, queue).
6. **Deep dive** into the hard parts (hot spots, consistency, failure handling).
7. **Scale and harden:** caching, replication, partitioning/sharding, async processing, rate limiting, observability, security, cost.
8. **State trade-offs** explicitly.

Key concepts: * **Horizontal vs vertical scaling:** add more machines (needs stateless services, load balancing) vs a bigger machine (simple, has a ceiling and single point of failure). * **Load balancing:** round-robin, least connections, consistent hashing (for caches/shards). L4 vs L7. * **Caching layers:** browser/CDN → API gateway → application cache (Redis/Caffeine) → DB buffer cache. Invalidation is the hard part. * **Replication:** leader/follower for read scaling and availability; leader election for failover. **Partitioning/sharding:** pick a shard key with even distribution and good query locality (avoid hot shards); **consistent hashing** limits data movement when nodes change. * **CAP theorem:** during a network **P**artition, a distributed store must choose **C**onsistency (refuse/slow some requests) or **A**vailability (answer, possibly stale). It is *not* "pick 2 of 3" at all times. **PACELC** adds: otherwise (no partition), trade **L**atency vs **C**onsistency. Money movement usually favours consistency; feeds and view counters favour availability. * **Reliability:** redundancy, health checks, retries with backoff, timeouts, circuit breakers, graceful degradation, backups and tested restores, multi-AZ deployment, defined RPO/RTO. * **Security:** TLS, authentication/authorization, input validation, encryption at rest, least privilege, audit logs, rate limiting, secrets management. * **Cost/performance trade-offs:** caches and replicas cost money; async processing trades latency for throughput; over-engineering costs time.

16.2 Design: URL shortener

- **Requirements:** create short link, redirect fast, optional expiry/custom alias, analytics; read-heavy; high availability for redirects.
- **API:** `POST /api/v1/urls {longUrl, expiresAt?, alias?}` → `201 {shortCode}`; `GET /{code}` → `301/302` redirect (302 if you need to count every click; 301 can be cached by browsers).
- **ID generation:** (a) counter/sequence → Base62-encode (7 chars give $62^7 \approx 3.5$ trillion codes) — predictable unless shuffled; (b) random code + unique-constraint retry on collision; (c) hash of URL (collisions, same URL → same code). A distributed counter service or pre-allocated ID ranges per app instance avoids a single bottleneck.
- **Schema:** `urls(code PK, long_url, created_at, expires_at, owner_id)`; `clicks` in an append-only store/stream, aggregated asynchronously.

- **Architecture:**

```
Client -> CDN/LB -> Redirect Service --(cache hit)--> Redis
                         | miss                          ^
                         v                               |
                    KV/SQL store (code -> url) ---------+
Create Service -> ID generator -> store ;  click events -> Kafka -> analytics store
```

- **Bottlenecks:** hot links (cache + CDN), DB reads (cache-aside with TTL), writes (batch IDs). **Failure:** cache down → DB can take the load temporarily (protect with rate limits); ID service down → pre-fetched ranges. **Trade-offs:** 301 vs 302; SQL vs key-value store; predictable vs random codes (enumeration risk).
- **Follow-ups:** custom aliases, abuse/malware scanning, expiry cleanup, multi-region, analytics accuracy.

16.3 Design: E-commerce order processing

- **Services:** Catalog, Cart, Order, Inventory, Payment, Shipping, Notification; Gateway; Kafka.
- **API:** `POST /orders` (with `Idempotency-Key`), `GET /orders/{id}`, `POST /orders/{id}/cancel`.
- **Data:** `orders(id, user_id, status, total, created_at)`, `order_items`, `inventory(sku, available, reserved)`, `payments(id, order_id, status, provider_ref)`, `outbox`.
- **Flow (saga):** Order `PENDING` → reserve stock → charge payment → `CONFIRMED` → ship → notify. Failures compensate (release stock, refund, `CANCELLED`).
- **Overselling:** atomic conditional update `UPDATE inventory SET available = available - ? WHERE sku=? AND available >= ?`, or a reservation with TTL. Flash sales: queue requests, rate-limit, pre-warm caches, reserve in Redis then reconcile.
- **Bottlenecks:** hot SKUs, payment latency (async confirmation via webhook), search load (use a search index like Elasticsearch/OpenSearch fed by events). **Failure/consistency:** outbox + idempotent consumers; reconciliation jobs for payments; exactly-once *effect*, not exactly-once delivery.
- **Follow-ups:** partial shipments, returns, coupons, multi-currency, price changes between cart and checkout (snapshot prices on the order).

16.4 Design: Online appointment booking

- **Core invariant:** a doctor's time slot has at most one confirmed booking.
- **Schema:** `slots(id, doctor_id, start_time, end_time, status)`, `UNIQUE(doctor_id, start_time)`, `appointments(id, slot_id UNIQUE, patient_id, status, version)`.
- **Prevent double booking:** (1) unique constraint on `slot_id` in `appointments` — the database is the final judge; (2) conditional update `UPDATE slots SET status='BOOKED' WHERE id=? AND status='FREE'` (check affected rows); (3) optimistic lock `@Version`; (4) `SELECT ... FOR UPDATE` pessimistic lock for a short critical section. Return 409 to the loser.
- **Flow:** search slots (cached, short TTL) → hold slot (expires in N minutes) → payment → confirm → reminders via a scheduler/queue.
- **Time zones:** store UTC, display in the clinic's zone; recurring availability needs care with daylight-saving changes.
- **Scale:** read-heavy search (cache/search index), partition by doctor or clinic. **Failures:** abandoned holds (expiry job), notification failures (retry, never block booking).

16.5 Design: Payment processing

- **Principles:** correctness over speed; **idempotency keys**; an **immutable ledger** (double-entry: every movement is a pair of debit/credit entries); state machine `CREATED → AUTHORIZED → CAPTURED → REFUNDED/FAILED`.
- **Flow:** client sends a payment with an idempotency key → payment service stores intent → calls the provider (timeouts!) → handles the response or webhook → updates state + ledger in one transaction → emits an event via the outbox.
- **Uncertain outcomes:** on a timeout you do not know if the charge happened. Do **not** blindly retry with a new key; retry with the *same* idempotency key or query the provider, and run a **reconciliation** job comparing provider settlement reports with your ledger.
- **Security/compliance:** never store raw card data yourself — use a PCI-compliant provider's tokenization; encrypt, audit, least privilege. Money as `BigDecimal` or integer minor units, never `double`.
- **Failure scenarios:** duplicate webhooks (idempotent handler), webhook arrives before the API response, partial failures between ledger and event (outbox), provider downtime (circuit breaker + queue + "pending" state).

16.6 Design: Notification service

- **Requirements:** email/SMS/push, templates, user preferences, retries, rate limits, high volume, at-least-once with deduplication.
- **Architecture:** producers publish `NotificationRequested` events to Kafka → Notification Service validates preferences/quiet hours → renders the template → channel workers (email/SMS/push) call providers → status events/webhooks update delivery status.
- **Design points:** separate queues per channel and priority (OTP SMS must not wait behind marketing mail); retries with exponential backoff and jitter, dead-letter queue; **idempotency key** per notification to avoid duplicates; per-provider rate limits and failover to a backup provider; unsubscribe/consent handling; PII handling; scheduling.
- **Trade-offs:** at-least-once delivery means a rare duplicate; exactly-once to a human is not achievable, so make messages tolerant of duplicates.

16.7 Design: Rate limiter

- **Algorithms: fixed window** (simple, burst at window edges); **sliding window log** (accurate, memory-heavy); **sliding window counter** (approximation); **token bucket** (allows bursts up to bucket size at a steady refill rate — common default); **leaky bucket** (smooth output).
- **Where:** API gateway (per API key/IP/user), plus protective limits inside services.
- **Distributed implementation:** Redis with an atomic Lua script (or `INCR` + `EXPIRE`) keyed by client id and window. Return `429 Too Many Requests` with `Retry-After` and `X-RateLimit-*` headers.
- **Trade-offs:** local in-memory limiters are fast but inaccurate across many instances; Redis is accurate but is a dependency — decide fail-open vs fail-closed if Redis is down (usually fail-open for general APIs, fail-closed for sensitive operations).

```
// In-memory token bucket (single instance)
final class TokenBucket {
    private final long capacity; private final double refillPerSec; private double tokens; private long
last = System.nanoTime();
```

```
    TokenBucket(long capacity, double refillPerSec) { this.capacity = capacity; this.refillPerSec =
refillPerSec; this.tokens = capacity; }
    synchronized boolean tryAcquire() {
        long now = System.nanoTime();
        tokens = Math.min(capacity, tokens + (now - last) / 1e9 * refillPerSec);
        last = now;
        if (tokens >= 1) { tokens -= 1; return true; }
        return false;
    }
}
```

16.8 Design: File upload service

- **Large files:** don't pass bytes through your app servers. Issue a **pre-signed URL** so the client uploads directly to object storage (S3/GCS/Azure Blob); use **multipart/resumable uploads** for big files.
- **Flow:** client requests an upload → service creates a metadata record `UPLOADING` and returns the pre-signed URL → client uploads → storage event notifies the service → virus scan/validation/thumbnail pipeline → record `AVAILABLE`.
- **Data:** metadata in SQL (`files(id, owner, name, size, content_type, checksum, status, storage_key)`), bytes in object storage, access via signed download URLs/CDN.
- **Concerns:** size/type limits, content sniffing, malware scanning, checksum verification, private by default, cleanup of abandoned uploads (lifecycle rules), deduplication by hash, encryption at rest.

16.9 Design: Employee management system

- **Entities:** Employee, Department, Position, LeaveRequest, Attendance, Payroll; self-reference `manager_id` for the hierarchy (recursive CTE for the org chart).
- **API:** CRUD on `/employees` with filters/pagination, `/employees/{id}/leaves`, approval endpoints.
- **Security:** roles (EMPLOYEE, MANAGER, HR, ADMIN) plus ownership rules (managers only see their reports); field-level protection of salary/PII; audit logging of changes.
- **Scale:** modest traffic, so a well-built modular monolith with PostgreSQL is the right answer; add caching for org data and async jobs for payroll runs. Showing this restraint is itself a senior signal.

16.10 Follow-up questions a senior interviewer may ask

- How would you handle 10× traffic? What breaks first?
- How do you design for zero downtime deployments and schema changes?
- How do you migrate this to multiple regions? What consistency do you give up?
- How do you test and monitor this? What are your SLOs and alerts?
- What would you simplify if the team was three people?
- What is the data retention / privacy (GDPR-like deletion) story?

Part 17 — DevOps, Cloud and Production Support

17.1 Build tools and Git

Maven lifecycle: `validate → compile → test → package → verify → install → deploy` (running a phase runs all earlier ones). `mvn clean package` = delete `target/`, build the JAR. **Dependency management:** transitive dependencies, `dependencyManagement`/BOMs pin versions; scopes `compile`, `provided`, `runtime`, `test`; inspect with `mvn dependency:tree`; resolve conflicts with exclusions or BOMs ("nearest wins" rule). **Gradle:** Groovy/Kotlin DSL, incremental builds and build cache make it faster on big projects; `implementation` vs `api` configurations.

Git: * **merge** keeps history and creates a merge commit; **rebase** replays your commits on top of another branch for a linear history — never rebase commits others have already pulled. * **Conflict resolution:** `git status` → edit the conflict markers → `git add` → `git rebase --continue` / `git commit`. Understand both sides' intent; run tests after. * Branching models: trunk-based development with short-lived branches and feature flags, or GitFlow for scheduled releases. Useful commands: `git bisect` (find the commit that broke it), `git reflog` (recover lost commits), `git cherry-pick`, `git stash`, `git revert` (safe undo on shared branches).

17.2 Docker

```
# Multi-stage build: small final image, non-root user, layered jar
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw -q dependency:go-offline
COPY src src
RUN ./mvnw -q package -DskipTests && java -Djarmode=tools -jar target/app.jar extract --layers --
destination /layers

FROM eclipse-temurin:21-jre
RUN useradd -r -u 1001 appuser
WORKDIR /app
COPY --from=build /layers/dependencies/ ./
COPY --from=build /layers/spring-boot-loader/ ./
COPY --from=build /layers/snapshot-dependencies/ ./
COPY --from=build /layers/application/ ./
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

(Layer extraction syntax differs by Spring Boot version: Boot 3.3+ uses `-Djarmode=tools ... extract --layers`; earlier versions use `-Djarmode=layertools extract`. Check your version.) **Image vs container:** an image is the immutable template; a container is a running instance. Keep images small, pin base image versions, do not bake in secrets, run as non-root, and rely on `.dockerignore`.

17.3 Kubernetes fundamentals

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: order-service }
spec:
  replicas: 3
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
  selector: { matchLabels: { app: order-service } }
  template:
    metadata: { labels: { app: order-service } }
    spec:
      containers:
        - name: app
          image: registry.example.com/order-service:1.4.2
          ports: [{ containerPort: 8080 }]
          envFrom: [{ configMapRef: { name: order-config } }, { secretRef: { name: order-secrets } }]
          resources: { requests: { cpu: 250m, memory: 512Mi }, limits: { memory: 768Mi } }
          readinessProbe: { httpGet: { path: /actuator/health/readiness, port: 8080 },
initialDelaySeconds: 20 }
          livenessProbe:  { httpGet: { path: /actuator/health/liveness,  port: 8080 },
initialDelaySeconds: 40 }
      terminationGracePeriodSeconds: 45
```

Key ideas: readiness decides whether the pod receives traffic; liveness restarts a stuck pod (do not make liveness depend on downstream services, or an outage restarts everything); **requests** affect scheduling, **limits** cap usage (a memory limit exceeded causes `OOMKilled`); rolling updates replace pods gradually; a `Service` gives a stable endpoint; `HPA` scales on metrics.

17.4 CI/CD

A pipeline (Jenkins, GitHub Actions, GitLab CI, Azure DevOps): **checkout → build → unit tests → static analysis (Sonar, SpotBugs) → dependency/vulnerability scan → package → build and scan image → push to registry → deploy to dev/test → integration/smoke tests → approval → deploy to prod → post-deploy verification and automatic rollback**. Principles: build once, promote the same artifact through environments; configuration via environment, not rebuilds; every change goes through the pipeline; fast feedback (fail early); pipelines as code.

Jenkins basics: a `Jenkinsfile` (declarative pipeline) with `stages`, agents, credentials binding, shared libraries, webhooks.

Environment-specific configuration: Spring profiles, environment variables, ConfigMaps/Secrets, or a config server — **never** different builds per environment.

17.5 Deployment strategies

Strategy	How	Pros	Cons
Rolling	Replace instances gradually	No extra capacity; simple	Old and new versions run together (needs backward compatibility); slow rollback
Blue-green	Two environments; switch traffic	Instant switch and rollback	Double capacity; DB changes must be compatible
Canary	Send a small percentage to the new version, watch metrics, then ramp up	Limits blast radius	Needs good metrics and traffic routing

Zero-downtime changes: backward-compatible APIs; **expand/contract** database migrations (add the new column → deploy code that writes both → backfill → switch reads → drop the old column later); readiness probes; graceful shutdown (`server.shutdown=graceful`); feature flags to decouple deploy from release.

17.6 Logging, metrics, tracing

- **SLF4J + Logback:** program to the SLF4J API; config in `logback-spring.xml` ; use levels properly (ERROR = needs action, WARN = unexpected but handled, INFO = business milestones, DEBUG = diagnostics); parameterized messages; **MDC** for `traceId` , `userId` ; JSON logs for log pipelines; never log secrets or PII.
- **Metrics (Micrometer → Prometheus → Grafana):** counters, gauges, timers/histograms; JVM metrics, HTTP server metrics, `hikaricp_connections_*` , executor metrics, Kafka consumer lag. Alert on **symptoms** (error rate, latency percentiles, saturation) and use dashboards to find causes.
- **Tracing (OpenTelemetry):** spans, trace context propagation (`traceparent`), exporters to Jaeger/Tempo/Zipkin; in Boot 3 use Micrometer Tracing with an OTel bridge.
- **APM tools** (Datadog, New Relic, Dynatrace, Elastic APM, Grafana stack) combine these with profiling.

17.7 Cloud, secrets and TLS

- **Cloud fundamentals (AWS names; Azure/GCP have equivalents):** compute (EC2, ECS/EKS, Lambda), storage (S3), databases (RDS/Aurora, DynamoDB, ElastiCache), messaging (SQS/SNS, MSK), networking (VPC, subnets, security groups, load balancers, Route 53, CloudFront), identity (IAM — least privilege, roles instead of long-lived keys), observability (CloudWatch). Know **regions vs availability zones**, managed vs self-managed trade-offs, and the **shared responsibility model**.
- **Secrets:** Secrets Manager / Key Vault / Vault / Kubernetes Secrets (enable encryption at rest, restrict RBAC); inject at runtime; rotate (dual-valid-credentials window so rotation does not cause downtime); never commit; audit access.
- **TLS/HTTPS:** TLS gives encryption, server authentication (certificates signed by a CA) and integrity. Terminate at the load balancer/ingress or the app; use modern protocol versions; automate certificate renewal (cert-manager / Let's Encrypt / cloud certificate managers); **mTLS** for service-to-service.

Part 18 — Production Incident Playbooks

General method: **(1) Stabilize** — reduce user impact first (rollback, scale out, disable a feature, shed load). **(2) Gather evidence** — dashboards, logs, traces, recent deployments/config changes. **(3) Form a hypothesis, test it. (4) Fix, verify. (5) Post-incident review** — blameless, with root cause and prevention actions. Always ask first: "*What changed?*" (deployment, config, traffic, data volume, a dependency).

Each playbook below: **investigate → tools → root causes → immediate mitigation → long-term prevention.**

18.1 API suddenly slow

- **Investigate:** which endpoints? since when (deploy correlation)? p50 vs p99? Check traces to see which span grew (DB, downstream, CPU, GC).
- **Tools:** APM/traces, Grafana, slow query log, `jcmd Thread.print`, GC logs.
- **Causes:** slow/missing index or changed query plan, N+1 after a code change, downstream latency, lock contention, GC pauses, connection-pool wait, noisy neighbour, cache miss storm.
- **Mitigate:** rollback, scale out, enable cache/circuit breaker, kill a runaway query. **Prevent:** load tests, query-count assertions in tests, SLO alerts, timeouts.

18.2 High CPU

- **Investigate:** `top -H -p <pid>` to find hot threads → convert thread id to hex → match `nid` in `jstack`; sample several dumps; use a profiler (async-profiler/JFR) for flame graphs.
- **Causes:** infinite/busy loops, regex catastrophic backtracking, excessive serialization, frequent GC (then it is really a memory problem), hot lock spinning, too many threads.
- **Mitigate:** restart the instance (after collecting dumps), scale out, disable the feature. **Prevent:** profiling in staging, regex review, back-pressure.

18.3 OutOfMemoryError and memory leaks

- **Investigate:** which memory (heap, metaspace, native threads)? GC logs, heap histogram, heap dump (`-XX:+HeapDumpOnOutOfMemoryError`), analyse in Eclipse MAT, find the dominator and path to GC roots. Rising old-gen after each full GC = leak.
- **Causes:** unbounded caches/collections, loading full tables into memory, listener leaks, `ThreadLocal`, large payloads, too-small container limits (`OOMKilled` by the kernel is *different* from a Java OOM — check the pod's last state).
- **Mitigate:** restart/scale, raise the heap *temporarily*. **Prevent:** bounded caches, pagination/streaming, memory alerts, load/soak tests.

18.4 Database connection pool exhaustion

- **Symptoms:** `HikariPool-1 - Connection is not available, request timed out after 30000ms`; latency spikes; threads waiting on `HikariPool.getConnection`.

- **Investigate:** Hikari metrics (active, pending, timeouts), `leakDetectionThreshold` warnings, DB view of active sessions/long transactions (`pg_stat_activity`), thread dump showing what threads do while holding connections.
- **Causes:** slow queries; a transaction wrapping a remote HTTP call; connection leaks (manual JDBC without close); open-in-view; `REQUIRES_NEW` nesting needing two connections per request; pool too small versus concurrency; DB lock waits.
- **Mitigate:** kill blocked sessions, restart, temporarily raise the pool (careful — the DB has a limit). **Prevent:** shorter transactions, timeouts (`statement`, lock, transaction), pool metrics and alerts, fix queries.

18.5 Thread pool exhaustion

- **Symptoms:** requests queue or are rejected, CPU low, latency high. **Investigate:** thread dumps (all threads `WAITING`/`BLOCKED` in the same place), executor metrics, Tomcat `server.tomcat.threads.max` usage. **Causes:** blocking calls without timeouts to a slow dependency, unbounded queues hiding the problem, deadlock, too few threads. **Mitigate:** circuit-break the dependency, restart. **Prevent:** timeouts, bulkheads (separate pools per dependency), bounded queues with rejection policies, consider virtual threads for I/O-bound loads.

18.6 Excessive database queries

- **Investigate:** enable SQL logging in a lower environment, `datasource-proxy`, Hibernate statistics, count queries per request in tests. **Causes:** N+1, lazy loading in loops or serialization, repeated lookups of the same data. **Fix:** fetch joins/entity graphs/batch size/projections/caching. **Prevent:** query-count tests, code review checks, DTO projections for list endpoints.

18.7 Kafka consumer lag

- **Investigate:** lag per partition (`kafka-consumer-groups --describe`, metrics). Is one partition lagging (hot key/skew or a stuck "poison" message) or all (slow processing/insufficient consumers)? Check rebalance storms (`max.poll.interval.ms` exceeded because processing is too slow).
- **Causes:** slow downstream call/DB in the consumer, a poison pill retried endlessly, too few partitions/consumers, frequent rebalances, a producer burst.
- **Mitigate:** scale consumers up to the partition count, increase partitions (affects key ordering), pause or skip poison messages to a DLT, tune `max.poll.records`/`max.poll.interval.ms`. **Prevent:** DLT + retry topics, idempotent processing, lag alerts, capacity planning, avoid long blocking work in the poll loop.

18.8 Downstream service unavailable

- **Behaviour you want:** timeouts → limited retries (idempotent calls only, with backoff + jitter) → circuit breaker opens → fallback (cached/degraded response, queue for later) → clear error to the client (`503` with `Retry-After` or `202 Accepted` for async). **Investigate:** dependency dashboards, circuit-breaker state metrics, error budgets. **Prevent:** bulkheads, SLAs/contract tests, chaos testing, async decoupling where possible.

18.9 HTTP 500 errors after deployment

- **Investigate:** compare error logs before/after; first stack trace; check config/secrets/feature flags; DB migration applied? incompatible API/event contract change? missing environment variable; dependency versions; readiness gate too early. **Mitigate: roll back first**, then debug. **Prevent:** canary releases, smoke tests, backward-compatible migrations, automated rollback on error-rate increase, contract tests.

18.10 Deadlocks

- **Java deadlock:** `jstack` reports "Found one Java-level deadlock" with the lock cycle; fix with consistent lock ordering/ `tryLock`. **Database deadlock:** the DB log shows the victim and statements (`SHOW ENGINE INNODB STATUS`, PostgreSQL log with `log_lock_waits`). Fix with consistent update order, shorter transactions, proper indexes, retry the victim transaction.

18.11 Intermittent timeouts

- **Investigate:** is it correlated with a time pattern, a specific instance/pod/AZ, GC pauses, connection reuse issues (stale pooled connections killed by firewalls/load balancers → set `maxLifetime` below the idle timeout), DNS caching, cold starts, retries amplifying load, CPU throttling in containers (CFS limits), noisy neighbours, packet loss. **Tools:** traces with per-hop timings, `tcpdump`/connection metrics, per-instance dashboards, GC logs. **Prevent:** consistent timeouts budgeted across the call chain (the caller's timeout must exceed the callee's work, but the total must fit the user's patience), keepalive/health-checked pools, load tests.

18.12 "Works locally, fails in production"

Compare **config** (profiles, env vars, secrets), **data** (volume, nulls, encodings), **environment** (Java version, timezone, locale/charset, file paths, memory limits), **network** (firewalls, DNS, TLS certificates, proxies), **dependencies** (versions, availability), **concurrency** (only visible under load), **permissions** (IAM roles, file system). Use the Actuator `/env` and `/configprops` endpoints (secured), a debug log level toggled at runtime, and reproduce with production-like data in staging.

Part 19 — Behavioral and Project-Based Questions

> **Important:** every example below is a *hypothetical template* with placeholders such as `<YOUR PROJECT>`. Replace them with **your real experience**. Interviewers can tell when stories are invented, and follow-ups will expose them. Use the **STAR+** structure: **S**ituation, **T**ask, **A**ction (what *you* did, "I" not "we"), **R**esult (measure it if you truly can), plus **L**earning.

19.1 What interviewers evaluate

Theme	They want to see
Ownership	You drive outcomes beyond your ticket
Technical depth	You know *why*, not just *what*
Judgement	You weigh trade-offs and decide with incomplete data
Communication	You explain clearly to engineers and non-engineers
Leadership	You raise the team's level (reviews, mentoring)
Resilience	You stay calm in incidents and learn from failures

19.2 Questions, intent, and answer structure

1. "Explain your current project architecture." *Evaluates:* big-picture thinking, ability to communicate complexity, awareness of trade-offs. *Structure:* business purpose → users and scale → architecture diagram in words (clients, gateway, services, data stores, messaging) → your part → key decisions and why → known weaknesses and what you'd improve. See the full example in 19.3.

2. "What is your role and ownership?" *Evaluates:* scope and seniority. *Structure:* "I am responsible for `<modules/services>`. I own design, delivery, production support and mentoring for `<area>`. Examples of decisions I own: `<…>`. I collaborate with `<product, QA, DevOps>`."

3. "Describe a challenging production incident." *Evaluates:* composure, method, learning. *Template:* "In `<YOUR PROJECT>`, after a release we saw `<symptom, e.g., rising latency / 500s>`. I first checked what changed and the dashboards, mitigated by `<rollback/scale/feature flag>` to restore service, then used `<thread dump / heap dump / traces / slow query log>` to find the root cause: `<cause>`. We fixed it by `<fix>` and added `<alert/test/process change>` so it cannot recur. Impact: `<real, measured numbers only if you know them>`." Emphasize **mitigate first, root cause second, prevention third**, and a blameless review.

4. "How did you improve application performance?" *Evaluates:* measuring before optimizing. *Structure:* baseline metrics → profiling/tracing to find the bottleneck → hypothesis → change (query/index, caching, N+1

fix, async, pool sizing) → re-measure → guard against regression (tests, dashboards). Never claim an improvement you did not measure.

5. "How do you handle high traffic?" *Evaluates:* scalability knowledge. *Structure:* stateless services + horizontal scaling; caching (CDN, Redis); database (indexes, read replicas, partitioning); async processing/queues; back-pressure, rate limiting, timeouts, circuit breakers; autoscaling; load testing; observability. Add one concrete example from your experience if you have it.

6. "How do you review code?" *Evaluates:* quality mindset and collaboration. *Structure:* understand the purpose first; check design and correctness, tests, error handling, security, performance risks, readability; separate **must-fix** from **suggestions**; ask questions instead of commanding; praise good work; keep PRs small; automate formatting/static checks.

7. "How do you mentor junior developers?" *Evaluates:* leadership. *Structure:* pair programming on real tasks, explain the *why*, give progressively harder work, constructive reviews, create documentation/learning paths, encourage them to present, give space to make safe mistakes. Provide an example of someone you helped grow (real).

8. "How do you handle disagreements in design decisions?" *Evaluates:* maturity. *Structure:* seek to understand the other view; move from opinions to **criteria and data** (requirements, risks, cost, prototypes/spike); document the options (an ADR — Architecture Decision Record); decide, or escalate when needed; **disagree and commit**; revisit if evidence changes.

9. "Describe a difficult bug you solved." *Evaluates:* debugging method. *Template:* symptom → how you reproduced it → hypotheses eliminated → tools used → root cause (e.g., a race condition, a timezone issue, a lazy-loading problem, a connection leak) → fix → a test that would have caught it → what you learned.

10. "How do you make technical decisions?" *Evaluates:* structured thinking. *Structure:* clarify requirements and constraints → list options → compare on criteria (fit, complexity, team skills, operability, cost, risk, reversibility) → prototype risky parts → decide and record it → review later. Mention preferring reversible decisions and boring technology when it fits.

11. "How do you ensure code quality?" *Structure:* testing pyramid, code reviews, CI with static analysis and quality gates, coding standards, refactoring regularly, observability, definition of done, tracking and paying down technical debt deliberately.

12. "How do you approach a legacy application?" *Structure:* learn first (read, run, observe production behaviour); add **characterization tests** around critical flows; add logging/metrics; refactor in small steps; apply the **strangler fig** pattern for large migrations (route features gradually to new code); do not rewrite everything at once; upgrade dependencies incrementally.

13. "How do you handle tight deadlines?" *Structure:* clarify what is truly required (**scope negotiation**), prioritize by value and risk, break down work and expose risks early, deliver incrementally, keep quality gates for critical parts, communicate honestly with stakeholders, and record any shortcut as tracked tech debt. Avoid heroics as a strategy.

14. "What would you improve in your current system?" *Evaluates:* honest critical thinking without blaming. *Structure:* pick a real weakness (e.g., missing observability, slow release process, test coverage on critical flows, tight coupling) → why it matters → how you'd approach it incrementally → expected benefit → how you'd measure success.

15. "Why should we hire you as a senior developer?" *Structure:* a three-part pitch: (1) **technical depth** in Java/Spring and distributed systems backed by `<real achievements>`; (2) **ownership** — you deliver and run services in production; (3) **multiplier effect** — mentoring, reviews, raising standards. Tie it to the company's needs (research them first). Keep it under 90 seconds.

19.3 Example: explaining an end-to-end project architecture (template)

Replace every placeholder with your real project. The shape is what matters.

> "`<YOUR PROJECT>` is a `<type, e.g., appointment booking platform>` used by `<users>`. At its core we have `<N>` Spring Boot services behind an API gateway.
>
> **Request flow:** the client calls the gateway, which validates the JWT and routes to, for example, the Appointment Service. The controller validates the DTO, the service layer runs the use case inside a transaction, and the repository talks to `<PostgreSQL/MySQL>` through Spring Data JPA.
>
> **Database interaction:** each service owns its schema; migrations are managed with `<Flyway>`; we use `<optimistic locking / unique constraints>` to protect against concurrent updates; list endpoints use `<projections/pagination>` to avoid N+1.
>
> **Cross-service communication:** synchronous REST for queries with timeouts and circuit breakers; asynchronous events on `<Kafka/RabbitMQ>` for things like notifications, using the outbox pattern and idempotent consumers.
>
> **Exception handling:** a global `@RestControllerAdvice` returns a consistent error format with a correlation id; business errors map to 4xx, unexpected ones to 500 with details only in logs.
>
> **Security:** `<OAuth2/JWT>` with role and ownership checks; secrets from `<Vault/Kubernetes Secrets>`; HTTPS everywhere.
>
> **Deployment:** Docker images built in `<Jenkins/GitHub Actions>`, deployed to `<Kubernetes/ECS>` with rolling updates, readiness probes and graceful shutdown; config per environment.
>
> **Monitoring:** Micrometer metrics to `<Prometheus/Grafana>`, centralized JSON logs, distributed tracing, alerts on error rate and latency.
>
> **Challenges and what I'd improve:** `<a real challenge, e.g., double bookings under load, solved with …>`; next I'd like to `<improvement>`."

Practise this until it takes **3–4 minutes**, and be ready to zoom into any box on the diagram.

19.4 Answer-quality checklist

- Use "I" for what you personally did, "we" for team outcomes.
- Quantify only with numbers you actually know; otherwise describe impact qualitatively.
- Show trade-offs and what you would do differently.
- Keep stories 2–3 minutes; offer details if asked.

- Prepare 5–6 reusable stories: a production incident, a performance win, a conflict, a mentoring success, a failure you learned from, a large delivery under pressure.

Part 20 — 50+ Scenario-Based Questions

Each scenario lists: **investigation / possible causes → solution → alternatives & trade-offs → interview-ready line.** Practise answering in 60–90 seconds.

A. Concurrency and data consistency

1. Two users book the same appointment at the same time. How do you prevent double booking? *Causes:* check-then-insert race. *Solution:* enforce the invariant in the **database** — a unique constraint on `(doctor_id, slot_start)` or on `slot_id` in appointments; catch `DataIntegrityViolationException` and return 409. Or a conditional update `UPDATE slots SET status='BOOKED' WHERE id=? AND status='FREE'` and check the updated row count. *Alternatives:* optimistic locking with `@Version`; `SELECT … FOR UPDATE` (pessimistic) for hot slots; a temporary hold with TTL. *Trade-off:* constraints are simplest and safest; pessimistic locks reduce throughput. *Say:* "I never rely on application-level checks alone; the database constraint is the last line of defence."

2. A payment succeeds but the order is not updated. How do you handle it? *Causes:* crash/timeout between payment confirmation and DB update; lost webhook; dual-write. *Solution:* make payment confirmation **idempotent** and durable: store the payment result and an outbox event in the same transaction; process provider webhooks idempotently; run a **reconciliation job** comparing provider records and orders; keep orders in a `PAYMENT_PENDING` state until confirmed. *Alternatives:* orchestrated saga with a timeout and compensation (refund). *Say:* "Assume messages can be lost or duplicated, so state changes are idempotent and reconciled."

3. A Kafka message is processed twice. How do you prevent duplicate business effects? *Solution:* idempotent consumer — store a processed event id with a unique constraint in the same transaction as the effect; or use naturally idempotent operations (upsert, set state); enable producer idempotence. *Trade-off:* exactly-once delivery is not generally possible end to end; aim for exactly-once *effect*.

4. Two services need to update related business data. How do you maintain consistency? *Solution:* avoid distributed transactions; use a **saga** (orchestration or choreography) with compensations and the **outbox** pattern; design for eventual consistency with pending states. *Alternative:* merge the services if the data really must be consistent atomically (maybe they were split wrongly).

5. Inventory is oversold during a flash sale. *Causes:* read-modify-write race. *Solution:* atomic `UPDATE … SET available = available - ? WHERE available >= ?`; reserve stock with TTL; queue requests; Redis atomic decrement then asynchronous reconciliation. *Trade-off:* strict DB locking limits throughput; Redis introduces reconciliation complexity.

6. A counter in a multi-instance service gives wrong totals. *Causes:* in-memory `AtomicInteger` per instance; `count++` race. *Solution:* database atomic update, Redis `INCR`, or event aggregation. In a single JVM, use `AtomicLong` / `LongAdder`.

7. A scheduled job runs twice because the service has three instances. *Solution:* distributed lock (ShedLock with DB/Redis), leader election, or a single scheduler component/Kubernetes CronJob; make the job idempotent regardless.

8. Optimistic locking failures spike (`ObjectOptimisticLockingFailureException`). *Causes:* high contention on the same rows, long transactions, large aggregates. *Solution:* retry with backoff (a few times), reduce transaction time, split hot aggregates, or switch to atomic updates/pessimistic lock for the hot path. Return 409 with guidance to the client when retries fail.

9. A DB deadlock is reported in logs. *Investigate:* the DB deadlock report (which statements, locks). *Solution:* acquire locks/rows in a consistent order, shorten transactions, add indexes so fewer rows are locked, and retry the victim transaction.

10. The same request is sent twice (user double-clicks "Pay"). *Solution:* client disables the button; server requires an `Idempotency-Key`; unique `(user, key)` record returns the original response.

B. Spring and JPA

11. A Spring bean is not getting injected. What do you check? Is the class annotated and inside the component-scan path? Is it created with `new`? Multiple candidates (`NoUniqueBeanDefinitionException` → `@Primary` / `@Qualifier`)? Missing conditional (`@Profile`, `@ConditionalOnProperty`)? A circular dependency? Wrong `jakarta` vs `javax` annotation? Use `--debug`, Actuator `/beans`, and the exception message (it tells you the type and the injection point).

12. `@Transactional` is not rolling back. Why? Checked exception without `rollbackFor`; exception swallowed; self-invocation; non-public/final method; bean not proxied; work on another thread; an inner `REQUIRES_NEW` committed; `readOnly` misunderstanding. Verify with transaction logging (`TRACE` on `org.springframework.transaction.interceptor`).

13. A JPA endpoint triggers hundreds of SQL queries. N+1. Confirm with SQL logging/statistics; fix with `join fetch` / `@EntityGraph` / batch fetch size / DTO projection; verify with a query-count test; prevent with code review and `LAZY` defaults.

14. `LazyInitializationException` in production only. Different code path accessing a lazy association outside a transaction (JSON serialization of an entity). Fix: map to DTO inside the service transaction, fetch needed data, never return entities from controllers; do not rely on open-in-view.

15. An API returns duplicate records. Check: a join multiplying rows (collection fetch join without `distinct`), missing unique constraints, non-idempotent creation/retries, pagination over an unstable sort order, `equals/hashCode` issues in `Set`s, duplicate Kafka processing. Investigate with SQL from logs, then fix at the source (constraint + idempotency), not with a `distinct` band-aid.

16. A query becomes slow as data grows. Explain plan → missing/wrong index, function on the column, `OFFSET` pagination deep pages, non-selective index, outdated statistics, large sorts. Fix: composite index matching filter/sort, keyset pagination, covering index, archive old data, partition, cache. Measure before and after.

17. After an upgrade, entity IDs or queries behave differently (Hibernate 5 → 6). Read the migration guide: default ID generation strategy and sequences changed, HQL stricter, type mapping changes. Run integration tests against a real database; compare generated SQL.

18. Data saved but not visible to another request. Transaction not committed yet, wrong isolation, read replica lag, cache serving stale data, or `@Transactional(readOnly)` with a stale first-level cache in the same

session. Identify which read path is used.

19. Bulk update ran but the entity shows old values. The JPQL update bypasses the persistence context. Use `@Modifying(clearAutomatically = true)` or reload entities.

20. Startup fails with a circular dependency after upgrading Spring Boot. Boot 2.6+ disallows it by default. Refactor (extract a shared component/event), not just `allow-circular-references=true`.

C. API, security and design

21. How do you securely store and rotate credentials? Secret manager (Vault/AWS Secrets Manager) injected at runtime; no secrets in code or images; least-privilege IAM roles; rotation with an overlap window (two valid credentials); audit and alert on access; short-lived dynamic database credentials where possible; scan repositories for leaks.

22. A user can access another user's order by changing the id in the URL. IDOR / broken access control. Check ownership in the service layer (`order.getUserId().equals(currentUser)`), use `@PreAuthorize` with ownership expressions, filter queries by the user, write tests for forbidden access, return 404 or 403 consistently.

23. Your JWT was leaked. What now? Short expiry limits damage; revoke the refresh token/session family; denylist the `jti`; rotate signing keys if the key leaked (publish via JWKS, support `kid`); investigate how it leaked (logs, XSS); force re-authentication; add detection (anomalous IPs).

24. The mobile app is on version 1 and you must change an API. Versioning or backward-compatible additions; support old versions during a deprecation window; feature flags; contract tests; telemetry on version usage; never remove fields abruptly.

25. How do you design an API that handles large lists? Pagination (keyset for large, changing datasets), maximum page size, filtering and sorting on indexed fields, streaming/export for bulk (async job + download link), consistent ordering.

26. A downstream service is unavailable. How should your service behave? Timeout, limited retry with jittered backoff (idempotent calls), circuit breaker, fallback/cached data or degraded feature, queue for later processing where acceptable, clear error/`Retry-After`, alerts. Avoid blocking threads indefinitely.

27. A file upload endpoint causes memory spikes. Stream to storage instead of buffering whole files; use pre-signed direct uploads; limit sizes; multipart limits; reactive/streaming APIs; check thread usage.

28. CORS errors from the browser in production only. Allowed origins differ per environment; credentials with `*` origin not allowed; preflight `OPTIONS` blocked by security rules; gateway vs service both adding headers. Configure `CorsConfigurationSource` per environment and permit the preflight.

29. How do you handle a poison message that always fails in a Kafka consumer? Limited retries with backoff, then publish to a dead-letter topic with error context; alert; fix and replay. Don't block the partition forever.

30. You need to expose a public API to partners. What is your checklist? Authentication (OAuth2 client credentials/API keys), authorization scopes, rate limiting and quotas, input validation, versioning,

documentation (OpenAPI), idempotency, webhooks with signatures, monitoring per client, SLAs, sandbox environment.

D. Performance and production

31. An application works locally but fails in production. *See Part 18.12.* Compare config, data, environment, network, dependencies, concurrency, permissions.

32. A thread pool becomes exhausted. *See 18.5.* Thread dumps, blocking calls without timeouts, bulkheads, bounded queues.

33. CPU is 100% after a release. *18.2.* Hot thread → stack trace → recent change; roll back; profile.

34. Memory grows until the pod is `OOMKilled`. Check whether it is heap (Java OOM) or total container memory (heap + metaspace + direct buffers + thread stacks exceeding the limit). Set `MaxRAMPercentage` so the heap leaves headroom, analyse heap dump, check native memory (NMT), cap caches.

35. Latency is fine on average but terrible at p99. Look at GC pauses, lock contention, retries, cold caches, slow queries on specific inputs, queueing, noisy neighbours. Use histograms and traces for tail cases; averages hide them.

36. Response time degrades only at peak hours. Saturation (pool/thread/CPU), shared dependency limits, lock contention, cache stampede. Load test, autoscale, add back-pressure.

37. A cache stampede takes the database down. Many requests rebuild an expired hot key. Use per-key locks or request coalescing, jittered TTLs, refresh-ahead, serve-stale-while-revalidate.

38. Logs are huge and costly; yet you cannot find the problem. Structured JSON logs with correlation ids, proper levels, sample noisy logs, avoid logging payloads, central search, tracing for flow.

39. Batch job takes 6 hours and grows. Partition work (chunks/parallelism), keyset reads, batch inserts (`jdbc.batch_size`, sequence ids), avoid loading everything in memory, commit per chunk with restartability (Spring Batch), move heavy aggregates to the DB, schedule off-peak.

40. How do you deploy a change without downtime? Rolling/blue-green/canary, readiness probes, graceful shutdown, backward-compatible API and DB changes (expand/contract), feature flags, automated rollback.

E. Microservices and architecture

41. Service A calls B calls C; C is slow and everything times out. Cascading failure: timeouts budget at each hop, circuit breakers, bulkheads, async decoupling, cache, reduce chattiness.

42. How do you trace a failing request across 6 services? Correlation/trace id propagated by headers, central logs searchable by id, distributed tracing UI to see which span failed or was slow.

43. Data needed by service A lives in service B. Calls are chatty. Options: API composition at the gateway/BFF, **replicate needed data via events** (local read model), merge services if the boundary is wrong. Trade-off: eventual consistency vs coupling.

44. A shared database between services causes release problems. Move to database-per-service with APIs/events; transitional approaches: views, then ownership of tables by one service, then extraction.

45. You must migrate a monolith to microservices. Strangler fig: put a gateway in front, extract capabilities one by one (start with low-risk, high-value, clear boundaries), keep data ownership in mind, build CI/CD and observability first, avoid a big-bang rewrite.

46. Event schema must change but consumers are not ready. Backward/forward compatible changes (add optional fields), schema registry compatibility checks, version topics for breaking changes, consumers tolerant of unknown fields.

47. Messages arrive out of order. Partition by entity key to keep per-entity order; version numbers/timestamps in events so consumers ignore stale ones; design handlers to be commutative or use state machines.

48. How do you ensure a config change doesn't take the system down? Config as code with review, validation on startup (fail fast), gradual rollout, feature flags, ability to roll back quickly, audit trail.

49. Multi-region requirement appears. What changes? Data replication strategy and consistency trade-offs (active-passive vs active-active), conflict resolution, global load balancing/DNS failover, latency, data residency laws, cost and operational complexity — justify with RPO/RTO needs.

50. A third-party API has strict rate limits. Client-side throttling (token bucket), queue requests, cache responses, batch calls, exponential backoff on 429, monitor quota usage, fallback behavior.

F. Extra practice

51. Your Spring Boot app starts slowly in Kubernetes and gets killed. Startup probe/longer `initialDelay`, reduce classpath scanning/lazy init, check external dependency waits, CPU limits too low during startup, consider AOT/native or CDS.

52. Your scheduled report exports 2 million rows and crashes. Stream results (`Stream`/cursor with fetch size, `@QueryHints`), write to a file incrementally, run async and notify, never build the whole result in memory.

53. `NullPointerException` appears only occasionally in production. Look at the stack trace line, check for racing initialization, `Optional` misuse, unexpected nulls from external data/JSON, lazy fields set later; add validation at boundaries; reproduce with the exact input from logs/traces.

54. Time-related bug: reports differ by a day for users in another country. Store UTC `Instant`, convert at the edges with an explicit `ZoneId`, avoid `LocalDateTime` for absolute moments, beware DST; test with different zones.

55. A new engineer asks why you chose a modular monolith over microservices. Team size, deployment simplicity, domain boundaries not yet stable, operational cost; designed modules with clear interfaces so extraction later is possible.

Part 21 — Frequently Asked Comparison Questions

21.1 Java core

Topic	A	B	Use cases / pitfalls
JDK vs JRE vs JVM	JDK = tools + JRE	JRE = JVM + libraries; JVM = executes bytecode	Develop with the JDK; JVM is platform specific. Pitfall: "JVM is platform independent" is wrong
String / StringBuilder / StringBuffer	String immutable	Builder fast, not thread-safe; Buffer synchronized	Concatenation in loops → StringBuilder
ArrayList vs LinkedList	Array, O(1) get	Linked nodes, O(n) get	ArrayList almost always; ArrayDeque for queues
HashMap vs ConcurrentHashMap	Not thread-safe, allows null	Thread-safe per-operation, no null	Check-then-act still needs `putIfAbsent` / `compute`
HashMap vs TreeMap	O(1) average, unordered	O(log n), sorted, range queries	TreeMap when ordering/ranges matter
Comparable vs Comparator	Natural order inside the class	External, multiple orderings	Use `Integer.compare` not subtraction
map() vs flatMap()	One-to-one	One-to-many, flatten	`Optional.flatMap` avoids `Optional<Optional<T>>`
orElse() vs orElseGet()	Always evaluates the argument	Evaluates only if empty	`orElseGet` for expensive/side-effect defaults
Checked vs unchecked	Compiler-forced handling	`RuntimeException`, optional	Spring: unchecked by default; rollback differs
Abstract class vs interface	State + constructors, single inheritance	Contract, multiple, default methods	Interface for capabilities; abstract for shared code
synchronized vs volatile	Atomicity + visibility + lock	Visibility only	`volatile` for flags; `count++` needs atomics/locks
Runnable vs Callable	No result, no checked exception	Returns value, may throw	Callable with `ExecutorService.submit`
Future vs CompletableFuture	Blocking `get()`	Chaining, callbacks, combine	Provide your own executor for blocking work

Example for `orElse` vs `orElseGet`:

```
String v1 = opt.orElse(expensiveLookup());          // expensiveLookup() runs EVERY time
String v2 = opt.orElseGet(() -> expensiveLookup()); // runs only if empty
```

21.2 Spring, REST and JPA

Topic	A	B	Notes
Spring vs Spring Boot	Framework (IoC, MVC, TX)	Opinionated setup: auto-config, starters, embedded server, Actuator	Boot is built on Spring, not a replacement
@Component vs @Bean	Class-level, scanned	Method-level in `@Configuration`, for third-party/complex creation	Both end up as beans
@Controller vs @RestController	Returns views	`@Controller` + `@ResponseBody` → JSON	Use RestController for APIs
@RequestParam vs @PathVariable	Query string/filters	URI path identifies the resource	`/users/5?active=true`
@Bean vs @Autowired	*Defines* a bean	*Requests* a bean (injection point)	Opposite directions: provider vs consumer
@Qualifier vs @Primary	Explicit choice at the injection point	Default among candidates	Qualifier wins
Lazy vs eager loading	Load on access (needs session)	Load immediately	Default to LAZY; fetch per use case
save() vs saveAndFlush()	Schedules at flush/commit	Flushes immediately	Rarely need flush; useful before native queries or to see constraint errors early
JPA vs Hibernate	Specification	Implementation	Spring Data JPA adds repositories
JDBC vs JPA	Manual SQL, full control	ORM: mapping, caching, dirty checking	JDBC/jOOQ for complex SQL or bulk ops
findById vs getReferenceById	Query now, `Optional`	Lazy proxy, no query	Reference is handy to set FKs
@Valid vs @Validated	Jakarta, nested validation	Spring, groups + method validation	Controller param validation needs `@Validated` on the class
PUT vs PATCH vs POST	PUT replaces (idempotent)	PATCH partial; POST creates/actions	
Filter vs Interceptor	Servlet-level	Spring MVC-level	Security = filters

`@Transactional` propagation summary

Type	One-line behaviour
REQUIRED	Join or create
REQUIRES_NEW	Always a new independent transaction
SUPPORTS	Join if present, else none
NOT_SUPPORTED	Suspend, run without
MANDATORY	Must already exist
NEVER	Must not exist
NESTED	Savepoint within the current transaction

21.3 Architecture and tooling

Topic	A	B	Use cases / pitfalls
Monolith vs microservices	Simple deploy, local transactions	Independent scaling/teams, distributed complexity	Start modular; split when pain justifies it
REST vs messaging	Request/response, simple	Async, decoupled, resilient	REST for queries; events for workflows
Kafka vs RabbitMQ	Distributed log, replay, throughput, partition ordering	Smart broker, routing, per-message ack	Event streaming → Kafka; task queues/routing → RabbitMQ
SQL vs NoSQL	Schema, joins, ACID	Flexible schema, specific scaling/access patterns	Choose by access pattern and consistency needs
Optimistic vs pessimistic locking	Version check at write; retry on conflict	Lock rows up front	Low contention → optimistic; hot rows → pessimistic
Authentication vs authorization	Who are you? (401)	What can you do? (403)	
OAuth 2.0 vs JWT	Authorization protocol	Token format	OAuth tokens may be JWTs or opaque
Unit vs integration tests	One class, mocks, fast	Multiple components/real DB	Pyramid: many unit, fewer integration
Docker vs Kubernetes	Packages/runs containers	Orchestrates containers across machines	K8s needs images from Docker/OCI builds
gRPC vs REST	Binary, contracts, streaming	Text JSON, universal	gRPC internally; REST publicly
Orchestration vs choreography (sagas)	Central coordinator	Event reactions	Orchestration for complex flows
Rolling vs blue-green vs canary	Gradual replace	Instant switch	Canary for risk control
Process vs thread	Isolated memory	Shared heap in one process	Threads need synchronization
Heap vs stack	Objects, GC	Frames, locals per thread	SOE vs OOM
Fail-fast vs weakly consistent iterators	Throw CME on modification	Never throw; may miss/see updates	ConcurrentHashMap / CopyOnWrite

Part 22 — Final Revision Checklist and Rapid-Fire Questions

22.1 One-page memory aids

- **Java:** equals/hashCode contract; String pool; HashMap (bucket, hash, treeify, resize); immutability; Optional rules; streams are lazy; checked vs unchecked.
- **Concurrency:** race → atomics/locks; `volatile` is visibility only; deadlock = consistent order; bounded pools; `CompletableFuture` with your executor; virtual threads for blocking I/O.
- **JVM:** heap (young/old), stack, Metaspace; G1 default; leaks = reachable but unused; heap dump + MAT.
- **Spring:** constructor injection; bean lifecycle; proxies (self-invocation!); auto-config via conditions; profiles; `@ConfigurationProperties`.
- **JPA:** persistence context, dirty checking, LAZY by default, N+1 fixes, `@Version`, no entities in APIs, Flyway not `ddl-auto`.
- **Transactions:** service layer, short, rollback only on unchecked by default, propagation types, no remote calls inside.
- **Security:** BCrypt, JWT validation, short access + rotating refresh tokens, ownership checks, CSRF vs CORS.
- **Microservices:** timeouts + retries (jitter) + circuit breakers, saga + outbox + idempotent consumers, Kafka ordering by key, observability.
- **SQL:** joins, window functions, composite index leftmost prefix, `EXPLAIN`, `NOT EXISTS` over `NOT IN`.
- **System design:** requirements → estimates → API → data → architecture → deep dive → trade-offs.

22.2 Rapid-fire (answer each in one or two sentences)

1. Why override `hashCode` with `equals`? *Hash collections locate by hash first.*
2. Can a `HashMap` key be mutable? *Yes technically, but changing fields used in `hashCode` loses the entry.*
3. Why is `String` a bad choice for passwords? *Immutable and lingers in memory; `char[]` can be wiped.*
4. What does `Collections.unmodifiableList` do? *A read-only view; changes to the original show through; `List.copyOf` makes an immutable copy.*
5. Difference between `Stream.toList()` and `collect(toList())`? *The first (Java 16+) returns an unmodifiable list.*
6. What is a happens-before relationship? *A guarantee that one action's writes are visible to another (locks, volatile, thread start/join).*
7. When does `ConcurrentModificationException` occur? *Structural modification during fail-fast iteration, not only from multiple threads.*
8. What does `@SpringBootApplication` include? *`@Configuration`, `@EnableAutoConfiguration`, `@ComponentScan`.*
9. Default bean scope? *Singleton (per container).*
10. Why not use field injection? *Hidden dependencies, no immutability, harder tests.*
11. Does `@Transactional` work on private methods? *No — proxy-based.*
12. Does a checked exception roll back by default? *No.*
13. What is the persistence context? *The first-level cache and change tracker of an `EntityManager`.*

14. Difference between flush and commit? *Flush sends SQL; commit makes it permanent.*
15. Why do indexes slow writes? *Each write must update every index.*
16. `RANK` vs `DENSE_RANK`? *RANK leaves gaps after ties.*
17. What makes an API idempotent? *Repeating it has the same effect, e.g., PUT/DELETE or a POST with an idempotency key.*
18. 401 vs 403? *Unauthenticated vs authenticated but forbidden.*
19. What is eventual consistency? *Replicas/services converge after some delay if no new updates.*
20. Why is the outbox pattern needed? *To update the DB and publish an event atomically.*
21. At-least-once vs exactly-once? *Duplicates possible vs processed once; the latter usually needs idempotency.*
22. What is a bounded context? *A boundary where a domain model and language are consistent.*
23. Liveness vs readiness probe? *Restart me vs send me traffic.*
24. What is a canary release? *Roll out to a small share of traffic first and watch metrics.*
25. What does `-XX:MaxRAMPercentage` do? *Sizes the heap as a percentage of container memory.*

22.3 Mock interview routine (last two weeks)

1. Pick 5 questions daily from different parts; answer aloud against a timer (90 seconds).
2. Solve 2 coding problems daily without looking, then write the complexity and edge cases.
3. Once a week, do a full 45-minute system design on paper.
4. Review your own project: draw the architecture, list 3 incidents, 3 design decisions, 3 improvements.
5. Prepare questions for the interviewer (team structure, on-call, deployment frequency, tech debt, growth).

22.4 Final advice

Senior interviews reward **clarity, trade-offs and honesty**. If you do not know something, say what you *do* know, reason out loud, and explain how you would find out. Verify version-specific claims (Java, Spring Boot, Hibernate) against the official documentation before the interview, since those projects evolve quickly.

Good luck!